1 — Enums.py

```
# Enums
import enum

class UserStatus(enum.Enum):
    active = "active"
    deleted = "deleted"

class Privacy(enum.Enum):
    public = "public"
    private = "private"

class Theme(enum.Enum):
    light = "light"
    dark = "dark"

class EmailDigest(enum.Enum):
    daily = "daily"
    weekly = "weekly"
    monthly = "monthly"

class ResourceStatus(enum.Enum):
    active = "active"
    inactive = "inactive"

class SuggestionStatus(enum.Enum):
    pending = "pending"
    accepted = "accepted"
    rejected = "rejected"

class StartupStage(enum.Enum):
    idea = "idea"
    early = "early"
    growth = "growth"
    scale = "scale"

class JoinRequestStatus(enum.Enum):
    pending = "pending"
    approved = "approved"
    rejected = "rejected"

class StoryType(enum.Enum):
    image = "image"
    video = "video"

class PostType(enum.Enum):
    professional = "professional"
    social = "social"
    image = "image"
    video = "video"

class ConnectionStatus(enum.Enum):
    connected = "connected"
    pending = "pending"
    blocked = "blocked"
```

2 — ResourceDownload.py

```
from datetime import datetime
from app.extensions import db

class ResourceDownload(db.Model):
    __tablename__ = 'resource_downloads'
    id = db.Column(db.Integer, primary_key=True)
```

```
    resource_id = db.Column(db.Integer, db.ForeignKey('knowledge.id', ondelete='CASCADE'), nullable=Fals
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)

    downloaded_at = db.Column(db.DateTime, default=datetime.utcnow)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships - ALL CORRECTED
    # resource relationship defined in Knowledge model as 'knowledge_resource'
    # user relationship defined in User model as 'download_owner'

    # HELPER FUNCTIONS
    def get_time_since_download(self):
        """Get time since download was created"""
        return datetime.utcnow() - self.downloaded_at

    def is_recent(self, hours=24):
        """Check if download is recent"""
        time_diff = datetime.utcnow() - self.downloaded_at
        return time_diff.total_seconds() <= hours * 3600

    def get_resource_details(self):
        """Get resource details"""
        if self.knowledge_resource:
            return {
                'title': self.knowledge_resource.title,
                'category': self.knowledge_resource.category,
                'author': f"{self.knowledge_resource.author_first_name} {self.knowledge_resource.author_
                'file_url': self.knowledge_resource.file_url
            }
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'resource_id': self.resource_id,
            'user_id': self.user_id,
            'downloaded_at': self.downloaded_at.isoformat(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'time_since_download': str(self.get_time_since_download()),
            'is_recent': self.is_recent(),
            'resource': self.get_resource_details(),
            'user': {
                'id': self.download_owner.id,
                'firstName': self.download_owner.first_name,
                'lastName': self.download_owner.last_name,
                'email': self.download_owner.email
            } if self.download_owner else None
        }
```

3 — ResourceLike.py

```
from datetime import datetime
from app.extensions import db

class ResourceLike(db.Model):
    __tablename__ = 'resource_likes'

    id = db.Column(db.Integer, primary_key=True)
    resource_id = db.Column(db.Integer, db.ForeignKey('knowledge.id', ondelete='CASCADE'), nullable=Fals
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)

    liked_at = db.Column(db.DateTime, default=datetime.utcnow)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

```
    # Relationships - ALL CORRECTED
    # resource relationship defined in Knowledge model as 'knowledge_resource'
    # user relationship defined in User model as 'like_owner'

    # HELPER FUNCTIONS
    def get_time_since_like(self):
        """Get time since like was created"""
        return datetime.utcnow() - self.liked_at

    def is_recent(self, minutes=30):
        """Check if like is recent"""
        time_diff = datetime.utcnow() - self.liked_at
        return time_diff.total_seconds() <= minutes * 60

    def get_resource_details(self):
        """Get resource details"""
        if self.knowledge_resource:
            return {
                'title': self.knowledge_resource.title,
                'category': self.knowledge_resource.category,
                'author': f"{self.knowledge_resource.author_first_name} {self.knowledge_resource.author_
                'likes_count': self.knowledge_resource.likes
            }
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'resource_id': self.resource_id,
            'user_id': self.user_id,
            'liked_at': self.liked_at.isoformat(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'time_since_like': str(self.get_time_since_like()),
            'is_recent': self.is_recent(),
            'resource': self.get_resource_details(),
            'user': {
                'id': self.like_owner.id,
                'firstName': self.like_owner.first_name,
                'lastName': self.like_owner.last_name
            } if self.like_owner else None
        }
```

4 — ResourceView.py

```
from datetime import datetime
from app.extensions import db

class ResourceView(db.Model):
    __tablename__ = 'resource_views'

    id = db.Column(db.Integer, primary_key=True)
    resource_id = db.Column(db.Integer, db.ForeignKey('knowledge.id', ondelete='CASCADE'), nullable=Fals
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    viewed_at = db.Column(db.DateTime, default=datetime.utcnow)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    resource = db.relationship('Knowledge', backref='views_list', foreign_keys=[resource_id])
    user = db.relationship('User', backref='resource_views_list', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def get_time_since_view(self):
        """Get time since view was created"""
```

```
        return datetime.utcnow() - self.viewed_at

    def is_recent(self, hours=24):
        """Check if view is recent"""
        time_diff = datetime.utcnow() - self.viewed_at
        return time_diff.total_seconds() <= hours * 3600

    def get_resource_details(self):
        """Get resource details"""
        if self.resource:
            return {
                'title': self.resource.title,
                'category': self.resource.category,
                'author': f"{self.resource.author_first_name} {self.resource.author_last_name}",
                'views_count': self.resource.views
            }
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'resource_id': self.resource_id,
            'user_id': self.user_id,
            'viewed_at': self.viewed_at.isoformat(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'time_since_view': str(self.get_time_since_view()),
            'is_recent': self.is_recent(),
            'resource': self.get_resource_details(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None
        }
```

5 — __init__.py

```
from .user import User
from .userAchievement import UserAchievement
from .startup import Startup
from .startUpMember import StartupMember
from .growthMetric import GrowthMetric
from .achievement import Achievement
from .calendarEvent import CalendarEvent
from .chatConversation import ChatConversation
from .chatMessage import ChatMessage
from .goalMilstone import GoalMilestone
from .idea import Idea
from .ideaBookmark import IdeaBookmark
from .ideaComment import IdeaComment
from .joinRequest import JoinRequest
from .projectGoal import ProjectGoal
from .startupBookmark import StartupBookmark
from .task import Task
from .teamPerformance import TeamPerformance
from .knowledge import Knowledge
from .knowledgeBookmark import KnowledgeBookmark
from .knowledgeComment import KnowledgeComment
from .notification import Notification
from .post import Post
from .postLike import PostLike
from .postComment import PostComment
from .postMedia import PostMedia
from .teamMember import TeamMember
from .suggestion import Suggestion
```

```
from .storyView import StoryView
from .story import Story
from .ResourceView import ResourceView
from .ResourceLike import ResourceLike
from .ResourceDownload import ResourceDownload
from .refreshToken import RefreshToken
```

6 — achievement.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class Achievement(db.Model):
    __tablename__ = 'achievements'

    id = db.Column(db.Integer, primary_key=True)

    title = db.Column(db.String(255), nullable=False)
    description = db.Column(db.Text)
    icon = db.Column(db.String(100))  # icon name or emoji

    category = db.Column(Enum('task', 'social', 'learning', 'milestone'), default='task')
    points = db.Column(db.Integer, default=0)  # XP points

    requirement_type = db.Column(db.String(50))  # 'tasks_completed', 'ideas_created', etc.
    requirement_value = db.Column(db.Integer)  # threshold to unlock

    badge_color = db.Column(db.String(20))
    rarity = db.Column(Enum('common', 'rare', 'epic', 'legendary'), default='common')

    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Relationships
    # user_achievements = db.relationship('UserAchievement', backref='achievement', lazy='dynamic', casc

    # HELPER FUNCTIONS

    def get_unlocked_count(self):
        """Get count of users who unlocked this achievement"""
        return self.user_achievements.filter_by(is_completed=True).count()

    def get_progress_for_user(self, user_id):
        """Get progress of this achievement for a specific user"""
        user_achievement = self.user_achievements.filter_by(user_id=user_id).first()
        return user_achievement.progress_percentage if user_achievement else 0

    def is_unlocked_by_user(self, user_id):
        """Check if achievement is unlocked by user"""
        user_achievement = self.user_achievements.filter_by(user_id=user_id, is_completed=True).first()
        return user_achievement is not None

    def calculate_rarity_color(self):
        """Calculate badge color based on rarity"""
        colors = {
            'common': '#6B7280',
            'rare': '#3B82F6',
            'epic': '#8B5CF6',
            'legendary': '#F59E0B'
        }
        return colors.get(self.rarity, '#6B7280')

    def to_dict(self, user_id=None):
        data = {
            'id': self.id,
```

```
            'title': self.title,
            'description': self.description,
            'icon': self.icon,
            'category': self.category,
            'points': self.points,
            'requirement_type': self.requirement_type,
            'requirement_value': self.requirement_value,
            'badge_color': self.badge_color or self.calculate_rarity_color(),
            'rarity': self.rarity,
            'unlocked_count': self.get_unlocked_count(),
            'created_at': self.created_at.isoformat()
        }

        if user_id:
            data['user_progress'] = self.get_progress_for_user(user_id)
            data['is_unlocked'] = self.is_unlocked_by_user(user_id)

        return data
```

7 — calendarEvent.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class CalendarEvent(db.Model):
    __tablename__ = 'calendar_events'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id'), nullable=True)

    title = db.Column(db.String(255), nullable=False)
    description = db.Column(db.Text)

    start_date = db.Column(db.DateTime, nullable=False)
    end_date = db.Column(db.DateTime, nullable=True)
    all_day = db.Column(db.Boolean, default=False)

    category = db.Column(Enum('meeting', 'deadline', 'reminder', 'event'), default='event')
    color = db.Column(db.String(20))  # hex color for calendar display

    location = db.Column(db.String(255))
    is_recurring = db.Column(db.Boolean, default=False)
    recurrence_rule = db.Column(db.String(255))  # RRULE format

    reminder_minutes = db.Column(db.Integer, default=30)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # user = db.relationship('User', backref='calendar_events', foreign_keys=[user_id])
    # startup = db.relationship('Startup', backref='calendar_events', foreign_keys=[startup_id])

    # HELPER FUNCTIONS

    def update_dates(self, start_date, end_date=None, all_day=False):
        """Update event dates"""
        self.start_date = start_date
        self.end_date = end_date
        self.all_day = all_day
        db.session.commit()

    def get_duration(self):
```

```
        """Get event duration in minutes"""
        if not self.end_date:
            return 0
        duration = self.end_date - self.start_date
        return duration.total_seconds() / 60

    def is_past(self):
        """Check if event is in the past"""
        now = datetime.utcnow()
        return self.end_date and self.end_date < now if self.end_date else self.start_date < now

    def is_ongoing(self):
        """Check if event is currently ongoing"""
        now = datetime.utcnow()
        if self.end_date:
            return self.start_date <= now <= self.end_date
        return self.start_date.date() == now.date() if self.all_day else False

    def get_reminder_time(self):
        """Get reminder time"""
        return self.start_date - timedelta(minutes=self.reminder_minutes)

    def should_remind(self):
        """Check if reminder should be sent"""
        if self.reminder_minutes <= 0:
            return False

        reminder_time = self.get_reminder_time()
        now = datetime.utcnow()
        return reminder_time <= now <= self.start_date

    def get_category_color(self):
        """Get color based on category"""
        colors = {
            'meeting': '#3B82F6',    # blue
            'deadline': '#EF4444',    # red
            'reminder': '#F59E0B',    # amber
            'event': '#8B5CF6'        # violet
        }
        return self.color or colors.get(self.category, '#6B7280')

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'startup_id': self.startup_id,
            'title': self.title,
            'description': self.description,
            'start_date': self.start_date.isoformat(),
            'end_date': self.end_date.isoformat() if self.end_date else None,
            'all_day': self.all_day,
            'category': self.category,
            'color': self.get_category_color(),
            'location': self.location,
            'is_recurring': self.is_recurring,
            'recurrence_rule': self.recurrence_rule,
            'reminder_minutes': self.reminder_minutes,
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'duration_minutes': self.get_duration(),
            'is_past': self.is_past(),
            'is_ongoing': self.is_ongoing(),
            'reminder_time': self.get_reminder_time().isoformat() if self.reminder_minutes > 0 else None
            'should_remind': self.should_remind(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None,
            'startup': {
                'id': self.startup.id,
                'name': self.startup.name
```

```
            } if self.startup else None
        }
```

8 — chatConversation.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import JSON
from .chatMessage import ChatMessage

class ChatConversation(db.Model):
    __tablename__ = 'chat_conversations'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), nullable=True)  # For group chats
    conversation_type = db.Column(db.String(20), default='direct')  # direct, group
    created_by_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    # Group chat properties
    description = db.Column(db.Text, nullable=True)
    avatar_url = db.Column(db.String(500), nullable=True)

    # Settings
    is_active = db.Column(db.Boolean, default=True)
    settings = db.Column(JSON, default={})  # Notification settings, etc.

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # created_by = db.relationship('User', backref='created_conversations', foreign_keys=[created_by_id]
    # participants = db.relationship('User', secondary='conversation_participants', backref='conversatio
    messages = db.relationship('ChatMessage', backref='conversation', lazy='dynamic', cascade='all, dele

    # HELPER FUNCTIONS

    def add_participant(self, user, role='member'):
        """Add participant to conversation"""
        from sqlalchemy import insert

        # Check if user is already a participant
        existing = db.session.execute(
            conversation_participants.select().where(
                conversation_participants.c.conversation_id == self.id,
                conversation_participants.c.user_id == user.id
            )
        ).first()

        if not existing:
            stmt = conversation_participants.insert().values(
                conversation_id=self.id,
                user_id=user.id,
                role=role
            )
            db.session.execute(stmt)
            db.session.commit()

    def remove_participant(self, user):
        """Remove participant from conversation"""
        stmt = conversation_participants.delete().where(
            conversation_participants.c.conversation_id == self.id,
            conversation_participants.c.user_id == user.id
        )
        db.session.execute(stmt)
        db.session.commit()
    def get_participant_role(self, user):
```

```
        """Get participant role in conversation"""
        result = db.session.execute(
            conversation_participants.select().where(
                conversation_participants.c.conversation_id == self.id,
                conversation_participants.c.user_id == user.id
            )
        ).first()

        return result.role if result else None

    def get_messages_for_user(self, user, limit=50, offset=0):
        """Get messages for specific user with timezone conversion"""
        messages = self.messages.order_by(ChatMessage.created_at.desc())\
                                .limit(limit)\
                                .offset(offset)\
                                .all()

        # Convert messages for user's timezone
        converted_messages = []
        for message in reversed(messages):  # Reverse to get chronological order
            converted_messages.append(message.to_dict(for_user=user))

        return converted_messages

    def get_last_message_preview(self, for_user=None):
        """Get last message preview with timezone conversion"""
        last_message = self.messages.order_by(ChatMessage.created_at.desc()).first()
        if last_message:
            preview = {
                'content': last_message.get_content_for_user(for_user) if for_user else last_message.pre
                'created_at': last_message.created_at,
                'sender_name': f"{last_message.sender.first_name} {last_message.sender.last_name}",
                'display_time': last_message.get_display_time_info(for_user) if for_user else None
            }
            return preview
        return None

    def add_message(self, sender, content, message_type='text', metadata=None, reply_to_id=None):
        """Add new message to conversation with timezone handling"""
        message = ChatMessage(
            conversation_id=self.id,
            sender_id=sender.id,
            original_content=content,
            message_type=message_type,
            metadata=metadata or {},
            reply_to_id=reply_to_id,
            sender_timezone=sender.get_timezone()
        )

        db.session.add(message)
        self.updated_at = datetime.utcnow()
        db.session.commit()

        return message

    def is_user_participant(self, user_id):
        """Check if user is a participant in this conversation"""
        return any(participant.id == user_id for participant in self.participants)

    def get_unread_message_count(self, user_id):
        """Get count of unread messages for user"""
        # This would require a read_receipts table in a real implementation
        # For now, return total message count
        return self.messages.count()

    def to_dict(self, for_user=None):
        """Convert conversation to dictionary with user-specific timezone"""
        data = {
            'id': self.id,
            'name': self.name,
            'conversation_type': self.conversation_type,
            'description': self.description,
```

```
            'avatar_url': self.avatar_url,
            'is_active': self.is_active,
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'participants': [{
                'id': user.id,
                'firstName': user.first_name,
                'lastName': user.last_name,
                'profilePicture': user.profile_picture,
                'timezone': user.get_timezone()
            } for user in self.participants],
            'last_message': self.get_last_message_preview(for_user),
            'message_count': self.messages.count(),
            'unread_count': self.get_unread_message_count(for_user.id) if for_user else 0
        }

        return data

# Association table for conversation participants
conversation_participants = db.Table('conversation_participants',
    db.Column('conversation_id', db.Integer, db.ForeignKey('chat_conversations.id'), primary_key=True),
    db.Column('user_id', db.Integer, db.ForeignKey('users.id'), primary_key=True),
    db.Column('joined_at', db.DateTime, default=datetime.utcnow),
    db.Column('role', db.String(20), default='member')  # member, admin
)
```

9 — chatMessage.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import JSON

class ChatMessage(db.Model):
    __tablename__ = 'chat_messages'

    id = db.Column(db.Integer, primary_key=True)
    conversation_id = db.Column(db.Integer, db.ForeignKey('chat_conversations.id'), nullable=False)
    sender_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    # Store original message content without time conversion
    original_content = db.Column(db.Text, nullable=False)

    # Store sender's timezone for proper conversion
    sender_timezone = db.Column(db.String(50), default='UTC')

    # Message metadata
    message_type = db.Column(db.String(20), default='text')  # text, image, file, system
    metadata_data = db.Column(JSON, default={})  # For file URLs, image dimensions, etc.

    # Status tracking
    is_edited = db.Column(db.Boolean, default=False)
    edited_at = db.Column(db.DateTime, nullable=True)

    # Reply functionality
    reply_to_id = db.Column(db.Integer, db.ForeignKey('chat_messages.id'), nullable=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # sender = db.relationship('User', backref='sent_messages', foreign_keys=[sender_id])
    # conversation = db.relationship('ChatConversation', backref='messages', foreign_keys=[conversation_
    reply_to = db.relationship('ChatMessage', remote_side=[id], backref='replies', foreign_keys=[reply_t

    # HELPER FUNCTIONS
    def prepare_for_sending(self, sender_user=None):
```

```
        """Prepare message for sending by adding time placeholder"""
        from app.utils.timezone_converter import TimezoneConverter

        if sender_user:
            self.sender_timezone = sender_user.get_timezone()

        # Create time placeholder in sender's timezone
        time_placeholder = TimezoneConverter.create_time_placeholder(
            self.created_at,
            self.sender_timezone
        )

        # Combine placeholder with original content
        return f"{time_placeholder} {self.original_content}"

    def get_content_for_user(self, user):
        """Get message content converted for specific user's timezone"""
        from app.utils.timezone_converter import TimezoneConverter

        user_tz = user.get_timezone()

        # Convert the time placeholder to user's timezone
        converted_content = TimezoneConverter.replace_time_placeholder_for_user(
            self.prepare_for_sending(),
            self.created_at,
            user_tz,
            self.sender_timezone
        )

        return converted_content

    def to_dict(self, for_user=None):
        """Convert message to dictionary, with timezone conversion if user provided"""
        base_data = {
            'id': self.id,
            'conversation_id': self.conversation_id,
            'sender_id': self.sender_id,
            'original_content': self.original_content,
            'message_type': self.message_type,
            'metadata': self.metadata or {},
            'is_edited': self.is_edited,
            'edited_at': self.edited_at.isoformat() if self.edited_at else None,
            'reply_to_id': self.reply_to_id,
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'sender_timezone': self.sender_timezone
        }

        # If specific user provided, convert time for them
        if for_user:
            base_data['content'] = self.get_content_for_user(for_user)
            base_data['display_time'] = self.get_display_time_info(for_user)
        else:
            base_data['content'] = self.prepare_for_sending()

        # Include sender info if available
        if self.sender:
            base_data['sender'] = {
                'id': self.sender.id,
                'firstName': self.sender.first_name,
                'lastName': self.sender.last_name,
                'profilePicture': self.sender.profile_picture
            }

        # Include reply info if available
        if self.reply_to:
            base_data['reply_to'] = {
                'id': self.reply_to.id,
                'content': self.reply_to.original_content,
                'sender_id': self.reply_to.sender_id
            }
        return base_data
```

```
    def get_display_time_info(self, user):
        """Get comprehensive time display information for a user"""
        from app.utils.timezone_converter import TimezoneConverter
        user_tz = user.get_timezone()
        return TimezoneConverter.get_time_display_info(self.created_at, user_tz)

    def edit_message(self, new_content):
        """Edit message content"""
        self.original_content = new_content
        self.is_edited = True
        self.edited_at = datetime.utcnow()
        db.session.commit()

    def mark_as_read(self, user_id):
        """Mark message as read by user"""
        # This would require a read_receipts table in a real implementation
        # For now, this is a placeholder
        pass
```

10 — goalMilstone.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class GoalMilestone(db.Model):
    __tablename__ = 'goal_milestones'

    id = db.Column(db.Integer, primary_key=True)
    goal_id = db.Column(db.Integer, db.ForeignKey('project_goals.id'), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    title = db.Column(db.String(255), nullable=False)
    description = db.Column(db.Text)
    order = db.Column(db.Integer, default=0)

    is_completed = db.Column(db.Boolean, default=False)
    completed_date = db.Column(db.DateTime, nullable=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    goal = db.relationship('ProjectGoal', backref='milestones', foreign_keys=[goal_id])
    # user = db.relationship('User', backref='goal_milestones', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def complete(self):
        """Mark milestone as completed"""
        self.is_completed = True
        self.completed_date = datetime.utcnow()

        # Update parent goal progress
        if self.goal:
            self.goal.complete_milestone()

        db.session.commit()

    def uncomplete(self):
        """Mark milestone as not completed"""
        self.is_completed = False
        self.completed_date = None

        # Update parent goal progress
```

```
        if self.goal:
            self.goal.milestones_completed = max(0, self.goal.milestones_completed - 1)
            self.goal.update_progress()

        db.session.commit()

    def update_order(self, new_order):
        """Update milestone order"""
        self.order = new_order
        db.session.commit()

    def to_dict(self):
        return {
            'id': self.id,
            'goal_id': self.goal_id,
            'user_id': self.user_id,
            'title': self.title,
            'description': self.description,
            'order': self.order,
            'is_completed': self.is_completed,
            'completed_date': self.completed_date.isoformat() if self.completed_date else None,
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'goal_title': self.goal.title if self.goal else None
        }
```

11 — growthMetric.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class GrowthMetric(db.Model):
    __tablename__ = 'growth_metrics'

    id = db.Column(db.Integer, primary_key=True)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id'), nullable=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True)

    metric_type = db.Column(Enum('user_growth', 'revenue', 'market_share', 'overall'), nullable=False)

    current_value = db.Column(db.Float, default=0)
    previous_value = db.Column(db.Float, default=0)
    growth_percentage = db.Column(db.Float, default=0)  # 156%

    user_growth_percentage = db.Column(db.Float, default=0)  # +45%
    revenue_amount = db.Column(db.Float, default=0)  # $45k
    market_share_percentage = db.Column(db.Float, default=0)  # 12%

    period_start = db.Column(db.DateTime, nullable=False)
    period_end = db.Column(db.DateTime, nullable=False)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # Change these relationships to use string references:
    # parent_startup = db.relationship('Startup', backref='growth_metrics_startup', foreign_keys=[startu
    # metric_owner = db.relationship('User', backref='growth_metrics', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def calculate_growth_percentage(self):
        """Calculate growth percentage based on current and previous values"""
        if self.previous_value == 0:
```

```
        return 0
    return ((self.current_value - self.previous_value) / self.previous_value) * 100

def update_values(self, current_value, previous_value=None):
    """Update metric values and recalculate growth"""
    self.current_value = current_value

    if previous_value is not None:
        self.previous_value = previous_value

    self.growth_percentage = self.calculate_growth_percentage()

    # Update specific metric type values
    if self.metric_type == 'user_growth':
        self.user_growth_percentage = self.growth_percentage
    elif self.metric_type == 'revenue':
        self.revenue_amount = current_value
    elif self.metric_type == 'market_share':
        self.market_share_percentage = current_value

    db.session.commit()

def is_current_period(self):
    """Check if this metric is for the current period"""
    now = datetime.utcnow()
    return self.period_start <= now <= self.period_end

def get_period_duration(self):
    """Get the duration of the period in days"""
    return (self.period_end - self.period_start).days

def get_trend_direction(self):
    """Get trend direction (positive/negative/neutral)"""
    if self.growth_percentage > 0:
        return 'positive'
    elif self.growth_percentage < 0:
        return 'negative'
    else:
        return 'neutral'

def format_value(self):
    """Format value based on metric type"""
    if self.metric_type == 'revenue':
        return f"${self.current_value:,.0f}"
    elif self.metric_type in ['market_share', 'user_growth']:
        return f"{self.current_value:.1f}%"
    else:
        return f"{self.current_value:,.0f}"

def to_dict(self):
    return {
        'id': self.id,
        'startup_id': self.startup_id,
        'user_id': self.user_id,
        'metric_type': self.metric_type,
        'current_value': self.current_value,
        'previous_value': self.previous_value,
        'growth_percentage': self.growth_percentage,
        'user_growth_percentage': self.user_growth_percentage,
        'revenue_amount': self.revenue_amount,
        'market_share_percentage': self.market_share_percentage,
        'period_start': self.period_start.isoformat(),
        'period_end': self.period_end.isoformat(),
        'created_at': self.created_at.isoformat(),
        'updated_at': self.updated_at.isoformat(),
        'is_current_period': self.is_current_period(),
        'period_duration': self.get_period_duration(),
        'trend_direction': self.get_trend_direction(),
        'formatted_value': self.format_value(),
        'startup': {
            'id': self.startup.id,
            'name': self.startup.name
```

```
            } if self.startup else None,
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None
        }
```

12 — idea.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum, JSON
from .Enums import ResourceStatus
from .ideaComment import IdeaComment

class Idea(db.Model):
    __tablename__ = 'ideas'

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(255), nullable=False)
    description = db.Column(db.Text, nullable=False)
    project_details = db.Column(db.Text, nullable=False)
    industry = db.Column(db.String(100), nullable=False)
    stage = db.Column(db.String(100), nullable=False)
    tags = db.Column(JSON, default=[])

    creator_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    creator_first_name = db.Column(db.String(100))
    creator_last_name = db.Column(db.String(100))

    status = db.Column(Enum(ResourceStatus), default=ResourceStatus.active)
    likes = db.Column(db.Integer, default=0)
    views = db.Column(db.Integer, default=0)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships - ALL CORRECTED
    # creator relationship defined in User model as 'idea_creator'

    team_members = db.relationship('TeamMember',
        backref='parent_idea',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='TeamMember.idea_id')

    idea_comments = db.relationship('IdeaComment',
        back_populates='parent_idea',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='IdeaComment.idea_id')

    suggestions = db.relationship('Suggestion',
        backref='parent_idea',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='Suggestion.idea_id')

    idea_bookmarks = db.relationship('IdeaBookmark',
        backref='bookmarked_idea',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='IdeaBookmark.idea_id')

    # HELPER FUNCTIONS
```

```
    def increment_views(self):
        """Increment view count"""
        self.views += 1
        db.session.commit()

    def increment_likes(self):
        """Increment like count"""
        self.likes += 1
        db.session.commit()

    def decrement_likes(self):
        """Decrement like count"""
        if self.likes > 0:
            self.likes -= 1
        db.session.commit()

    def add_team_member(self, name, position, skills=None):
        """Add team member to idea"""
        from models.teamMember import TeamMember
        member = TeamMember(
            idea_id=self.id,
            name=name,
            position=position,
            skills=skills or {}
        )
        db.session.add(member)
        db.session.commit()
        return member

    def add_tag(self, tag):
        """Add tag to idea"""
        if not self.tags:
            self.tags = []
        if tag not in self.tags:
            self.tags.append(tag)
        db.session.commit()

    def remove_tag(self, tag):
        """Remove tag from idea"""
        if self.tags and tag in self.tags:
            self.tags.remove(tag)
        db.session.commit()

    def update_stage(self, new_stage):
        """Update idea stage"""
        self.stage = new_stage
        db.session.commit()

    def get_comments_count(self):
        """Get number of comments"""
        return self.comments.count()

    def get_team_size(self):
        """Get number of team members"""
        return self.team_members.count()

    def to_dict(self, include_comments=False, include_team=True):
        data = {
            'id': self.id,
            'title': self.title,
            'description': self.description,
            'projectDetails': self.project_details,
            'industry': self.industry,
            'stage': self.stage,
            'tags': self.tags or [],
            'creator': {
                'id': self.creator_id,
                'firstName': self.creator_first_name,
                'lastName': self.creator_last_name
            },
            'status': self.status.value,
            'likes': self.likes,
```

```
            'views': self.views,
            'commentsCount': self.get_comments_count(),
            'teamSize': self.get_team_size(),
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat()
        }

        if include_team:
            data['teamMembers'] = [{'name': tm.name, 'position': tm.position, 'skills': tm.skills}
                                   for tm in self.team_members.all()]

        if include_comments:
            data['comments'] = [comment.to_dict() for comment in self.comments.order_by(IdeaComment.crea

        return data
```

13 — ideaBookmark.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class IdeaBookmark(db.Model):
    __tablename__ = 'idea_bookmarks'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    idea_id = db.Column(db.Integer, db.ForeignKey('ideas.id', ondelete='CASCADE'), nullable=False)
    title = db.Column(db.String(255))
    content_preview = db.Column(db.Text)
    url = db.Column(db.String(500))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Relationships
    user = db.relationship('User', backref='idea_bookmarks_list', foreign_keys=[user_id])
    idea = db.relationship('Idea', backref='idea_bookmarks_list', foreign_keys=[idea_id])

    # HELPER FUNCTIONS

    def update_content(self, title=None, content_preview=None, url=None):
        """Update bookmark content"""
        if title:
            self.title = title
        if content_preview:
            self.content_preview = content_preview
        if url:
            self.url = url
        db.session.commit()

    def get_idea_details(self):
        """Get idea details"""
        if self.idea:
            return {
                'title': self.idea.title,
                'description': self.idea.description,
                'industry': self.idea.industry,
                'stage': self.idea.stage,
                'creator': f"{self.idea.creator_first_name} {self.idea.creator_last_name}",
                'likes': self.idea.likes,
                'views': self.idea.views
            }
        return None

    def is_recent(self, days=7):
        """Check if bookmark is recent"""
```

```
        time_diff = datetime.utcnow() - self.created_at
        return time_diff.days <= days

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'idea_id': self.idea_id,
            'title': self.title,
            'content_preview': self.content_preview,
            'url': self.url,
            'created_at': self.created_at.isoformat(),
            'idea': self.get_idea_details(),
            'is_recent': self.is_recent(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None
        }
```

14 — ideaComment.py

```
from datetime import datetime
from app.extensions import db

class IdeaComment(db.Model):
    __tablename__ = 'idea_comments'

    id = db.Column(db.Integer, primary_key=True)
    idea_id = db.Column(db.Integer, db.ForeignKey('ideas.id', ondelete='CASCADE'), nullable=False)
    content = db.Column(db.Text, nullable=False)

    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # comment_author = db.relationship('User', backref='idea_comments', foreign_keys=[author_id])
    parent_idea = db.relationship('Idea', back_populates='idea_comments', foreign_keys=[idea_id])

    # HELPER FUNCTIONS

    def update_content(self, new_content):
        """Update comment content"""
        self.content = new_content
        db.session.commit()

    def get_author_name(self):
        """Get author's full name"""
        return f"{self.author_first_name} {self.author_last_name}"

    def to_dict(self):
        return {
            'id': self.id,
            'ideaId': self.idea_id,
            'content': self.content,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'fullName': self.get_author_name()
            },
```

```
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat()
        }
```

15 — joinRequest.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum
from .Enums import JoinRequestStatus

class JoinRequest(db.Model):
    __tablename__ = 'join_requests'

    id = db.Column(db.Integer, primary_key=True)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id', ondelete='CASCADE'), nullable=False)
    startup_name = db.Column(db.String(255))
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    first_name = db.Column(db.String(100))
    last_name = db.Column(db.String(100))
    message = db.Column(db.Text)
    role = db.Column(db.String(100), default='member')
    status = db.Column(Enum(JoinRequestStatus), default=JoinRequestStatus.pending)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    startup = db.relationship('Startup', backref='join_requests_list', foreign_keys=[startup_id])
    user = db.relationship('User', backref='join_requests_list', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def approve(self, reviewer_id=None):
        """Approve join request"""
        self.status = JoinRequestStatus.approved
        self.updated_at = datetime.utcnow()

        # Add user to startup members
        from models.startUpMember import StartupMember
        member = StartupMember(
            startup_id=self.startup_id,
            user_id=self.user_id,
            first_name=self.first_name,
            last_name=self.last_name,
            role=self.role
        )
        db.session.add(member)

        # Update startup member count
        if self.startup:
            self.startup.update_member_count()

        db.session.commit()
        return member

    def reject(self, reviewer_id=None):
        """Reject join request"""
        self.status = JoinRequestStatus.rejected
        self.updated_at = datetime.utcnow()
        db.session.commit()

    def cancel(self):
        """Cancel join request (by user)"""
        self.status = JoinRequestStatus.cancelled
        self.updated_at = datetime.utcnow()
```

```
        db.session.commit()

    def is_pending(self):
        """Check if request is pending"""
        return self.status == JoinRequestStatus.pending

    def get_duration(self):
        """Get time since request was created"""
        return datetime.utcnow() - self.created_at

    def to_dict(self):
        return {
            'id': self.id,
            'startupId': self.startup_id,
            'startupName': self.startup_name,
            'userId': self.user_id,
            'firstName': self.first_name,
            'lastName': self.last_name,
            'message': self.message,
            'role': self.role,
            'status': self.status.value,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'isPending': self.is_pending(),
            'duration': str(self.get_duration()),
            'startup': {
                'id': self.startup.id,
                'name': self.startup.name,
                'industry': self.startup.industry
            } if self.startup else None,
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name,
                'email': self.user.email
            } if self.user else None
        }
```

16 — knowledge.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum, JSON
from .Enums import ResourceStatus
from .knowledgeComment import KnowledgeComment

class Knowledge(db.Model):
    __tablename__ = 'knowledge'

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(255), nullable=False)
    title_description = db.Column(db.Text, nullable=False)
    content_preview = db.Column(db.Text, nullable=False)
    category = db.Column(db.String(100), nullable=False)
    file_url = db.Column(db.String(500), nullable=True)
    tags = db.Column(JSON, default=[])

    # Author info
    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    status = db.Column(Enum(ResourceStatus), default=ResourceStatus.active)
    views = db.Column(db.Integer, default=0)
    downloads = db.Column(db.Integer, default=0)
    likes = db.Column(db.Integer, default=0)
```

```
# Image storage
image_buffer = db.Column(db.LargeBinary, nullable=True)
image_content_type = db.Column(db.String(100), nullable=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

# Relationships - ALL CORRECTED
# author relationship defined in User model as 'knowledge_author'

comments = db.relationship('KnowledgeComment',
    backref='knowledge_resource',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='KnowledgeComment.resource_id')

bookmarks = db.relationship('KnowledgeBookmark',
    backref='knowledge_resource',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='KnowledgeBookmark.knowledge_id')

resource_downloads = db.relationship('ResourceDownload',
    backref='knowledge_resource',
    lazy='dynamic',
    foreign_keys='ResourceDownload.resource_id',
    cascade='all, delete-orphan')

resource_likes = db.relationship('ResourceLike',
    backref='knowledge_resource',
    lazy='dynamic',
    foreign_keys='ResourceLike.resource_id',
    cascade='all, delete-orphan')

resource_views = db.relationship('ResourceView',
    backref='knowledge_resource',
    lazy='dynamic',
    foreign_keys='ResourceView.resource_id',
    cascade='all, delete-orphan')

# HELPER FUNCTIONS
def increment_views(self):
    """Increment view count"""
    self.views += 1
    db.session.commit()

def increment_downloads(self):
    """Increment download count"""
    self.downloads += 1
    db.session.commit()

def increment_likes(self):
    """Increment like count"""
    self.likes += 1
    db.session.commit()

def decrement_likes(self):
    """Decrement like count"""
    if self.likes > 0:
        self.likes -= 1
    db.session.commit()

def add_tag(self, tag):
    """Add tag to knowledge"""
    if not self.tags:
        self.tags = []
    if tag not in self.tags:
        self.tags.append(tag)
    db.session.commit()

def remove_tag(self, tag):
    """Remove tag from knowledge"""
```

```
        if self.tags and tag in self.tags:
            self.tags.remove(tag)
        db.session.commit()

    def update_status(self, new_status):
        """Update knowledge status"""
        self.status = ResourceStatus(new_status)
        db.session.commit()

    def get_comments_count(self):
        """Get number of comments"""
        return self.comments.count()

    def get_bookmarks_count(self):
        """Get number of bookmarks"""
        return self.bookmarks.count()

    def is_bookmarked_by_user(self, user_id):
        """Check if knowledge is bookmarked by user"""
        return self.bookmarks.filter_by(user_id=user_id).first() is not None

    def to_dict(self, include_comments=False, include_author_details=True):
        data = {
            'id': self.id,
            'title': self.title,
            'titleDescription': self.title_description,
            'contentPreview': self.content_preview,
            'category': self.category,
            'fileUrl': self.file_url,
            'tags': self.tags or [],
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name
            },
            'status': self.status.value,
            'views': self.views,
            'downloads': self.downloads,
            'likes': self.likes,
            'commentsCount': self.get_comments_count(),
            'bookmarksCount': self.get_bookmarks_count(),
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat()
        }

        if include_author_details and self.knowledge_author:
            data['author']['profilePicture'] = self.knowledge_author.profile_picture
            data['author']['company'] = self.knowledge_author.profile_company

        if include_comments:
            data['comments'] = [comment.to_dict() for comment in self.comments.order_by(KnowledgeComment

        return data
```

17 — knowledgeBookmark.py

```
from datetime import datetime
from app.extensions import db

class KnowledgeBookmark(db.Model):
    __tablename__ = 'knowledge_bookmarks'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    knowledge_id = db.Column(db.Integer, db.ForeignKey('knowledge.id', ondelete='CASCADE'), nullable=Fal
    title = db.Column(db.String(255))
```

```
    content_preview = db.Column(db.Text)
    url = db.Column(db.String(500))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Relationships
    user = db.relationship('User', backref='knowledge_bookmarks_list', foreign_keys=[user_id])
    knowledge = db.relationship('Knowledge', backref='knowledge_bookmarks_list', foreign_keys=[knowledge

    # HELPER FUNCTIONS

    def update_content(self, title=None, content_preview=None, url=None):
        """Update bookmark content"""
        if title:
            self.title = title
        if content_preview:
            self.content_preview = content_preview
        if url:
            self.url = url
        db.session.commit()

    def get_knowledge_details(self):
        """Get knowledge post details"""
        if self.knowledge:
            return {
                'title': self.knowledge.title,
                'category': self.knowledge.category,
                'author': f"{self.knowledge.author_first_name} {self.knowledge.author_last_name}",
                'views': self.knowledge.views,
                'likes': self.knowledge.likes
            }
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'knowledge_id': self.knowledge_id,
            'title': self.title,
            'content_preview': self.content_preview,
            'url': self.url,
            'created_at': self.created_at.isoformat(),
            'knowledge': self.get_knowledge_details(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None
        }
```

18 — knowledgeComment.py

```
from datetime import datetime
from app.extensions import db

class KnowledgeComment(db.Model):
    __tablename__ = 'knowledge_comments'

    id = db.Column(db.Integer, primary_key=True)
    resource_id = db.Column(db.Integer, db.ForeignKey('knowledge.id', ondelete='CASCADE'), nullable=Fals
    content = db.Column(db.Text, nullable=False)

    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    knowledge = db.relationship('Knowledge', backref='knowledge_comments_list', foreign_keys=[resource_i
    author = db.relationship('User', backref='knowledge_comments_list', foreign_keys=[author_id])

    # HELPER FUNCTIONS

    def update_content(self, new_content):
        """Update comment content"""
        self.content = new_content
        db.session.commit()

    def get_author_name(self):
        """Get author's full name"""
        return f"{self.author_first_name} {self.author_last_name}"

    def get_knowledge_title(self):
        """Get knowledge post title"""
        return self.knowledge.title if self.knowledge else None

    def to_dict(self):
        return {
            'id': self.id,
            'resourceId': self.resource_id,
            'content': self.content,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'fullName': self.get_author_name(),
                'profilePicture': self.author.profile_picture if self.author else None
            },
            'knowledgeTitle': self.get_knowledge_title(),
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat()
        }
```

19 — models.zip

```
PK■■■■■■■■■<q[■■■■■■■■■■■■■■■__init__.pyPK■■■■■■■■■s[■W|N■■■■■■■■■■■achievement.pyU[o6■~■@$■■
■■■l∆T;■!)%^&■■%KR■■(`{([Z*C■■y■■■S■■■{Ao■■96vW5e'u■■*e■gg■h9|■*3■■7g■,3t+%d■Y:■s■FW■■■/j■'<3■■
:5T~Bw;■gcqgr>'cD*■■F}'U■b;K■&2YDK■o■■}/■md■■CA■a^#WKA|6jAgX■|lrL[?■U■]gL■■sx■0,■■DE31u^K■#IS■■1-w
L*b■■■■■■GC■■HJtVHYG■■■
Ð0A+■]■■gJω!qW■■■%■)}t'
Hzt■p■c■S,@■Sm■■■Mc|■K@■d"5F`%
y:■■■`A'■7S■H?K[k■^Cvr:O17$|q2Y■■nVYwW/Fr7ìvf■■~[&'■y)=,N■Gedsf■1■NDH]■n■b1DDk<■!■ml,■:`■7i@Q■5■■■
=ycM9>Z■`■(R■■w■Ta
[txg■L■d#lu|t|k]$■X■05.V■><>B2W■K2@30x9a■■%\sZ■l@LN■}■■`RsT■DvM■p■d~xdrFdXEQmqBUWt■■[■E!■a■!
 ;■2?DGX]2■■=■#l]R■O8■qEUtRε
■1■■l3Q■■P■V06s■s■■Bh■■nD■;ke{■DNKd& '5■■2Q■■]b■j]?■9Nw8(■tX■4■*3]■%L■g5■j!d'&P■(I|YYzi=SDS■_|{Az
■&9v_S4■,eIx■■}l%■■%?f8■"■■"k■■dG■C@C/t&uRx■
LBY!■
:f■RYxh_I1Pm■■B]#&G.
4-H($}bL■G■■k8p■■)GufGJ~D[-■t■■■
>■l%t■ACYAvL■2`■v■■XXLz■tLnyE■5J,he2z5&TY■x■a■A2|1[■$■■■■■C■■Pn&U■c■c4%■x{s■d.■ ■7A/o■G■(■p3?■■r
w; u■/SI1M■■n■[JE■fv■Hg■■■■■.!@;B"0/-S5Lq&■S=■ky:■Sv■■$.WZ1■■5Cl'&Q■!>?mzFS■■■■s7s■W|■■Pig■■s5~I
Am■}3+■d■lKRK■++■■Rd}v2x■■■"i■"■■EKM■O■■G4LjM5■S
■3<■beTS■X+5P:-h■_(■{ ■Q■x08{■■|s■mEP+■io~
)KI:■Sm"3=r■Xi7*V|■d|0IVdC/m■kE60oS■■
yCk■■■. ■0[BHH■wVek9A■%■■■{|■l■D?LgB67N■ /?W■//■
w■EhBr"+"!dI■
■~NNN^■E■"'"@■■_^)'N■■[■d■%■■so\a2X■
J@7`y■■'zXw7L■■ioM■L(k■■&;■■■beunT■^-t■y,E
s{^■■■■-STg■■?■E;rQUç■{*qf"■■■■m■&gQhm■■c?■_■c■wLgy_3■P&■LA■?CDt+k■Lj■z■LT0ClMMe>3!■%■AwnaXMt 8■~
Y;■■3■E=■■x6E/~Iq[,■1■|Rc}
```

5va=■:c■■(!8-Dh_■=■rW■■GDYs;ded■E)v0D:')■XM
■■■`■~?^uA5`■gw.■0v'■Y;,■".<gnqA
8p+N#■)"■■K<■Y■1ÉPnaTK,'
T■7<■C■■Lq{r■]■y ■fL■Rp■tO■3°!|.dC■s((N!■■■q[5ub■pU&afM■V■■&■■'■G`■=\■■=Ug!■>0g;xo|8■xMK■kEL■■
vTL8■MNm■1}■l■
&■b-72\w-■k%■■■7■:4■BF■U■■P■l3A■0Zh9gT■■7■k■■y■o~■
■■
dG*L nPA■crl■■^@■■k■f■Qi■■!g■KC'W■,0>V~$J_*G5kW■E■xcr{ib^<E5■■q#■=V+;■5vy■gy2wr|■■■■■!■■R■tg["B■■N
-}41 %■;6es■jp;}A■~_■]■;■■Z</H
■.P■■wo■}■@n4hZ■ 6)■■BGuIW8.8kV#6P7■oKTw■b%$n■jsEConO■y

Jn7$:■■"■H■?■■5■■■■■\:(Jg■3■==w>EC5■l■■'i
|XV)■f#Z■■?8QI■AnA■8OY#■h■AM■■%q#;@<h■■3k■,[U6 ue■#LyB@#e=Kn■BU%+V■H■.d!%■'■j2■d£)JH■j|0s`N,U<g;0■
■gvP■rxR■@=■ c■■"{U■l0RQL■F03<■)■/^■p&bJ}h■■■■.■xqMK■■■5■■g~B:A8j=■■■■mb■YA'p■■4<JH■]■N0ZB,■wScp
e.
■Fjt6Q}<}iJA■GN_yI/jy&O■<h■■ M■.d5Otcz%■L■■MR1?j'
(54■■*)kXw89jGWec■■@y7Q■■T7v■■■~T■■*}■oH[O>■)%\>■w■0c+cM6fw~■6]%+I!NH^#v■d;"cw■ {■v■)^■)`g■kK?■■
■■ "Q?■+X"■-■■FLOnC ■+!■■Bcg*K6T■■PK■■■■■■■■■■r[■■j■■■W■■■■■■■Enums.pyMN0■r(l`■ nX■u■■R■1■Q■■
uJ■^nC■Z]5:<=■■`#■O*,■0z■■Y■K■
V■H■v6â■j■_!■■1VO^wdt■$7E:■3,_vNl■■I|}■■H■:xJE{■'p■?76■w■s■■PK■■■■■■■■■ds[■■f■■■■■■■■■■■goalMils
/)0\B■V■■/#■#5B3rZp]0■+■■■)■k■h^p■■■b`■G SqSG■R■\■o-Ak/Q■E!+A*p|9■■R)]yk■A■■
\S5;■■Dd■`■8Y■{FWgm[4Z■e■?VNZe?■U[■■■umCt■■■RNrvC[■-■Q8[■(fQO■*fg+#>cA!m(_.onK■Kk_■■■\aF~Ke'■A4■|V
■"}^■\M%■v■■JW■]■wz■R<IT!@■?ÄD8i%'Z{S8@;
Qm;T;
Q#0$D■Bt■k&!/i■■-■<VMWbx/1@N■[■■{Kx+9z
(■w_■r■PK■■■■■■■■■hs[■*■■■■■■■■■■■■growthMetric.pyX[o6■~■@$-lo■■x%]6■■■F:■VJg■U")i■:<■Hi]l/■.
;D■■7-9<■b9pRUs$3(86CcU[C4■CH]A*ba■Ga■gg■H|■*\B■vR*u6ZdBA■■59j()1%[■ni■3`■S6■}US OM'■■$jB■m■■Xoow
xeDWf2G■
P ■■T]■@qQx■p|oiK)T<yE■a"WEH■r■|4Fx%u^■2w■
■x<■ctrk7\'1en+]WoO8e~i■'?{■4D5Z]■3P■■u
B-K■6t■U■~_■sTW■>b[■■OP`.dC■■IF■■B
■dr=■s■@l■■■■^■■oSQ+
nm[■vA[■nf■9■dv■■b■eDoHXBK■P-■■T■■■■z■■N■,W
S>>m■FZ]■■■SF■■PH:zu^5B■#■YPh■x■■Pf1s■5\■J-"]axJlw=■x6■(\■■s7`;7n■1■.■4À■EZ%S0d■X3Dt+o■■63M■$Dq■

bf[7W
|B
SX=ZH5;*X\]k("rs spg■EiDdD■■■■■TG■,■ÞuY'$■CC■lh!■■P■eD■■q<W\SDx■■;■■6b?■■<x<v~i$■z■h■■■■S>1w0■/■
^u8]#]WD■■■■■■■V"s#■■P■■\■■x8JOe■jJQ}■Hc■lLRi$P■&2■■■&RQ/l]■■PK■■■■■■■■■s[
■■■■■■■■■■■■idea.pyXKo6■■■■■$+$■
,hvE■CÆ■]J
;$EQe;Y■■~$K■
■-!■■L■SPI+2-■■?@q■.'■■7
KoZ-H^■Z:■Z■9e■r(gDJs■■■/■■d"■■■)!■4■k2D■xs/\W
V &Z■M■N■[■4UV■■IP■i■a
k■■;Pk\e■(B|■VE-tR.■■OGzv/IT:]■"■~■U'l $CK■
2■=F■0CoN,qlY■{>9■}z?4/6O■■■!■■=yR;■&■,|O■■■"■Z6Y■J(■■|+Xt■_■9■■?
X■M■■■■,■■■0e■■■R.!lc■i<?FW+FiK1\J■w/(biP7B|■/o7.■S■[■b2■■tZ■NNN■rF])<■
9
<■ ,Jt■M■Yv■n■■■■■eW_■~■~F::5gt~Kty:[&|W00sQ
,>P<i■LR■j~x■■[laFjq■-&}$5mikb+"xy■h■+#8■■■jQ56F1■■1k*lr9=4{00:■"■;0■■■L
M!■4■Os;26zg{■XpoXv-njxCx]V272=■op2■■Q1$QgH
■■2s■\■9vwV■■owC■_{|G9■■Ag■/■7l■C■:WCf■■■■■!myg21Ö95nCf#U■■o■>■■]■'^■a■c4!Y=■9l■ss.H`N■_V[JYb6JK
]Q■c6}v■'GM■Z■PK■■■■■■■■■s[h.■■■■■■■■■■■ideaBookmark.pyV]o0■}`R&■i"■`b■X'4YnrYub;@5s|;■<■9&2OI":PC
P■91■f■r■l■iu■■■)g■■■■kQ&■>■)i[|1b<"xQN@R$
■Uy■■Ef&Ma■rN#D■p
YQ(■n:'8M■■W■■■sgX$h■fyy
fs■Bjk&T■9■94■■■-y9?U(■G■?9r■■■)■■'■5■S]■■V+1$■:■(tf■"w 6xG■■+;■■E■■I
4■'%*^■_h#■■_/7_■)RB!V
*■■b3iUe■otk2h.v■■■2~\&■8er,■ ■a{■(50Er=qlX■■1h■<:|■]6¸■%A■2#O}■KSp■■=■T$h■x■■JS■6■f~I■%S4Rij3■■■
■]T{P=iT■8]c■F)■ll5■W9■■■!VIA#■■0■Â■qLzv<.■9s■.;■W3■■■PK■■■■■■■■■■s[zR■■■■■■■■■■■ideaComment.pyT
s■■■■v*■<A■5■?nUI■■■■gW■ipBj■b:lc[wii■q;>{|^|x"z]}_oQ(u~■kR4
_?^hW■'1■,K■p9■■m■Rq3■}aG0zT■■&$Es■y%F) ■3\■■K_F■#i■Uxo■7■LQd■■[u$7:■Y_■■■MnXc^v;[(;■f7+■■■5■Iu■
7 Fb■■b!&■■_Ip■YR■}S[,B■Mt■&a■|jNJ■■■kz■l■O}^q {■■q<DBh*4Z6■hDTSN1■J:p■■Y8t#??■p!■■■BK■■Gk%"■},■v
■■■■k■■
■n■■JtB■■■ElS■S`Q^5Lz■■>■■M8u■$■■■p■
rg2■lY&1■l■kc■pG`DxR■v
7 "sWd*i■y■■| oHb■2+■n■w y■6#E{M■m■zi■/$" N}■.■■■■tAfQ■TM#■O4■aWdU?lt#:X:-
Cno-D■OdKT■■ftuZ■■■Q■J■û■■:1■jX■^■#■WR■XI■ELdE■Fgx] uG■yJCf■`g■j|B■9■■(1■<]=/PK■■■■■■■■Ds[Z■.■■
V■■■Rw/JL:5pE■W-■a■T{72■IL■%■Ô&Q^d■b■gk/9■(■&S■/_D2M6■8 ■U/■!LF9■■jI+"■k@b■j■!JK■/2
c■aj■
PF?*3\ 'Z= Bg99[RH6<■#![5■0XF■azM)z;!#KaH;■9■IH[■Q UJ^ƒJ3■■G1`=■aU■#i■8■■{■■■']■■)`t■O■v^-F■l■%P■

?Fb■■■■n"M+w%0>.@}■:9kx*■■%`a■eG$F:z*4Z■f;9Tufni■X■
gwL■'CU+6■■Nj4■L■0Njs.■■YAsbdM■Fw■■M(SI■l■c`■;■6o-kZ,■i1■■9Bkw■■■{Stn■■■■d^?(■m■46nEsb■YNg}| ■■P
i8■u"i~V)m`{G■=■V■I■3 <■te+eA■iAd:■f■Y@r■J■3d■■Ce;■e@)Itl(tWsLTAO
(2J■■Kk■=)■■■0■hFi■y■yfœhINB■/■WbId!k9a45Pb[■~{<■/j^aaii'gE■iF6■?+o■CHY!l■eU=r■■■Yg'N*-Σ■lvl■■!wPE
tY■o■z■Q■■"f2■im■xZ1*■_■F<v■■$■TFn
2`
%`n■S!■■00■}■sB■V■■QM■Z`■-0■■]t.■=
vP44+!■+Ig/■Z)Rw7 xA■Y■*!G■■■■t^E&|oG2■■Z^T■
~Qs<G■?:$p■■Gjv(g{■■■wS■>4#■■RB^■Nt<■T■q.■`;{■■■Z■KT"~■PK■■■■■■■■■s[i■■■■ ■■■■■■■knowledgeComm
%K■■+■■■■■m$V7■■V,Ay■■|:!(5■HK$$z2h■E_#■$Ka■■;PKR)V■{Q5,■■Z*■T}-■■<Z■)0■H*Z,9■W&0a
^■E`XkP■;U,■
+?■nlqk;■EN■s_■Hkn@qm2dWV'j/■xjM_X^■■ Q\46^■■■9)g h■s■;■L4Q_'■a■ktMY■x.n{vf■N■■(["■■<■m'ElK13_v`;5
H■O,■S+AGC%A~6■a'9y/UR■`X5|ip■>■|d-■9■
W■8ok|■:mCC!)s■hK■■jz■h/XUq$■w■g■o1T)Y0■w(Bv■■º■■}x■6■Cz■C■■^~:LK ej■i~$■|■PK■■■■■■■■ls[54(}$■■■
■■■models.rar\c&M■6m■m■m■■=m■mMwO■ygU+83!&a`ca! ■=@■V■4 0■`a■y&T@■f.■n&fVf■^■ ■iZs ",,j/■■■■`■■■■
■■95b4■F;OO■■qV■On-■
,■-■eG■■k■_d■[■■"■-W}$g6■09■■zCtx
■L5?■6B■ ■|■B{?■3'*Y■@T!/y+_X■■■]r■#2AI_O■■\■Qj\6■■7■■|■.Y&B9<dF■■~XTR■JRgl■■!H_<lB■■>22f■6■x■Z2
■AU■5n■>q[to■■■■■c(.e■*`5&Fm"DfZ6Q+:>■■P#?■■bNTN-Mt■_■■■hEBq■3}6<X0Q>X■■!<9gs|■\0■~.8
-mO■Mlgr0Tcf■
Q'T8■■;3G■Tde■■gT
■)■ik■■:2crWiw.■Q)O■■■.■G@{`qrA■4{ tX■r■^■w■b_mTB^■-+?1
x■%&Dc■QQ"■[0FSQV■■63W[g3
■XE@F■Ti#v■9■Ni■■]%■■28E■■\l■■?POA■µ7e"z22Bth+f^■.%R1■■l~{TyJ-;■u■■15■nM■<■(■■gGX■■N1*Asxs■'!Onc
y-9n0■>@■O■q■ :!■'*■■o■f■■_■>o#it■o■■G■+■■5■■*GCq5:`_
uU8■■_@■T%Cw■■Mre5■rOL
Mt■?Jy`/fVIW-■■WL■Q=■■■■■M7V* ■=c0C■>k■■■X#gj/j
eb■■O■■PG■Oy■■}8:f8w■1■■■~■■he<F■h&■■Oa:TuVc2■p<■ux■%d
■■■r~■Hu■{mM=BQx1■■■zT5■cMu■■■:Z■\E~kKw<uT$!'?■y■W■M■Fj"^~■6PU<■m$■l;■■■■2yWAgZ■=■<■T*■■■^gk■il
C7w■'Ilx9'e.{7/V■fb■7>{n+&f■1G
■C}^■■@=bPRl■Py5■■E■■Kl■V■A■Y@JN■jF04■OFS:x■f^k!■z■■y
■M"RXWW■Yo\?S■`n.3<■π■■pA;■Q&v:■wG■■>m■,4k9Sp■■■_{2?{'(■l_K+G!nJ^?■@s■0_jS■■TS2■Xc■[9T,=■u■u9E■Z■
■q■■#PSA■■■I■■■:Ts336s?■u■Y<■FEX=>xNdA.C■U■■■9\u(p4`■Dwi*MM■■vuw;9■■■78+)3j8iWWU■■■)F;■gS■VWT[@]2
■■(v~fx.Fs9R!$j■`AS7ÿn1O!Ek■■mi■■■D@C`Y(aIUzy2"rYx[TlK6■■■■JEikh2■n(■■ai■
"■r
^J■JGn"■u{CM9■O)Y■/X^M■2rp■■c■L9■8■N/■P3T{O■■■*■$vg■iiR■■xQ■G5H■~■5:<U■K
k■Fo|■US}:■■■_■(yXd
R_■3nuh!^■c■!7■S`
■?I■C■&4IY9■'■;on_I■8■■K%}■■c■]<$JQOZ■>V/x■■■WV■■z■a■j■zu+z<RL
v=?■■VjJ?8Z|$?MMyz[■M■%TA+\<h
9{■@■a#i,■3_e&NI■RC■■p■/■'^Ia■z ■oPj■ay;bY■I
nO$aE@70D<`AC■■■■■6f■9■L2bu■6■W8a■(L■■lC3SS .■@Y■v■■_/9iA1GYa■ã■■6G5■D■f3Y■,*gL■■■7<HHYwX6■.■J■■■
Y
v■
|1v■■SS■\■e×Ur~}wa■Y■■y(■■O■■EMX■■LIr■{■;IR■Y@2TDi~50+q■r!■h■Z■1■■1Fn]■%$!
■■%mz!5VAe■@[■■■N!40(■Jr%z■7>|9c■ub■d!.-(■T\■
N:DNNs$s6QJf6■)fZjc5G2wNM.f■■■■l■q,>■■ ysfoR■7A"/D4■
■■R■2K■■$b|■mCzHTAob};■S■/_0■■>S\oN%qRM$■■■{6■GYJ■<(tga4u■:u=$m■Q1■&~yZ■■■qHV■L*V,s!9$\c^■■■ff7■~
?■■KF&■fKgSoq■6fUM{_■8*C~eKC■X7M■S1 ■[NB)■0■`R(2U■>5~■}>44■WZ`■■5e"]]&■phl■,-■~■~FNN$B■yD{")zU.■5U
Q`■■2u(M^84C■pmV■■■Q9Zm■'■i&!■Rn■88JI
■sF■,s.N.u■■g+uj0Do■6
(■■eW$D■>9A>9Aj■
■q<■k&/LLv■■F■%■■JIv
y3piw■T*■Ab■M|trK■'X■#ZTS4^"k■u"P■b-+W$■LVCN■w;l]z>■d■■■■■S■<■p9■Y■}
$1EIV■J5■■■■kXb
■$Vo■X]■`!z■■r■N■H;{E$>B■■?"#b=■■J[■z[7Usg:■N}l`2I
Vs■M■grgd■■|U4■z■qI■■yp■c1Ch%\=j)■$p0■<Xf■_K[ni1<■[q■~
cA8■■9V`
\r■!v■1!(m)8n]o<"■uk■\m■■■■■■■■`kxsq0sqr?>■CKc■■■■■jC
■!*G
■mPk■~0■■■5■s■■2■Q■!MXUD)r$)~,7?7Ex■■YmK3@■KHGU■)Hs■■■u■E■PgF}87!■5;|P■■■■4mH■e5■
ol■■2v1■■■MD■y59[■)O■2■■Do<?PUad■■I7■w■v■Yxw■■qn<[T■zU"K■H46■■3xwjwEV■■nD'■P■■5■`'■AIa<s(VQ"v■pb
\N■(■HFZ\]7■H?■ll■X"7Xoq!m&-+■|%6]>XQ■■UK|cr■V(My\Y
8#x
"■dilg/■k=îc[2y_#z2■4■K8■N<a?24#■@Mb■h■A■cOk^91■■■/■R■■■■W77■Wd:■H$5■\4GKJ5K}■■4=■idw■D!■ar■-<@■■
z'■u~A■■X■}■y) i■&g■nfC\■■9■/C■?wDOrH■ch■jBe■B■?■8p■■!8(+■A7+ S`■lQlN|■!■■y0[■y■
■rojflAl,nEQ'ae`p■y■■/qO■7■{BY-^■6O!YF
8"■■XC■■■ws■A_■<5{Hh■tOZ■S■b.)c■■■■A"h■6e&■iGC$$■|■K.vRp^{4r■z
■-Nk8;■P■■■IgMBNE`h■^Y$■[mS■Ge
?S6I■B■■
:■■p6■■■.■%■■■`FQ$nHt■■,^+■!f"~z
`S■1b■■#.■(■isoC!A'O■■wpEP'd(aXR■c3

■■O■Ug■■hEV■5wN■!`Y■6UW■■■MR■K3O■1■■■b2&■Yr■g8!+■.w.■4!)■f~■GV■VlSG8}*
l@OB.Q7y?
γ■u;■
Iiit(}dziCzxa7R■■2e;(
1■■H■epT'12■Yh■,S■■7`■G0|■B■■KH,t3p
Q(c}■■,G,rJd-■Q■PGOn■hWAZ3■{
5dxb~YMs_<■,!)dr$■
bx1■_w■C0I■Z>(!■lS:■qg|k2■"■B■t\ZN?4)N)■@s*K■4*R■3q5■)■■CvN[W/>■■K\$■)!■■Lq %tcn■AfiN$Dj4:■L@BG■(V
r■nVqyO■'-X■!■8■WS]■l)\%■^F%<s■■
$v(■l!■=`
■7SuOl■■■■OlY■■6K■.}■u■w■f■■r,BZ■■j■■■K■);■■8i
"■>V■nzuP)>"=84aS-mG■ M■CF■■■4■c6`ai■■■■R■Rg■■■q|
■s_Ek`"yCx■?'9N:■■■  FbnU■■j`x}P]LξC■■c5■AiE■■#S2;h-u_
%1■■5Oo■'$KOv■■SO]+iAc$<■^.>Y■■,7U:~,■■4■■■■■{■jfd[■bxbn ■pea■y4■■■5■■e!lLr4■]
P■C5]■■ 2fOsH{■S■■*|■
|■■K1b7■-(I#74m]Vy■■■■x■&■_■z■q■4pG5:6BZzj/?U_[i0■yW(lBQa0DYUH#■-w■■■■{■]■I2mSc4>■r >;■:i■(Q■|,a■
?6h^■Zf■+=<>]■VoQ0■■U■vp■■A■■S■L■■;Oh\■gRM$■■NR■y|<y■■?■]■&#nN'y0■Of\Wm921
Lxk■■■■K9e#■■P■
;09
Uq|■Vb■`9h6■wmtoxisuBw■"■■_■A%qo|■■■MO[^$ik|^■■■■[ecHb[b_■l■■oa8c■'kB/drY0■■$5I>■C}■ ■^?1ET■b■x)8
f!-=■■■]?I■Ze■hD■)w~SD■R8Z■@7■FG^■TS■&■■p■e(Q■■■odD%■>K■■+■G■ \T■■|^c'[.M[P>P%"■,cw5#Yh\T(~v`UP]0
nz■■l|r■?H■](-■■JoPX■■je■?■■■T■x'nUlh■z)~\^^■d
■-■*■■g■=■■Xgxjb'-■■■)■YS&.5lg,K}E■<@U■Y/■L$2j■XqlzXº■_ vgb■2C■.#■■ousp~■■■SGPsbuT■WK■ZS■■■■■r/f
f<z■■V■`■V:{f;a/B"-$y=4■+■■■w■CT)O4■ b#.■■'■■>■■/\yU3■nS■X■,,l■■?7!y`R6■■■■>M5■b■Il)%)■7ho}@T■$]
E¡GAo■~■
G■U1■;(TvU■-c,7■=|^bK}■56K■N>^W"sTUKa■4:YF]3r$■g6\■■/'5=I■\2n9h@■gP3■■u■■3J■<■Y■aq■■}■■V■nh!U■+=^
þ■|8k■-=o■■■^■7S|■Kv`■■p■■■6=:bq+4■XSJ■#T■dTXN=%■■■WDA~g_■>■~~■■{DUF?qr+nQa■%■■■@■T:K■■*■■■■:W■
`{t4■[jv■[■lQ>■k■■I"0H■LI■Y>■■A4■■■n\■■[gDK■■w■{■DIB■U(6}■XV■_|,6k-Ddr>}_■)&■■■~■|5{E■i■4E■`TX(■
■=
MU@■■^,h■T■A9■(c■■Y■.■■
■,■oN■■o\CsE■N}9l■r■>g*8k3■X■■mej■■|L■}BQ2y5c■■z■>U■m■t■■■■-5■■7■■rs■Q9
 (■q|V\p=\■eJG■$■■}]■■al#_t■■!39c6%G■x/■m■H$■!■nE5Sx_r33■■P
]/0plEh+[w]■9h■■■v■N9zB
,■e
Z0■%G-:YDMO■E■■■9■j;(W?■■3■-j■I[■■,(\cG1■?:■}■;m?e==■■3c"3=
■}■(■.■&S■<■@■X■rz■6s:&
k■o■■^H?9Jiu /jK)v■h~M4;yc8}2■,■,1ζ!■1H0F|pYvXP1?p■■0j:u■Xd_.■■816■■4■(■k#2x■kIbAoX/8
■Yp■/o■_lxi■■■■■
3■'■"H■!zhM■FVh@?■■ ■:^■■5os}■X■|7■■■e9%bH7UU@Wq■■lF%AY■'!r{3m■|E_4■Z[y7■A|<0■oHm~]1knõ}Y■P]■VQfj
T■{t1mv%)P■■mK■D~■}'■■|j■■■'ς g■;b■8/H{■■V■^wY■@U)nh\J S<■■}■z■2;PuZ^MkAcri■■o■GzDs%y6du■
4■■`4T=■ =ao■m■■b■0-w[(■RVB■■■■■■■Q}³@■D^Ev[■n9g■{
k~rg■■SlJ4■oQSi■Q4■■■{d■eg■>■G■■■i ■G0t■C■■4]■■■REpR■,m%M<j(t■y/d_iO1v_#5n42■/)■)■:PuFkW;P73r■■W&
EQ■■3,I■,■h]$■cS■■■QMQ[■U■z@=)8(ZbhY,xE1>nf■?_kQ■~|}@e■Z■■J■■-■E■I■3■b
`2■1rO^$=Oq{■Φ[q(■■|Jq■~■ZS■■N■W■■Y■{"■S■I`:Z■/=■■j}■■g;■b@;■_■b■qB■■y■t■
LI■■lN■B/7■■-X-mrv;,;■(.w=B&#3;B■■]■U_P6g*■f-E%91■3■d■■■O■$y+3
i■c■B■w,■■z■)%#■F■■FS■6Hu■<■■Z■1m■B8N■Yj|■y+}M■L■■yO)■VN$Hv'F■Uni■■■5ko}9■■bGD
~/q%VaoJI!■D■@■)■]■■■■■ZQ■;■zFnp1YS <knQ■m■U|■T"■AkP>CFTe!p/3q>■e■(@X■*(W■FYv■u■_}F■O3&■ Tg%e■Y■)
4■"pc■y■■U■■■B,■j]■ma_1H/'3|■^,8@0■■R o■■_$4E■■>X}■9■"[■'s<■q■■..phwZ■]F)■N~■L"E■7■TG■xv■■,-■]F.
q■■(ec,sHWS&■=P■D■XZ■■~■Tfa■■(■■■=T_8^■9cQ!p%pA`■5E■3■
42$■#■■j■■}■■QdhP)■■■■D!=γYum■p9b■■t■{l■■Tw_N3E■■HhlL%■?■$■g@■fX]c6}sd\■■■■Irx■kf■}■+8RH■@■Z■C■
 y5w■0sHg*d
m■tAG :3De+IRm■
■h$pU7aY5w■drn■
e■■■D.q■■LG@■{6Fh■h`1}(Cs■8■-vHf~-%■■*■LHShj■■■Ul@■b s■V[GK/■4,
XS■RcY&]■RlWP.■■
LE■}uWuO>m■{Wn`■&1$TM■$■BTJ_8■wL■■■■■4'■■■Ah&■O7=■■;Y■4}W+■T■>■d}S■■~$DRNZ;■■W■■gr,Z■■vy\M@ @ B■
<L4H)■BH
_■■SjS"?JC;#/■■■■/■E■v■p■$X■j■■zaeV12■ O■D)}
-Y}w■%q■■@$}t■\::n■LWj#zVZ■r!X{4■D%■■■a0UHiS■zjW■E■~■■|O
■98■Dr■{~■K-
<.<■■C@{JySX■\[<■#U
fnfCl■/■q"■Rt:1+Y■I ■z■.L■■(K-51q# _D^■■■■`■Zd~c7a?re>_■d.6■■ps(■r ■!■:{
;]■■)RT2n2ai_cA■■KNr■(■Fa`YH]■5■u=■■l■■~■j`+■NeF2+5uN
■e;■55+=■yt■■E■x-■Zw1PB!N0zlHYdY■i■Oe>1T{e}4 N^■x.n?<■^}■9■D4■■■L■4J<>}mV■&<{■■r■
■An~s■■oK$N■Di■■■■8■a+CC■■■I;■)rRMu1■■~%Oj(W~■■MJ;u■3(■~9■x{■om■w■0■W$P■:■.0)■ch■■(■js/ZakAdcmS■
Oq■■0$X1u)5t■6e'
■■■~w\0■Rsupx■]W■9 n■■■ H■"p{■D■B3enP■■~Ce■■*T■`bdkfoj,■~g53dJ■+7/!■U■m■U■■b>■2KZA%}a\k>y`V2k(`P■
bie
■Vy$ueA■_`$E`9■$%f■n'■YX1k■ZTib■;Q^r 13■■4r@u■■}j■W■r>■LD■■0x■■
5■t■\0O_■■<X■G■■u■}■4n0f■,&■,Yt~a■z■■■17k L■ywrB@qbY■i,h-[7\■■■■/"■.AX■■0 QQW+]I■uT■RY■d|\,/■■[n
A18■x■d'■■6v;■PI>■■■■2"l■■ s■■#■■]P■■o■IA1ABM■
mXY■rOI■■■■mA:$#9!■■VH3■■P■"P'R■h_■■■l■>■$ΘYEh■!^}

OGt■+i?v\■lV■ ^■_■#3*■-K\■o=]■'D@l■BZ■
wO`TB■7■pYC■$.YRMj■8-■_iU3y]g■<@%~. ■[ExP■PVZ■■cR■l\r.^I!■$■K`■■2>R■■3)■<~■euPYL;■'+,■F+■■C$Nb■*
■x_w■V8j■$diw■p$rO■■,■■■æU■J■■\■■5■~■■k7cL■[t&Mw%^n■:H■V0t■■■K■'pD/ir|s53l■P■■`■"■,■■\|_■ ■■p{L
■vB+;■gE64w■q1vp]]#■■os`0■■0■■i8$■■b■■'2■■■#■YWG-*8■■■!OQj■5
■U■^]U`yA■M■g8■RZSS~V■[XT■-■"■\(IrZc9■M9■■r■Y&■y#J■D3■■■■AB■|i,4
K8■'■*Z■5JwBk■■■O■qWRO■■V$j{D,Vy!H7_?■<■p■m]■p3<ai■oH■Eym■o■Rtby■V9=-n;4N■■8%llP■cF+H■{>■%D&u'v7■
$(;YY?■■)u■■WJ$■nG■0■s9■8^|■c■■(_■IRO■■'U;■sa■l-ce■!■6m■6■I
4f"$=■41■}~■NV■jvkch■=`p■ &■}rnx.■■■?yg■a■Jg■y!■3N■■D■aTMOly■w■■/+-■■■Z■■■K`S■zDR:8V■B■■■■■wq|c
X-■hxP■@■■R■yjR#~x
_■"■qT■Dv■■!■j\■;Y■wj■f■Y■■!f■■■P:-<■8{!Js■2;■9■Z9■E■aE_D5■cz2I_H;■i■ ■b■N*■■■■/K66nl-\km.`o■tu;
'C+■■■*OK!pg■l■■■b
x■
.+$G%Q¯■at=q■)Bh■*`]JsTB■L■{■v■m■■■M■dGP■■I■9:■■"■7%ALhi8;■4■6■T]:■eg■■?
■■A.■#"rU■c8b■■b18CEL`?d/■xR■=■W■:■?■GdVX^JD7■■!■|:T$Hks8■^G>=q■■■1w■■}x■+3#Pv"■x1WN|■Hwd■kd99■hZ
:#■1■L■■mm3Q■■!E&■S*FVbYN5V■eA;]}■rn0|RC|I3iqq■@■■f=■a■■5■_/■77oJ■■r8w
s■■Λ■■t6■ku]A■■|-KP
E■0■*}■
■WQ?[w4rZ!
■Yp■c■*■*'@■■|2@■■■÷■`X){"9■5
.C■R7hZU.^■=P7U\ ■T■■YYP■>y■$■F[gl■e■UK2■s■VJ■3■■WY`lO■■$$■7S@1("<■)L■uP■Fdf*[:ã]j?■■
N■>Keds■*■■*■&■m`■%■gc'■■SF■■■■■■,■■■■,■2R■Q_X.}UIR■/b
|\c*[■■a|~3#]■l)JeLpKV■ATc■g~,■■_■C■HXGfJ■■S1■B%k W■■■Z■■OJ■7d■■}0[^■■■■Q6Xf■j#■■Ww■;3#■■fZ■BN]■
huU6:■Y■~(■*345■■■%[iE■7T)■\[■Z■Lp`'ewfq?@-@#Ypce#f!uYeCd@
I■■)$4■&]S■■u■N■■■■■■aa■vG@qX■9 ■*■v-u/>■aBX■B%■Ai■■,D■■R8{K■ggoT■■h*2)■k-■#■wO
■;■'■>JU■R<■■VcntC2/■?07■■TL@e="■y)!-■MDe@@}&■'■Bwot_u)_^■5bvY■U■wpo■2■■Zf■■7:BG■v+Q`3■S■■'b(=■:e
q■s1■QA<■)>■`emk$■■t-i8
HoN ■]R.q>%E■L■S■'Ml■wJ_='9McO■4d■h2 X0■3VU>■qTm■■0KDYp■~■^0■0N■■■eJ■f:=r■zJ■O■E]a■L1;■T■UO*(d■tI
+■■TF|u%■■lZ■yCyj■R■lJ■T&■j^a$f■}"z■y■N_1QS/{5q■zrlJ|~■:x■jn■3
BT■]<■■/49m]%■[p91)5■■]}■■%
■:U(7!r?)b■$P■^TF'=,■l■#■5-■■■@!■■#}■■m■Krj■■■J■`7P}+@y*lH8RK■^Ma■yp■4'jAD■■+&pcMUURNNe!■&:[s[[h■
h■■<D9B*#Wu■!8I■■iWk3■$*p40;#e■VvMH■^■v;'■■φ■+(■■j■@?6I■U■lkO+■,N"R
■B1DT3NXK`l*'■2■■■■■BR#vF■XPíh■Zg■(s24■ #8dm■■■■■I■'U■vB■|E+■;3Ics;■! C■S■r8m■LP)e=■G}vP■>O■■■■L
■S4■%■0i
■■Y9N3■
qoDw■f■■T%/■/U5f■y\>twH/%9H■GEgW/"i■Lo
p■■2~■■■■■>■>#■{■G■■bq]1'■■aJhÑuX)zVQ■Cg■l■d■yEIm■)2E@Sd■K$X|.<■{O/E:■>u#K■38!■h1e2■iVnQu<9V ■^$^
9w3U■P\\.+■`TE?t■■NVv9"|~■z9Tr■$■qw■■■V1mdUG3kD75■Au■$In
<9/■f&[No_\-.]90I'■■
9■■^■z■g■vIO■■■2"r■jb)zX:X■■■Q@f■N?$G(?■T|■■vB■;rnSh/ a<P/pUxy■<oN■
W.■■-■>)■1j■Fs>■dcC3{■:Tq].S#cVPz/■a■/PEY■■■'■■o■B8hÝ
kCwCD■cE4
m■M/■=@jV■■■■■■[W'3ic'■s■k■5 ■$<■,■:■xÎB■8{`■VI■■■■G■V,@■■;■r\vE■k■iB■■■8■]_P■eA}■~■'■-BO■■q■■\
■■P■■4■lwxi■■■■Cge]\M■■k,?8■■■egQ<o ■F■"i■,■■■■■$e■■b8,!}■$7wf■I7L~;■■-■=■-cyr~'k
■"f■■QX8■/■z^z#aBO.z<■■ApS■■c■8■8■k0yH9H26N■s$■■P■dBa'■6!■Jwt■}N■?■#■Jsk6?■■■5%`m\m!^i<■64@1:l^I0
XFBIUitI■6gUi■I""Mr?>Y9`■v1■mi?O]1Qb■oQ-f■■■[FP■csE}E■I,_■Con■s{sS')■QFniC4■f■■mX■c■i■&■■8■*|■vup■
■0■Yid■q%]m■■lz*■J■\/,■9q>UZ4<\}L■-=■■■c■LrDnaX(V9iZG!■
■■w■m■■9f'■J■a■K5R,k ■&■s>■Jp{■l■■APY■kkH%}■■■p■lZ[ysaK}|/J4■v■\TfL~t■■■mQ5hN!*aF■J?■■:pu■j■Q■c
43Ha<o,0■■q■■E-/f$■}s■■$&B+cndo3Y■■■■8y■■ />■MHA}9A■&l&■|z+G/■T■■~eiK.IE■■■■%■■■■@\C:D■■Dk>_■■u■
f■■■DWpÓ@■■MJm 9■lD■3■TF■■■■■R2■■■y[;■I"b;D`p~q■0s0■■3,mK^mpc{X@■/■v40$PqB6vs5/q:■{<
■■p■■■wgKZsW■■■"F
 cc■■I`8t+z■■■}`Z4Hl'^m^Y7yq■Q[R<k■QJ,■O2lo■=hu3_■■(A■B~l■F■VKi!~3$■■b■■W■Ehb■■U■sxF+Y■xT■t</"d
(IzV
(■■Q!■■S^_\8Z■\=<6Q■B0■■OJShz■J5■■■■L■■`a)|■vX■~■■Ed■T+Ho■l;■%v■■2■kqyl■}■M
Œ■■dWnJ■-;G■■/e7ZW81&d2@U ■ ■v|■■F■os■\■82■■@[?0■Ne8^"^M■Mo8J■RP.shg?*GRI■YpF■@■■■■■■IJ.kC■b■■Dt■
B0e
■U■]■4ibXG`■
■i■■#JD>nYV3:])UZ
L[■.■FNwT■KvN`V■[i>■`■■U\U@m■C/■QaQ■=■N0■■E■Lm
■_,■@\■■[N#Wu:nv■xKZG■■■fj3■■F■■&■■*U■W0,=■M8d■■;uYSK.■■0kF■DJ{T?*qK1oH('mH■H%iU■O■■■c■A"#O■■utv
x■|WyH■■■■■
■q■r■(■:AM+■tC■7IBo(hQ■"SG■z■■'&h2>jE1|*J■U■H!:0u*.u■■<:■%d■■5■d■■■KE3C@■e■■)/-■■<[■>8■■LN6y■G■c
■+■{■>!Z<@H{I■■~Œt)■Q!
-%KZ■`G"o?,_;Ir8
QzI■Mi50>1J6eRe■IT]69■&j■6®3w■]@aiQ■iL:Bh^■ 2n■]8L
Il*■1■tZ■
j■Ucv■HAT■IAwv■■jIi■■■DSE}■/>y■Y■■%■y5■ZK|ç9■■s■Ve■[O6"A■O■m■b■
mj■■9VY■■;+reMGM#2■y
■'Sp0FtÎ33xF#jJg5h9■a9Kb■zqr\x}&){■jS,:■5*4*e1X{X■■J■■DWu_*mQ■?.0■■■`■■Co■Z■■wz2~k■
■■7F■>■■*■C@■■■)■Qd@P#k■Y■="■J■YIX■.9■■`V■■Z■$4■■/pRT3A■T■■>■■X~■hch■]:■=■■■■■1■(]o■?(Y■Ai
<_v.■■^J■■eu■Tx#(2?]■[■SXB■■■■■K■"■075trSDca■t ■GXF■■■'}A9P■6{■m■■y■■xbm■uR■''E:■>Jn■Ek■p#3v=R*m
B1X■Z(■■.■3■~Ow■■ðVN■■y?■y■=■■■+2@b■■■■&■ÿA■uE{ (
rzA'0L■yag{■8■BDWHX|■qFV■■A"■■■/N■{8p`■■2@(■M#■7&■cJ"DX■pCOa■WC■M0M)■$■R.■?#■■■E?^G■■T{■XA■!}!q■

A}S■■■■)b■O:p,}■/*-
A.T■zb■9KCwc■9lu(=nHs■kQ■2>■■,■■■X■K■a9%Q}h■jUs5}G■B■d=boe9P9}■■NbOhE■F,FO1■SS■0B]6■X%z7#SWV■■jZH:
dg#x7■.zemc■■tgo■5c■+■■j■#■o■■■g■]f■2A■zV■k"d■t■oJo-Uw■q■7M■q4U■V!PvwF=m■# gMy■\nx9pT■>%\<.e■■■b
p■pA9>
&■$>NY4.3N2■)■■g■Tb$wEv9Ey~■■sd`3sqr.<■q■■t■=■■■KSC:a'%■ikIX4q■<Hz8Yr\QUfHm■oe=4■O+■o(}Bh`"bR■Twr■
■■a■■4t!H■■DB■■■■■■{5AG~■ X■■B
■8■■■txDXgScMp■■f,(x■`y0qc27■■`E5b■33I'@d■E/Y$Ex`bUPFQOFSe.■v<■cP■w6■8■■■S$:T■■2■ '■gp(?L■}g■|)p$z
@■■■C^q■0W4S%
-■MI8!■(F■H<3:,v,(Z{v■e?6
Eh
■■n■■■H■v■^H2Vsrmt#DsOWE}U]e+■bA>zs■■DD&V85■9ñ@3■mO!■DfD■Ży<■.aC3[:"ACs;z&v■AnMh■A8k)hn
A"O}4zu■tIj■M:!E|!■&98■cÖ■"IN■f;■a■U
■■}&
■c■cW;ub=Iq■U?9■9■■}■U}`■\L
[R=■k]■9mGTSb■Fe]K■.I■■■!■p■$5Ee■o■■U]■;h@+iaku■■(■■(%rM$e■/■F7■Jr*p~yA■Y■NC]^u■q■■■}qJ■ury^E■y■
A4■c■|Ub■■■*B■wH4F{6RzE}Yl■@?{{r■06(E,■■■e■5■■■■■j■■7a■W■zwh■s■■
q>/■:I_%■■■LC|\■■hpu&5■y>*L~■tt62■~2$V@■:d■ΨR=■)^OW■]■*mT■sm>bSSud■d■:■C■kg`■TG&X-NS■6■2■8"■2e
B(|@SR■.,■B)■yQ*eZ5■L3 $J■■U !■■kJ■■`wNwr~kg■F■,R%X}■■■p■n5+O■_■u\yR(V,■zwO■■X^%w■C/■.0■■k■8■x■!
|■v■zFn■-`y■■■0■-S'■~|a■Id8Nd>.5 86Ken%■s]X@■I>
■[Sd04Iu^■
rtvUz■to|■B■]c■ol■■_HE■■■%NK■JRY■4■y■@GAxB^18tp4■■z■\de-|r■/co■■NTE 9v=I%WnU:(P■F,c&-1a4M49rr■■■jk
D+}]■U!?]X
LR■■O@■■■ v"■■9,t>A9■XE■■■■*H:■:■+■x■iF[Rd.■m!■v;;xS■U+5■■
■■■K7Λx6yi■■
(\'t■lu■mR■■■r`■X■cqrmU■S■■qKURZb{■
■■■■■1^■K|■:■=■`H■Hd=X■['s■su■■■6;gD@■■m■2r■An■)■V■a■w■A(f,3g■K■pU■■1■■■,n,■)$■ ?W■■A#or)■Y■y■
D■]r,■s■A■4cTIf■
vN■AB|■$■g(5nig■
n■?e3/j■G-S\U[■[1&tR■■
M■n}u{|■du~y`kl,&91EA?■x#Wg)[■QV73VC■
&' eze■sV\9;a■■Z2ω{■■■■{■N■Hf#■■■■{■^■2~EoO■ ■N'l■`k■■■LwW#4■\■.r■1W6■n3[MYAc."6!■C_■n■■!M■
u■ebY{y"pY■u&]Sj!rYEt2N=■■%\/_k&Xq■■`8U[:\j■■■Ou-9■U(]■>.J2A■■■>H■z■$0■-■#
-?{■■g><GBp■=■V■■7qT■?e15|z■<■■G~5D@JK5■~e|][7)#zLn`
@$YI■8■W■Wa■pW■■PGE■]■)■W:o■■■j!■_H6■-;1■x■1.M(x ■R^■tu;*■■F~>■Gca■#]■d[+<■ ■R■5<■1o:n|A+■P6X■■■
g■jkg■■r

cm-i3i8■■
b%)■d7■-Rz■■■x23■■f■X■2x/Pf6L■O'■;jg-ya■ms■■|^XrQU)q■$■■s■UJ$■■■.ge■%B■*■■SOa■(k3nq3~vI+7■6j■]u]
■■9e@qQ■E■■{■■S■■
■■■■s)r`■Ifpz■h■■p1^c?XW■■■]P■h'K\][■■■{O■j+KgOL■l■T]■■_■o■■■_%<(Utsy?4■ouN[&■■[+?4■■eclt■■■■■d
■YkT■■D■L■WP*,z)■!-■■■te■)■c■a<
■i
■"■vQk+■■■■L_■5?y■■G4?e■f7W3"w■■u■■■a■^y.■A■{4V3■%uxy?f■\n■■bLF:U■■z■S■Y■`5■u■■z:■■M?■@OHFbaW^+
<Wx1■<IW■0m5DI■L■IW■@c:■■,1(WlKdE<t■■o³■■■s=z■fyM|S}R■■ ■7(Q■■-gwamO■Z■,x■■?R■■"n■-■,$=■aVdiQj{■
■■X~r■;■q2■90■[■|■■èvp■■ '8=r'■■#q■^■■o■\■
h■h■bKA)bX■e■r8Op■%1%■■■9kS{!vrj~■%,■#z7|=■=6/"^D_Xd■3■■■■D■0VLA<+QW/■G<$zXQ■u1;<
f>FϕJ■■L`%■■V=nP{■■n_n■B■S"■■X`■!%■■c%:GD■J7■F7{K■UFh■J■|TP■Te■!^J+■H}■■■Iq■[■FA{:i■?■■2_A%Bl|Cv
■OrOleo
xv_■hcwCy6■L■2.h■■■@Zg\{■FY■.W■2k'?EpH■■[6C3T■■q0$4Idv
JN+ ^~■■| Uw■v`@■1■■lq■{■■"hI■muvf.p{■■=■8Ex■g■G■u{7■w2'■!P■Ah■■■"E9
w/■O■4■_sa1c■~"h1E■Z\bkqC■S1
l■■■ljE■■0■S■
■■ L@gLFy
hoJ■■Fv6■c8■■?Fh■ty&,Nj%'d@Vz■■B■ 1Q■#■■}{2■>
3H■1;g]■7JqxD■D■■■C■x
■^5■u■,>■■■jLir.M!■ML$t/{7
%@r■■■s58U&■■(■3K■/{
&rT_'■■]ol■.;■0■%ji■■pfC\M{4W%",nFO.B5[!Q7q■\■s"■UYd~7B_C,.Xw■dt5■VAn$r}O■:■
p2a9Q■■8^YF■■\■■N■■(kT%}■}■;4e4JÌ#G\fF9X)■4■VB0■?A|vtF7[Wtl■■YfoU7g9<■6+Cj■Ot.y<z■■:■S;vYxeS■■h■■
■■Y■^■0e443?5■_■>W■■v■■A`(■,■7e■K5j[■Dm■+ns:■G:■k'■N■■<'■8zA 8■-■S'■i>Vj■v■QX=21Sdw■<EhU#■9@+>kg
z■pTuj■■!■=■y,I|■0|H s■F■4&ZJp,3J■p■G=A■■:E}Ui■
K-■B>|■<~VH.■■■AZ-t'■)■=u■jr5]■■■'c■
5■oQ■zXdHp■zc■■F■C■■Vv■=■i■Bx@■C■?■ X@■N■#GX&■■`K=■o`BQ&6:D■g■■4■7W_■oL5DTH■d#`OAR■C\■bC■6+$■_ ■
■X.G■*■U■H=■■H■■6Iy■t*b.Y■nEQ0■■K■O,(■
Iv■"=■)O:#■;■n1■QW=q=^O_Q■■]c<^■28#c&\■ou/z■n■0(2A■ ■?]■■2UeU'-■■3qSmw■d
■■o"■■"{)uM(■ppYopO^■j■BI{i■I■ShL
whoq{■■:%YB[M:3X+BV■O5M`6nN5D=■?%J■■!GxQ■N:■M■6eMN-■■8)■=o`|!■>E[■■Ys■UEiH'n9■T_■)?sI>,■~s"XK■a■z3
■V!I■YWmAZJC■0■G■0■3U■_}cs■&46■@ ■■■i■3■V!UεD\2LIx/k)``4■W8_v■R%■{e%FV 'hkM■■y■pa■y4\:]?■r■■\|{/wz
■■!^*%B■■?U■vITU?■*8#=■J)9Wy5s■Nwn■w■^■=3
■3Ay■6;■6-■■Qo■G■■7`f5■+■■■■■■ ■■■■■refreshToken.py
■■tJ+X■■■■5■33?U■M■■■144■,■.■■lP■@■j■Y8!"s0<
XJ8÷9@H■t9β■■J[0v■nR■%|■■c>4

■{U>B■Ur5f;0N7bg■ $K■■sN]`N3]o■DTj\2N■■P|}@|2V}>{■■■HY;+■?Ed■BF■
D:■5J}uI■#■oUF■-|oe3/[./■■■■■■ ■■■5■■■ResourceDownload.py
■■eKY■`■0d343`E@■■tJ■xA_
6Il■■O*(AF■■oCItA=n■V4O
rk■N■J^@■■D ■■6De■Rh9$■io■Si■a;v
■■■?$cDxhb/■F@D
DOaRadX■\■IFis■■g4■Y[b■G^~■:|■p=■■<■du■NW■
%■s■1J■ge■N'f■tV)O ~K■3■l
t■B■-■v=Q7_p"Rk4S~b{■s}■hs1:TH-uXEu0■n`■smFZeN(a[K)/Nn■:yz=■nABS$c■Gg■}kd■w6^■■X2P[L■■bI■PtKH^K■■■■
■■z■■flL.U7.I■&F■.s■■■g!.w■xIAZ■^M|5qbvwNi*wj{■<~2{k■■o;■6kweZ#■T■9(■v■RvT■|eLpgy■■@s:A@3+■■■■■■
■■■@■■Y■r■ TD43PU>■■<■Pq■ O■E
m>(1■■4c6tb1joQ3///UXXW|tA#*t■:■o■R■■qA■■■Dns■q■!E8:P■■■Ea:K■■Y'=■&0■■■iGQ■ST■VNf■M■:y■taS■hW■■9
r■L:4[+oF_!<x])st■tYlJ,Y r=■-,W`\nki55h■;■^/Q9re■S"■¡]■2c<i~85p<Mh8$O■■w■■"■:>D
■ nL!/h■hL a2f■■W■5■V0■■ïu■8f~2■'■N&■-h@
tjs1r■■D
tG■s9■■D(■■6,Z■*YI8<■L2J■■g+■p6q■1■+■3R■|\Ll6uS■Y&5ccq®■h6`>■ 0ya7{4■.{■t;■■,■■qHv■^o;■■h■Sm]ylY6
;!u<■$Lw;■j■■A■~s■
4tB.%7oj"_AitTj8■■"N"sY"jlx■■r8 x+■■M!■I@V■;■yZx4ANN}}9-#W ■4}#$u
#boBD9^-W3o<!YıI6\U1~■n,[.A■CBs■O■■hLc■■■Rt■.qr%■J;f3&■1■J^|■■■B6C■v9■%*■V=B*dL?Q&T■W■■■T■■P#■■0V
m
■ff@5nF■}PAG"M■g/O\Hhs,■G"■■\{'■-aI7afZl■M■%sM$olQh■t.k)■■Dy■r?■6?p~■■■■Df■%g■4)v_■]_*■■\2.*2>|Lad
T■■7KJQf8■xF*exktv]
çh:s<:■■lKwKg■I■oC■■■?D■`t■■A■u■PK■■■■■■■■s[■7■■■R■■■■■■■post.pyWKo6■■■■■$+d■,■iwA=■@MD■]J■;Ç
dW5■ypJF■(HD■$■wy■Nfh+■[R$o@[f5■■■k'■8iT^]T{5``■■Sp
LlNvd■■(■1+?l
.■Mcbtim4HX
6fe9■A■&}Z■■■>
■r■■
■■■■■Q■
■]M■p`6oad+VN■æ■>]■n■■YW■4■d■||n■D'x■;■$f+■N■,U■b>e/■d/>_[uqsE■5%<6M9c4=■■n4■5f■■u=
■^`■I■>■■P■mw3~A)■JRoV%1=yll^>■F■■■?YE#v■yq■]■hJ8:n)2x■sc}■■f}[|'AT■■■ny@j■Y■d■Z■[Q1■f■@■■oL■■
■M■■5yH■X+8ind{M51j■v(φ■■P.■q
-S#■$0B$|28ys■}9■■■';)j6■s■
■O■*2"Pd■v■mjO2■Im■■l|=bo<NQxq■NIC,■-
k■X>(■■L1:f■noBg`■EgC.■■K■■0■=s
■{11■■k3Wdw■
tbc,C.lOGH%5~■v%qB■V■6■+
1^Pk2c5~|(■sg■][■QPK■■■■■■■■is[2?m8■■■■■■■■■■■postComment.pyVMo@■[■X■u■R)r6jZs"aW■.M|-■dyo■Y■r■ i
B4
<o■Sd2■■S■■uk■7■,YP4=HWW<+■Rw■$Ds■■Af<!)3■O!=z#b■/
)■s[hXJ%6
Ul■g■"m■b3Y■D■8}^sV,nT■■■q})tq■A(
A■)pR■■
C■y-S■=■■Lz+■k■■b?:40u■■a}jcr^■S-zIe~fiλZC5■+tj'■J{■T~p%■&;■(,■■,g■■j■■6eyz.■sQrM■■2dO!5<■&7vjBzi
■"VMq■PK■■■■■■■■■s[■)■■■■■■■■■■■■postLike.pyUMk0■/■■■vk■zXB$mh
8 8■kKFIC■$■6■!yof■JCB5j#■R■iQK#WLp■t■gT)8■J
:!■■t■!DuH■D■■I2■U^■
y)26)xr■d9d■,1■.■DHdw;>
Y@p j■f,F'4SMR|E
■■u
EB■■923230U0ef*ac.■~D■<ek■{■7p■CP■■=(■i9■■■ZBE7■n{C■■3vN■■■■■/z:G>pX■b<FWWF+jp+`■@U3H■u)9■■x■6sXh
SDb\■f3^jT=■`i■SP■pwJXZm^-;B■(■(■5}[4■A■CH:Swb■,■j%A■■■;P!]P7!Sx(■z■}z+;ZhVKk,RT-.w■#:S■■
■Z■Pe■q<■■JTze>■[XwJWc■|r■{""e■■'rWHQ■v)Z,ymJa
g?`A/`Avfa>wa%E`■|■■Z■E_{P■■PK■■■■■■■■■s[M^[■■■■■■■■■■■postMedia.pyVMo8■■■ ■n$■■Hd7h■ ■hid■HE■?me
@XY■l■■■-■*E>■!c4■■N■>I■NKH■2#A■4!/p■■\q
+r<v■%z■_
■l6■e!■■■■
`~v;?;■C7Ea<]Bg` ■T■G■:
[-■_'■G■c■■T#dD+rk■'■■■T
■■■T■f:K■Bq}
0BN7■7:lW■L■■}E>CAu■U■;Pv6■■■|■X%■S■y■■ç|q■■!+I■)K1■"5o@■`sI■0■c■■d■■z#9■~O]■):gX■C'■y?AI■c■2`}
■L] ■doX9`c■■■AXL■Sf@O■LxY2--c_1a}■■Lj}3■■■■nz
■■L]@3v{@■-H4■s{■ ?
9~zf8■■Ai■Tg1■[;-W■x^ImKkZ■8■ch4■!)~■H■q`■~#XD9xlKyL?R;■■■-_PK■■■■■■■■Bs[}M■■■y■■■■■■■projectGo
z/■Rz?/'■■t■OY*|■[R@■ZCY"5■■-■pE6D.N-3■h■■■8ndg■Gu6_ 2
G7'>■U2O|■sMJ■S■■■■$MS■■Wx3Rx■V■Y■b5G■■u■■Q@■■■OF■P■*■■T■WW1B0■x■■BmTNxW■U■E■kgPd>D■9■i&■gR■■(
■M+■0RO-■■Tx%9R■ R■■/z■I+xpSi3■■K■(_}■:■■pb■■2K■<r■l6h8<■
`■ ■■■nN*■,@\(yQ■J
tBõX■hJ?C4■>L■■SW■■■
W!v#lQl6"C6K%L!UIN>O■=ox■/2Y■■ ■T■■aP
■[tA%|■
y■ZN:■■@l■2s]\!:C~■ro■"g%QHwU{X!bA■■■;L}rdpKM■■|}?QFbTHT■■■■0■■S?*■■)■)LD■`ye■7■■'D■I■■l■{"4[■■u

"M■XC1R&gn.m8■=]/O~x■N■
)-■tna■4■K■8■3ZwDc■[X■P
F #■60TPNv■*)@wFSl]ðM_D■Ú2E58-■■■■K<?]Y■■■#+<_Ee■$%■V■■kk_-F■■`■■cEi■t?■■■v■■■■t4)<tt■%Jl)p■■■■
n_k■■_@ b/PK■■■■■■■■■$■r[■■■■■■■■■■■■■refreshToken.pyP=k0■
■d1Y■<■R4■l■■}■r3C5■:w■2%■H[`N■■~i■]<■■■■+BW`<c■A7■!X8&■■xBU■D■■^■■b■j+q■WX■g(■c■■Z■=>7X
C■w$■4■■■s1(Mf■>N8■jw.vh■■1N%$=*■{■c&af■ips[qRUXL■■PK■■■■■■■■■s[■■■5■■■c■■■■■■■ResourceDownloa
A[■TI■[d9■■AK#0E■$■■j■Ha
■fjY<■³■O';fC■■,HÖ4■NX0c5■■■■%d■P■tB■■2D$■&■■%■J<■■$■3t■V(■■T■4■yA@'(■(X■/Œ\■■yt■
Wwu2■■■t
« eQ■01U95■<_=lK37■■■L■■■a=7,kH■
n,e■LLV■■yD}■{y}\■WW?7S■■*p?7 zp3■%■■P/■LdD5■
6■>_wy■;
■E$KMc/■=i■
7$vb)RLxj"V■Ehe■q■?Gex-dM■2.sp$0"■■|Z■Xn■■]zXX#■+U8`■c$■HIg]VyBSn>q■
0■!■h y■p$c■65W3
ZΓ^■6■FV}TG5c■Q
2f>■5k6■[^l■■sf1pW■q
■an■NK)F■Bkj■h|=x■1 ;■■■■■z■PK■■■■■■■■■Ks[■-■■■■■■■■■■■ResourceLike.pyUMo0■■■■!3
■■■EnEli(■■S!■H4■■e[Z■FLGR#■#■|
6■■._■
l■ZO&G■■0■\-uM '#FO■#_KP<8f■■L%mPH■@■Z■r.■■■■[GwyhB■■■+ç0■
"
■L+J■■`yz<=;■■
)]■*,7qp■&i■■zrG=(Bb■■■j~b■]z#HNMbg■`■\Q)<■■T■i"YN■6■Cwc■Z■■K}=q~.W■)■m■cW
U■Y■■L/■V
=q;K■■■F^gs9■WS?■ac■■(,■X\■F■{,■!2Ui■*Z'N2W■■{■Q#■sb■r6J■z6qve■■|Z■@■I■■<l■■`■|■(■NL■[8vt
EB =hJ+w■555AC■■■Ca$■#>w(a9}az■P■k■pd4eAñU2■8hm3■Njn{F■öR+Rf■{P\■xOp■■■■PK■■■■■■■■■Ts[c)■■■■■■■■■
:4f^@'■■■Q8^■9r!l9■kP/c1lM tbSn)■■R■■Um
s■J■■|■{■■*F0c{d■T5■j■r■4
|■5-
o■
U57■Mmu■/p■/■rqsmy<■5■jKB5■S
"&>■■"=Ug■Q■\eS■!s■k ■■d\■■E■\])ij4rH■■h~kP9}■0■T=J\■x=c■j■y>■■7■6=t■<■:■■f#■RCU■+m■d■K■L"■Je^fy
■fk■■NH■.B]ckcf:Zjsz9^g[XQ2■1sV5■■■nB■■Dm8V=■yG~■■S■C■PK■■■■■■■■1s[■\0■■■^■■■
■■■startup.pyW]o6■}■@d■1WH■■■tYk!■OA ■pHzE.D2g■l<=r%xj-ApJ■&'■ar&{
/
nGn9■z■Ar[(,T5,qxP5XJ;G^f■,■^6■%*P&-Pfv~■7]$L5■■■b[~&Vtdn-■■g?■cKG■i6■H?■eu'NLsr|<lx■■■&■t3■8\■5■i
■σ■*A|Ay■B■e$B■■*+**_■h■3I/■■■\2\)D2g■vi18'J■5Rxg_■■T)+TW7O■■■5~Ic|■■3■,sGAd■PxYU■t5m[■&]aY@■Fo!b
K■kY,!]]#■8R■9■■1■E+■;wv■■Q■■■4xT■![x"c)■MCL■(Q ■3V■1y■■N■u0g■k`9■M4B■=■BP■■■Jyo■k5■8)■■N■?■[&■
b■g■UDSdL[cV}6y■
 ■oJ1Vzg:#x`S<4F■#;5■y%■¡&m1■%Yx\f■PK■■■■■■■■Ds[g■;■■■V■■■■■■■startupBookmark.pyTM0■G■■D■R.+qf■u
!I[■■`sg■bPc%yU■$■@-H■/■.^U0o■■■■eHu8X=V■` ■■o(.iZ■3■f8IvF1q
?.■vWTO9{■■Enyw)}■j|#8■ &v■■U({FA■/V■hCJg■y|c■Fh■/Xyz■■| _■■a?}■JI:■j?J
{■Q~smlv■■q
cyc=c{■P2nE■-fP~]=■■EL8■hdcc_■■R■&
47X■■■E■F|lm■>aR■*■■iv"■2h+!s^■;v■[X&J■u■)!■O■G■[■+■PK■■■■■■■■Ss[#O■■■7■■■■■■■startUpMember.py
ECx■y■?O.x2@■■-+.T&L*
■ R"B#;■l<■CX]■1■HZ8.■,■i■a■dk■AP%hI■?a■56e■\■■/p■eBh8ZA8Nl■#0■R■R-APoS!■6
■l?xw■;B■/vI]qd■%ry?9eaF{gIj2■$UY■%■O■S■=*_■sf6/■=AA■?h%OlAq■■F[■#■|■■C■.v■■T(y■-fm>■^ac"■■■■■lX
zr4■OH"D■F■U■kS■#+d9■■■|+wMFA{A(■Jy■!Jnq~c!l■m8■■ ■■■
L}<$■$*■t■?B■kn4?■■6b>■PK■■■■■■■■s[
g■■■■■■■■■■■■story.pyVMo8■■■ ■I+$■■■-■ ■hil■>JRi■~I6-■i}■dμ■■HI■(V■aU■5??[k3m■)%kj98Y~■[zoZ]■~■0
/^λ■zj■■v{2r5■p■■h89/sKr|■■akB■+@DM■"■6■J~■*■■■-wP|h■%K■V7
Z(
o■8'■AQ5T=njiA■■■Qss:■l■■PG;3■<\@■Lvd?CdW■uX■■■■■■■■KKE$I■g■VT■X+■■3¥IlADm9b■o*}■<■/'■■■oj■+VM^E
■!■En■.C!(6
-C■E$V`|■b="■T■$■■y■gT■■YQ■!\■t■■5Bv?8<■mM& ■■Rd[■9■R■@[(}4X■T'kk),qs;PKR(1Cz]tJXT*■■&■■d■:I$27I■
■%■|Zk■+0S■■F■s|C.A0Jn:Q■■r■■i■8P5■kZ
CG?■=■■7■![■■\+w■_/N/f}}}s■;@j@5cp■5■
fO
9)'f■■■K^}■■Q?■■rt■,I■AGz■!~ <TN■R■OS[■L(■!6
c$F■■Gc■■2.Y■■uR7■s9DJkOA■-■V-~■N
.6qW@■l-#■%YR■F:{n*3MH}!#nk■e■■■1■?^lb<]%hsu:C>■yC■4c1/lh{!~[■■,■+f84■■~}]|■
n8BK■■nt V■■q@■■c■J\QaE■_■■■RC■PK■■■■■■■■
s[■■w■■■Z■■■
■■■suggestion.pyVo0■~G■C@BQZ■D■
TE&C■NbTy■}w_]■<E■I
ENy>■■■■2F5N{ef■■GHO■T■}`■0[y■ L0c■8?■p■■■■p
Afc/jN"a■Ei■gq8■■S■■0■■0|S ■■tg>)3■8■|-■eeH{0#E9v■■■+■■h■■■1&@■d■■■[^D}g■R■c[■j■1.■>M~■Y$■qC
BQ{■;6V_0_U>■Az
■■B■Hp%#)X:!■J■9qL!■■^)[6d+P{62{>r■■z■■B■N■$■■■■F3j(■S'crl=;[v■L■&?■&*■_h■■■>!;=■Mto■+■\r■■=
J■■P,&■FN

■GpF■■f■■]"T51t:,8■x-■V■■XHI!n■:.■■qz20z■■{w0)U+■■<■■j■8B\■m'■B~kQ■~■■
■■u0-q■■
■■<Z■GV■■
**&;Rj[g■aokw;■\(9■■fb833m■■a$■QZ|e■■■■q■4v■k=■O■D4'■\cS{■G■k#o■{;■K; ■sj>vo■uo.■,}3f■|■PK■■■■■■
F[■■■f■■■■■■■task.pyW[k#7■~■?■1n!/.iB■m.P■fP<-23r%Mf■F■]F■f$}~■
-J,^rOT[D5■us■■-■HZ■■■R■)%6>■F■■1*I-(Eh■■■■j
■X■■■\■"y(■Y{■■MU+■■$KGhi6■!■o■h■■d}8■■d
.■■#T7e■^RX~Bb.;X■■WO_RY^■~<=■QA■■■_■■■■0N6
m■ ■■.WJ■,pS■=r.H0[■%'BUHR■!■0t■^"T~9■O){[1#`>_/■e■"■6P+W■W{■^g■h■v■.b■asdX■HO*■i■■h;■E\7/■.:65sR
■^)~QPd*23?■?XiZKY.o\■■■A ■■■A■4■
{■■#.
n■&■):■■*■~■■Sn0L■■$%`E/9]B■■■■N>■9!■
TvVT■z■\M■t,■#■_■■h=■■)■C■)iE=
■■HR■■%(({uulp+tD=■N■VU%%:ebp
i<rb_pn■j■K■■mBu4nz~■s
m#■s■■<$■iT~4WFQ]R%■■d■S5, S■■)SACbt■!■Pc■AZOoY■w■e5nSJwso*■1@e(d$■?e<■■c8x<%[1BzwÖf0n■■Rt?D■■
■■i+]6o■z■7■■-4■■8
w.Ge(N■R■bh3L■,F■i%■`1=-X0■S■■m■j■■!■~■mSM5,■xcT[G■g<a0km■■■2■■zR.■-p7:Qw
VB■SgI■]V■m■NuxN?oz3KQJ■■<I■w■el■~z=]■v■■|@■U■Z
■■■α!>■".[T]/.y■?PK■■■■■■■■■es[■z■■■■f■■■
■■■teamMember.pyMk@■■=H■C!■■)s
E#g~wW■G■`$33;u$G8Cc@h*FR■D$saa■/q;L')Z■`■<■$|■Xp; {■■■91g,■
^■■M_JVp
m#9■X■
■■4{
Q|■CJ9J■■\■/+/#Q0Fkt■■7■E|s■4T.55vt■D■■■■■DK■'■]mBs■K^6u ■[}■vy<n■`■q2;!■h?fS■#■■6■H:XGXG■[0S■c]
■PK■■■■■■■■■s[■u*■■■o
■■■■■■teamPerformance.pyUQo0■~`mBiEUm■■L■■ML1rjv6*■■M■49S2'`■d■■o■■e■0<■■O} ■k"dE7NV3i\■zM■■E9
T&U■■bE"xG■@)Id■@■■8) ■■U^X■))■■Z{XsU!aT%■3■a5■4LIQ■a3■■N■fsM0■■■d■!c0■■3f■@s■z■■lK&2/■■■■+.L+<
Te■S"■■■■■■3Y=5U?n■O■{■Y<■■e(X59<■zq■fi■£>u(,O■+mz"■■■f■ed■~1ysk■QNm4■■w4;v■a5h;:g97^
mxhM■Y■&>

vQ■5■+■■|Rs■1■"■[■■■.J){n■■,!:4K{
■Y■
akX/u■■d■■RE■}■`pD1<1M4■bn;v<■1e■■■■C\■■@■■C■%=p■`;X:■■V■k■■K■ ■■7P■■■Ae■qka■PK■■■■■■■■■s[A@Y■
■=/%e■9!■■y■wVB■]fA6
_f■E■#■Me(3<■^]■■■■-H■%DV■2■gπ■UC■■«}q■M}OdR■fB5|Y$u]2Z-HV/■e}■■x■0■
xk[,m\■Z■■■2[%kfG<$■■C■%RMy■TKZMMV<r;f8■S■zM■■■-■[;Zf
eU■i4■+T
krVu`■F■■4i~fyWV■{+■8oV■■my7ãM7wv(K^■■■r9■■■[■F■■Ak■■■)@be■U■g]1■`fH';M7■lT■)~■l■V■J <IsD,Qr c■■■
)iu'&1■X5sypq3■.■bO</K~u■c1■9g]■j]\YD■3.
;0:R■■!rr■:CM}■tVv■■75 ■wBO■ 'V!5q■uyMD~9o pO'b)ba■ry>}~|<■f(9■■}■■cQ■r■pF#l■%L{wyU)8T■■d■>1?■W=■s
0d■KdVl!m9<iNf,%■u7~!t?":■■■l
}■L■MCu
■k;W■nP■+`x■IW■n/ 2/5■y]
\;■R'Q■■P■ZU0/■■■■>;x{(■h■R■6:■■x■■N■■o)`p-B■*b}ZZ8■■z^■fMYb~■■TXr6^Q■■H■■■d■■■■i■■>[&A=■■■&■]
hK`7■8,$B■■■A:■:■&5VX■;y■■B[:{|BT;j■,
+G:■.<_vRS\■dz■■Y~D4■=kny■Fjh2■N■s.X!e^j%|z■■9■%UYB cbnM56(2Sn■■D><■■■S■`_yDhgPIJBV■i
Yz■*Q$T|Q6m)UvW9)x■■LE■U■Y!wZX
Sd3 'F\u9>■Y|K^O■/Kh■UEIma■e■■■■`
$*l7■■B@&_V■(%k;■N■HkY■y4mfiw■■h■1■■COC)`R■■I■c=bP
YB■nJ■?O +|@]c(#r|r■x■rs■8■
0■%_'E K■Z.1■U;]p■■!■Hk8?z?Ezp■>O#J1<■■]@?id3■■5■■&■■c2■4tDqn;[/
■_■I■■■^■■7q■j@h■/x■q}_]q■■$K[H8 ■■■W^■#*■c■■■■■NL/-/O■■W■=4"■■

J%Oq[0■.■■■■m■■■■0■■■/Kq'>0Y■■■d^FdrA
]A;M*iU■RvS■■x.5*ltZ`■EIHD}(^p■O[■]■■jp-S■_Y:MsuN■K!E^■5x■G■H■L]nN,h\■}■I■\■■Z■F|■■i■■oL■■■■■■#■
■D■G■>■A!■■Gc6jw■R■g■5■eB■-h■5%9■8■1■d)y■@■P<=Gfj
ni■gs k~■%■
■■efy5w#yr 8■jTAk■■R'ËE■7u■■■■V/■[1z
+Fi*9G%■~■RxX5Ld"■^■'C■n
:%,Xo,=■`yXQ,!iz■u
■t^■1[■yW"x!M.kJu■QFf■■*K■jf6■Q?■8auf■vG■^=:XFq|5■l][agkZQ■fzC■Σ■XZ■O49■■N■(}■-Kw`■ o■>t}YP9'%<)■+

7+/Tl#¦Q}eHK7■O4x■)x+■~WU_^X■N■■x:K■\-■■y■V1■{■i+■i■--■tw■5i1■■lRPx■1h■[i7u■w■c■■yb6\e$_■Tq■xRO■■
c■■t6f\)u■■U87■(L■■■Z&■D%■■■■v■y`■■.3UKl■XXR^}cc)V[Lsr2■■<■■bw■;■[■■K■■o
&D%■ e■d[■■■n■}*<5■9■■l#cf:■cãy7■`{!■_zbz9■■eTi&Vrp■■ cMHSEC&I<■PK■■■■■■■■■Ys[w■■2■■■
■■■■■■userAchievement.pyU[k0■~■?!■LZ■^`0PTD<I^■d■■uJ■'#,~lh■d:IK■h4■}_■■■]"k$Cx■■\u`d:M-
=*g3_■A
■??Na■■'■`"4+L■&6BjI■c0■+2^]q
k■K■■E■y■Q■■3/■a■TUH|xXY\tIj■m■zS3g"@■|0D=TD%■.,t■■■XKP
 ca■

■
"■h■4^■■■a■S■_?■0Ke=\Y:kX■Y■HFC■aF6■iJ)■■h4$w,#W+■|9■zsZ■P-■.:u*■GGGsgE'pA45IU'6■f1Jp\C■y■■Q■,N■s
■yR^;}m■■X*mS■■■)■■■GGlC$%5pkY■■xr■5YUvH■■L■\(■1■g[:■q■4.■/,■{■=■2S\■2Q■|(<v■i■■oR■X■jB~■)<Yxm■
■■■■■■■■■■■■■■■■■■ ■■■i ■■idea.pyPK■■■■■■■■■■■s[h.■■■■■■■■■■■■■■■■■■ ■■■z%■■ideaBookmark.pyPK
■■■■■■■■■■ ■■■g:■■models.rarPK■■■■■■■■■■■■s[■=&0■■■a■■■■■■■■■■■■■■ ■■■■■■notification.pyPK■■■■
■■■■■■■■■■ ■■■■■■startup.pyPK■■■■■■■■■■■Ds[g■;■■■V■■■■■■■■■■■■■■■ ■■■■■■startupBookmark.pyPK■■■■
g■■■■■■■■■■■■■■■■■■■ ■■■■■■■story.pyPK■■■■■■■■■■■■■s[5■■■■■■■■■■■■■■■■■■■ ■■■■■■storyView.pyPK■■■
s[■■w■■■Z■■■
■■■■■■■■■■ ■■■■■■■suggestion.pyPK■■■■■■■■■■■Ls[
F[■■■f■■■■■■■■■■■■■■■ ■■■:■■■task.pyPK■■■■■■■■■■■es[■z■■■■■f■■■
■■■■■■■■■■ ■■■■■■■teamMember.pyPK■■■■■■■■■■■s[■u*■■■o
■■■■■■■■■■■■■■ ■■■
■■teamPerformance.pyPK■■■■■■■■■■■s[A@Y■■■I■■■■■■■■■■■■■■ ■■■N■■■user.pyPK■■■■■■■■■■■Ys[w■■2■■■
■■■■■■■■■■■■■■ ■■■■ ■■userAchievement.pyPK■■■■■■■%■%■■■■■.$■■■■■

20 — notification.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum, JSON
from .Enums import SuggestionStatus

class Notification(db.Model):
    __tablename__ = 'notifications'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    type = db.Column(db.String(50), default='system')
    title = db.Column(db.String(255))
    message = db.Column(db.Text)
    data = db.Column(JSON, default={})
    is_read = db.Column(db.Boolean, default=False)
    read_at = db.Column(db.DateTime, nullable=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    user = db.relationship('User', backref='notifications_list', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def mark_as_read(self):
        """Mark notification as read"""
        self.is_read = True
        self.read_at = datetime.utcnow()
        db.session.commit()

    def mark_as_unread(self):
        """Mark notification as unread"""
        self.is_read = False
        self.read_at = None
        db.session.commit()

    def is_recent(self, hours=24):
        """Check if notification is recent"""
        time_diff = datetime.utcnow() - self.created_at
        return time_diff.total_seconds() <= hours * 3600

    def get_related_data(self, key=None):
        """Get related data from notification"""
        if key:
            return self.data.get(key)
        return self.data

    def to_dict(self):
```

```
        return {
            'id': self.id,
            'userId': self.user_id,
            'type': self.type,
            'title': self.title,
            'message': self.message,
            'data': self.data or {},
            'isRead': self.is_read,
            'readAt': self.read_at.isoformat() if self.read_at else None,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'isRecent': self.is_recent(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name
            } if self.user else None
        }
```

21 — post.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum, JSON
from .Enums import PostType
from .postComment import PostComment

class Post(db.Model):
    __tablename__ = 'posts'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    content = db.Column(db.Text, nullable=False)
    type = db.Column(Enum(PostType), default=PostType.professional)
    tags = db.Column(JSON, default=[])

    likes = db.Column(db.Integer, default=0)
    comments_count = db.Column(db.Integer, default=0)
    shares = db.Column(db.Integer, default=0)
    saves = db.Column(db.Integer, default=0)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships - ALL CORRECTED
    # user and author relationships defined in User model

    post_likes = db.relationship('PostLike',
        backref='liked_post',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='PostLike.post_id')

    post_comments = db.relationship('PostComment',
        backref='parent_post',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='PostComment.post_id')

    media_items = db.relationship('PostMedia',
        backref='parent_post',
        lazy='dynamic',
```

```
        cascade='all, delete-orphan',
        foreign_keys='PostMedia.post_id')

    # HELPER FUNCTIONS
    def increment_likes(self):
        """Increment like count"""
        self.likes += 1
        db.session.commit()

    def decrement_likes(self):
        """Decrement like count"""
        if self.likes > 0:
            self.likes -= 1
        db.session.commit()

    def increment_comments(self):
        """Increment comment count"""
        self.comments_count += 1
        db.session.commit()

    def decrement_comments(self):
        """Decrement comment count"""
        if self.comments_count > 0:
            self.comments_count -= 1
        db.session.commit()

    def increment_shares(self):
        """Increment share count"""
        self.shares += 1
        db.session.commit()

    def increment_saves(self):
        """Increment save count"""
        self.saves += 1
        db.session.commit()

    def add_tag(self, tag):
        """Add tag to post"""
        if not self.tags:
            self.tags = []
        if tag not in self.tags:
            self.tags.append(tag)
        db.session.commit()

    def remove_tag(self, tag):
        """Remove tag from post"""
        if self.tags and tag in self.tags:
            self.tags.remove(tag)
        db.session.commit()

    def is_liked_by_user(self, user_id):
        """Check if post is liked by user"""
        return self.post_likes.filter_by(user_id=user_id).first() is not None

    def get_recent_comments(self, limit=5):
        """Get recent comments"""
        return self.post_comments.order_by(PostComment.created_at.desc()).limit(limit).all()

    def get_media_count(self):
        """Get number of media items"""
        return self.media_items.count()

    def to_dict(self, include_comments=False, include_media=False, user_id=None):
        data = {
            'id': self.id,
            'userId': self.user_id,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'profilePicture': self.post_author.profile_picture if self.post_author else None
            },
```

```
            'content': self.content,
            'type': self.type.value,
            'tags': self.tags or [],
            'likes': self.likes,
            'commentsCount': self.comments_count,
            'shares': self.shares,
            'saves': self.saves,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'mediaCount': self.get_media_count()
        }

        if user_id:
            data['isLiked'] = self.is_liked_by_user(user_id)

        if include_comments:
            data['comments'] = [comment.to_dict() for comment in self.get_recent_comments()]

        if include_media:
            data['media'] = [media.to_dict() for media in self.media_items.all()]

        return data
```

22 — postComment.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum
from .Enums import SuggestionStatus

class PostComment(db.Model):
    __tablename__ = 'post_comments'

    id = db.Column(db.Integer, primary_key=True)
    post_id = db.Column(db.Integer, db.ForeignKey('posts.id', ondelete='CASCADE'), nullable=False)
    content = db.Column(db.Text, nullable=False)

    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    post = db.relationship('Post', backref='comments_list', foreign_keys=[post_id])
    author = db.relationship('User', backref='post_comments_list', foreign_keys=[author_id])

    # HELPER FUNCTIONS

    def update_content(self, new_content):
        """Update comment content"""
        self.content = new_content
        db.session.commit()

    def get_author_name(self):
        """Get author's full name"""
        return f"{self.author_first_name} {self.author_last_name}"

    def get_post_content_preview(self):
        """Get post content preview"""
        if self.post:
            return self.post.content[:100] + '...' if len(self.post.content) > 100 else self.post.conten
        return None

    def is_recent(self, hours=24):
```

```
        """Check if comment is recent"""
        time_diff = datetime.utcnow() - self.created_at
        return time_diff.total_seconds() <= hours * 3600

    def to_dict(self):
        return {
            'id': self.id,
            'postId': self.post_id,
            'content': self.content,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'fullName': self.get_author_name(),
                'profilePicture': self.author.profile_picture if self.author else None
            },
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'isRecent': self.is_recent(),
            'postContentPreview': self.get_post_content_preview()
        }
```

23 — postLike.py

```
from datetime import datetime
from app.extensions import db

class PostLike(db.Model):
    __tablename__ = 'post_likes'

    id = db.Column(db.Integer, primary_key=True)
    post_id = db.Column(db.Integer, db.ForeignKey('posts.id', ondelete='CASCADE'), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    liked_at = db.Column(db.DateTime, default=datetime.utcnow)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    post = db.relationship('Post', backref='likes_list', foreign_keys=[post_id])
    user = db.relationship('User', backref='post_likes_list', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def get_time_since_like(self):
        """Get time since like was created"""
        return datetime.utcnow() - self.liked_at

    def is_recent(self, minutes=30):
        """Check if like is recent"""
        time_diff = datetime.utcnow() - self.liked_at
        return time_diff.total_seconds() <= minutes * 60

    def to_dict(self):
        return {
            'id': self.id,
            'postId': self.post_id,
            'userId': self.user_id,
            'likedAt': self.liked_at.isoformat(),
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'timeSinceLike': str(self.get_time_since_like()),
            'isRecent': self.is_recent(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
```

```
                'lastName': self.user.last_name,
                'profilePicture': self.user.profile_picture
            } if self.user else None,
            'post': {
                'id': self.post.id,
                'content': self.post.content[:100] + '...' if len(self.post.content) > 100 else self.pos
            } if self.post else None
        }
```

24 — postMedia.py

```
from datetime import datetime
from app.extensions import db

class PostMedia(db.Model):
    __tablename__ = 'post_media'

    id = db.Column(db.Integer, primary_key=True)
    post_id = db.Column(db.Integer, db.ForeignKey('posts.id', ondelete='CASCADE'), nullable=False)
    data = db.Column(db.LargeBinary)
    content_type = db.Column(db.String(100))
    file_name = db.Column(db.String(255))
    file_size = db.Column(db.Integer)  # Size in bytes
    caption = db.Column(db.Text)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    post = db.relationship('Post', backref='media_list', foreign_keys=[post_id])

    # HELPER FUNCTIONS

    def get_file_size_formatted(self):
        """Get formatted file size"""
        if self.file_size < 1024:
            return f"{self.file_size} B"
        elif self.file_size < 1024 * 1024:
            return f"{self.file_size / 1024:.1f} KB"
        else:
            return f"{self.file_size / (1024 * 1024):.1f} MB"

    def is_image(self):
        """Check if media is an image"""
        return self.content_type and self.content_type.startswith('image/')

    def is_video(self):
        """Check if media is a video"""
        return self.content_type and self.content_type.startswith('video/')

    def is_document(self):
        """Check if media is a document"""
        return self.content_type and (
            self.content_type.startswith('application/') or
            self.content_type in ['text/plain', 'text/html']
        )

    def get_media_type(self):
        """Get media type category"""
        if self.is_image():
            return 'image'
        elif self.is_video():
            return 'video'
        elif self.is_document():
            return 'document'
        else:
```

```
        return 'other'

    def update_caption(self, new_caption):
        """Update media caption"""
        self.caption = new_caption
        db.session.commit()

    def to_dict(self):
        return {
            'id': self.id,
            'postId': self.post_id,
            'contentType': self.content_type,
            'fileName': self.file_name,
            'fileSize': self.file_size,
            'fileSizeFormatted': self.get_file_size_formatted(),
            'caption': self.caption,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'mediaType': self.get_media_type(),
            'isImage': self.is_image(),
            'isVideo': self.is_video(),
            'isDocument': self.is_document(),
            'post': {
                'id': self.post.id,
                'content': self.post.content[:100] + '...' if len(self.post.content) > 100 else self.pos
            } if self.post else None
        }
```

25 — projectGoal.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db
from .goalMilstone import GoalMilestone


class ProjectGoal(db.Model):
    __tablename__ = 'project_goals'

    id = db.Column(db.Integer, primary_key=True)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id'), nullable=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    title = db.Column(db.String(255), nullable=False)
    description = db.Column(db.Text)

    progress_percentage = db.Column(db.Float, default=0)  # 78%
    milestones_completed = db.Column(db.Integer, default=0)  # 8
    milestones_total = db.Column(db.Integer, default=0)  # 10

    is_on_track = db.Column(db.Boolean, default=True)
    next_milestone = db.Column(db.String(255))  # "MVP Launch"

    target_date = db.Column(db.DateTime)
    completed_date = db.Column(db.DateTime, nullable=True)
    status = db.Column(Enum('active', 'completed', 'on_hold', 'cancelled'), default='active')

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    user = db.relationship('User', backref='project_goals_list', foreign_keys=[user_id])
    startup = db.relationship('Startup', backref='project_goals_list', foreign_keys=[startup_id])
    milestones_list = db.relationship('GoalMilestone', backref='project_goal', lazy='dynamic', cascade='
    # HELPER FUNCTIONS
```

```
    def update_progress(self, completed_milestones=None, total_milestones=None):
        """Update progress percentage based on milestones"""
        if completed_milestones is not None:
            self.milestones_completed = completed_milestones
        if total_milestones is not None:
            self.milestones_total = total_milestones

        if self.milestones_total > 0:
            self.progress_percentage = (self.milestones_completed / self.milestones_total) * 100
        else:
            self.progress_percentage = 0

        # Update track status
        self.is_on_track = self._check_if_on_track()

        # Auto-complete if 100%
        if self.progress_percentage >= 100 and self.status != 'completed':
            self.complete_goal()

        db.session.commit()

    def _check_if_on_track(self):
        """Check if goal is on track based on target date"""
        if not self.target_date or self.status != 'active':
            return True

        today = datetime.utcnow()
        days_passed = (today - self.created_at).days
        total_days = (self.target_date - self.created_at).days

        if total_days <= 0:
            return self.progress_percentage >= 100

        expected_progress = (days_passed / total_days) * 100
        return self.progress_percentage >= expected_progress

    def complete_goal(self):
        """Mark goal as completed"""
        self.status = 'completed'
        self.progress_percentage = 100
        self.milestones_completed = self.milestones_total
        self.completed_date = datetime.utcnow()
        self.is_on_track = True

    def add_milestone(self, title, description="", order=None):
        """Add a milestone to the goal"""


        if order is None:
            order = self.milestones_list.count() + 1

        milestone = GoalMilestone(
            goal_id=self.id,
            user_id=self.user_id,
            title=title,
            description=description,
            order=order
        )

        db.session.add(milestone)
        self.milestones_total += 1
        self.update_progress()

        return milestone

    def complete_milestone(self):
        """Complete a milestone and update progress"""
        if self.milestones_completed < self.milestones_total:
            self.milestones_completed += 1
            self.update_progress()

    def change_status(self, new_status):
```

```
        """Change goal status"""
        valid_statuses = ['active', 'completed', 'on_hold', 'cancelled']
        if new_status in valid_statuses:
            self.status = new_status
            if new_status == 'completed':
                self.complete_goal()

            db.session.commit()

    def get_milestones(self, completed_only=False):
        """Get milestones with optional filtering"""
        query = self.milestones_list
        if completed_only:
            query = query.filter_by(is_completed=True)
        return query.order_by(GoalMilestone.order).all()

    def to_dict(self, include_milestones=False):
        data = {
            'id': self.id,
            'startup_id': self.startup_id,
            'user_id': self.user_id,
            'title': self.title,
            'description': self.description,
            'progress_percentage': self.progress_percentage,
            'milestones_completed': self.milestones_completed,
            'milestones_total': self.milestones_total,
            'is_on_track': self.is_on_track,
            'next_milestone': self.next_milestone,
            'target_date': self.target_date.isoformat() if self.target_date else None,
            'completed_date': self.completed_date.isoformat() if self.completed_date else None,
            'status': self.status,
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'days_remaining': self._get_days_remaining()
        }

        if include_milestones:
            data['milestones'] = [milestone.to_dict() for milestone in self.get_milestones()]

        return data

    def _get_days_remaining(self):
        """Calculate days remaining until target date"""
        if not self.target_date or self.status == 'completed':
            return 0

        today = datetime.utcnow()
        remaining = (self.target_date - today).days
        return max(0, remaining)
```

26 — refreshToken.py

```
from datetime import datetime
from app.extensions import db

class RefreshToken(db.Model):
    __tablename__ = 'refresh_tokens'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    token = db.Column(db.String(500), nullable=False)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

27 — startUpMember.py

```
from datetime import datetime
from app.extensions import db

class StartupMember(db.Model):
    __tablename__ = 'startup_members'

    id = db.Column(db.Integer, primary_key=True)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id', ondelete='CASCADE'), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    first_name = db.Column(db.String(100))
    last_name = db.Column(db.String(100))
    role = db.Column(db.String(100), default='member')
    joined_at = db.Column(db.DateTime, default=datetime.utcnow)
    is_active = db.Column(db.Boolean, default=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    # user = db.relationship('User', backref='startup_memberships')
    startup = db.relationship('Startup', backref='startup_members_list')

    # HELPER FUNCTIONS

    def update_role(self, new_role):
        """Update member role"""
        self.role = new_role
        db.session.commit()

    def deactivate(self):
        """Deactivate membership"""
        self.is_active = False
        self.startup.update_member_count()

    def activate(self):
        """Activate membership"""
        self.is_active = True
        self.startup.update_member_count()

    def get_full_name(self):
        """Get member's full name"""
        return f"{self.first_name} {self.last_name}"

    def to_dict(self):
        return {
            'id': self.id,
            'startupId': self.startup_id,
            'userId': self.user_id,
            'firstName': self.first_name,
            'lastName': self.last_name,
            'role': self.role,
            'joinedAt': self.joined_at.isoformat(),
            'isActive': self.is_active,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'fullName': self.get_full_name()
        }
```

28 — startup.py

```python
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum, JSON
from .Enums import StartupStage

class Startup(db.Model):
    __tablename__ = 'startups'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), unique=True, nullable=False)
    industry = db.Column(db.String(100), nullable=False)
    location = db.Column(db.String(255))
    description = db.Column(db.Text)
    stage = db.Column(Enum(StartupStage), default=StartupStage.idea)

    logo_data = db.Column(db.LargeBinary)
    logo_content_type = db.Column(db.String(100))
    banner_data = db.Column(db.LargeBinary)
    banner_content_type = db.Column(db.String(100))

    positions = db.Column(db.Integer, default=0)
    roles = db.Column(JSON, default={})

    creator_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    creator_first_name = db.Column(db.String(100))
    creator_last_name = db.Column(db.String(100))

    status = db.Column(db.String(50), default='active')
    member_count = db.Column(db.Integer, default=1)
    views = db.Column(db.Integer, default=0)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships - ALL CORRECTED
    # creator relationship defined in User model as 'startup_creator'

    members = db.relationship('StartupMember',
        backref='member_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='StartupMember.startup_id')

    join_requests = db.relationship('JoinRequest',
        backref='target_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='JoinRequest.startup_id')

    startup_bookmarks = db.relationship('StartupBookmark',
        backref='bookmarked_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='StartupBookmark.startup_id')

    startup_tasks = db.relationship('Task',
        backref='parent_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='Task.startup_id')

    project_goals = db.relationship('ProjectGoal',
        backref='parent_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='ProjectGoal.startup_id')

    team_performance_records = db.relationship('TeamPerformance',
        backref='parent_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='TeamPerformance.startup_id')
```

```
    growth_metrics_startup = db.relationship('GrowthMetric',
        backref='parent_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='GrowthMetric.startup_id')

    calendar_events = db.relationship('CalendarEvent',
        backref='parent_startup',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='CalendarEvent.startup_id')

    # HELPER FUNCTIONS
    def increment_views(self):
        """Increment view count"""
        self.views += 1
        db.session.commit()

    def update_member_count(self):
        """Update member count based on active members"""
        active_members = self.members.filter_by(is_active=True).count()
        self.member_count = active_members
        db.session.commit()

    def add_member(self, user_id, first_name, last_name, role='member'):
        """Add a new member to the startup"""
        from models.startUpMember import StartupMember
        member = StartupMember(
            startup_id=self.id,
            user_id=user_id,
            first_name=first_name,
            last_name=last_name,
            role=role
        )
        db.session.add(member)
        self.update_member_count()
        return member

    def remove_member(self, user_id):
        """Remove a member from the startup"""
        member = self.members.filter_by(user_id=user_id, is_active=True).first()
        if member:
            member.is_active = False
            self.update_member_count()

    def update_stage(self, new_stage):
        """Update startup stage"""
        self.stage = StartupStage(new_stage)
        db.session.commit()

    def add_position(self, role_name, count=1):
        """Add available positions"""
        if not self.roles:
            self.roles = {}
        if role_name not in self.roles:
            self.roles[role_name] = 0
        self.roles[role_name] += count
        self.positions += count
        db.session.commit()

    def fill_position(self, role_name):
        """Fill an available position"""
        if self.roles and role_name in self.roles and self.roles[role_name] > 0:
            self.roles[role_name] -= 1
            self.positions -= 1
            db.session.commit()

    def get_active_members(self):
        """Get list of active members"""
        return self.members.filter_by(is_active=True).all()

    def to_dict(self):
```

```
        return {
            'id': self.id,
            'name': self.name,
            'industry': self.industry,
            'location': self.location,
            'description': self.description,
            'stage': self.stage.value,
            'positions': self.positions,
            'roles': self.roles or {},
            'creator': {
                'id': self.creator_id,
                'firstName': self.creator_first_name,
                'lastName': self.creator_last_name
            },
            'status': self.status,
            'memberCount': self.member_count,
            'views': self.views,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat()
        }
```

29 — startupBookmark.py

```
from datetime import datetime
from app.extensions import db

class StartupBookmark(db.Model):
    __tablename__ = 'startup_bookmarks'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id', ondelete='CASCADE'), nullable=False)
    title = db.Column(db.String(255))
    content_preview = db.Column(db.Text)
    url = db.Column(db.String(500))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    user = db.relationship('User', backref='startup_bookmarks')
    startup = db.relationship('Startup', backref='bookmarked_by')

    # HELPER FUNCTIONS

    def update_content(self, title=None, content_preview=None, url=None):
        """Update bookmark content"""
        if title:
            self.title = title
        if content_preview:
            self.content_preview = content_preview
        if url:
            self.url = url
        db.session.commit()

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'startup_id': self.startup_id,
            'title': self.title,
            'content_preview': self.content_preview,
            'url': self.url,
            'created_at': self.created_at.isoformat(),
            'startup': {
                'id': self.startup.id,
                'name': self.startup.name,
                'industry': self.startup.industry
            } if self.startup else None
```

```
        }
```

30 — story.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum
from .Enums import StoryType
from .storyView import StoryView

class Story(db.Model):
    __tablename__ = 'stories'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    media_url = db.Column(db.String(500), nullable=False)
    caption = db.Column(db.Text)
    type = db.Column(Enum(StoryType), default=StoryType.image)

    views = db.Column(db.Integer, default=0)
    expires_at = db.Column(db.DateTime, nullable=False)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships - ALL CORRECTED
    # user and author relationships defined in User model

    story_views = db.relationship('StoryView',
        backref='viewed_story',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='StoryView.story_id')

    # HELPER FUNCTIONS
    def increment_views(self):
        """Increment view count"""
        self.views += 1
        db.session.commit()

    def is_expired(self):
        """Check if story has expired"""
        return datetime.utcnow() > self.expires_at

    def get_time_until_expiry(self):
        """Get time until story expires"""
        return self.expires_at - datetime.utcnow()

    def is_active(self):
        """Check if story is still active (not expired)"""
        return not self.is_expired()

    def get_viewers_count(self):
        """Get number of unique viewers"""
        return self.story_views.count()

    def get_recent_viewers(self, limit=10):
        """Get recent viewers"""
        return self.story_views.order_by(StoryView.viewed_at.desc()).limit(limit).all()

    def has_user_viewed(self, user_id):
        """Check if user has viewed this story"""
```

```
        return self.story_views.filter_by(user_id=user_id).first() is not None

    def to_dict(self, include_viewers=False, user_id=None):
        data = {
            'id': self.id,
            'userId': self.user_id,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'profilePicture': self.story_author.profile_picture if self.story_author else None
            },
            'mediaUrl': self.media_url,
            'caption': self.caption,
            'type': self.type.value,
            'views': self.views,
            'expiresAt': self.expires_at.isoformat(),
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'isExpired': self.is_expired(),
            'isActive': self.is_active(),
            'timeUntilExpiry': str(self.get_time_until_expiry()) if not self.is_expired() else None,
            'uniqueViewers': self.get_viewers_count()
        }

        if user_id:
            data['hasViewed'] = self.has_user_viewed(user_id)

        if include_viewers:
            data['recentViewers'] = [view.to_dict() for view in self.get_recent_viewers()]

        return data
```

31 — storyView.py

```
from datetime import datetime
from app.extensions import db

class StoryView(db.Model):
    __tablename__ = 'story_views'

    id = db.Column(db.Integer, primary_key=True)
    story_id = db.Column(db.Integer, db.ForeignKey('stories.id', ondelete='CASCADE'), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    viewed_at = db.Column(db.DateTime, default=datetime.utcnow)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    story = db.relationship('Story', backref='views_list', foreign_keys=[story_id])
    user = db.relationship('User', backref='story_views_list', foreign_keys=[user_id])

    # HELPER FUNCTIONS

    def get_time_since_view(self):
        """Get time since view was created"""
        return datetime.utcnow() - self.viewed_at

    def is_recent(self, minutes=30):
        """Check if view is recent"""
        time_diff = datetime.utcnow() - self.viewed_at
        return time_diff.total_seconds() <= minutes * 60

    def get_story_details(self):
        """Get story details"""
```

```
        if self.story:
            return {
                'media_url': self.story.media_url,
                'caption': self.story.caption,
                'type': self.story.type.value,
                'author': f"{self.story.author_first_name} {self.story.author_last_name}"
            }
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'story_id': self.story_id,
            'user_id': self.user_id,
            'viewed_at': self.viewed_at.isoformat(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'time_since_view': str(self.get_time_since_view()),
            'is_recent': self.is_recent(),
            'story': self.get_story_details(),
            'user': {
                'id': self.user.id,
                'firstName': self.user.first_name,
                'lastName': self.user.last_name,
                'profilePicture': self.user.profile_picture
            } if self.user else None
        }
```

32 — suggestion.py

```
from datetime import datetime
from app.extensions import db
from sqlalchemy import Enum
from .Enums import SuggestionStatus

class Suggestion(db.Model):
    __tablename__ = 'suggestions'

    id = db.Column(db.Integer, primary_key=True)
    idea_id = db.Column(db.Integer, db.ForeignKey('ideas.id', ondelete='CASCADE'), nullable=False)
    content = db.Column(db.Text, nullable=False)

    author_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    author_first_name = db.Column(db.String(100))
    author_last_name = db.Column(db.String(100))

    status = db.Column(Enum(SuggestionStatus), default=SuggestionStatus.pending)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    idea = db.relationship('Idea', backref='suggestions', foreign_keys=[idea_id])
    author = db.relationship('User', backref='suggestions', foreign_keys=[author_id])

    # HELPER FUNCTIONS

    def update_status(self, new_status):
        """Update suggestion status"""
        self.status = SuggestionStatus(new_status)
        db.session.commit()

    def approve(self):
        """Approve suggestion"""
        self.update_status('approved')
    def reject(self):
```

```
        """Reject suggestion"""
        self.update_status('rejected')

    def get_author_name(self):
        """Get author's full name"""
        return f"{self.author_first_name} {self.author_last_name}"

    def get_idea_details(self):
        """Get idea details"""
        if self.idea:
            return {
                'title': self.idea.title,
                'description': self.idea.description,
                'industry': self.idea.industry,
                'stage': self.idea.stage
            }
        return None

    def is_pending(self):
        """Check if suggestion is pending"""
        return self.status == SuggestionStatus.pending

    def is_approved(self):
        """Check if suggestion is approved"""
        return self.status == SuggestionStatus.approved

    def is_rejected(self):
        """Check if suggestion is rejected"""
        return self.status == SuggestionStatus.rejected

    def to_dict(self):
        return {
            'id': self.id,
            'ideaId': self.idea_id,
            'content': self.content,
            'author': {
                'id': self.author_id,
                'firstName': self.author_first_name,
                'lastName': self.author_last_name,
                'fullName': self.get_author_name(),
                'profilePicture': self.author.profile_picture if self.author else None
            },
            'status': self.status.value,
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'isPending': self.is_pending(),
            'isApproved': self.is_approved(),
            'isRejected': self.is_rejected(),
            'idea': self.get_idea_details()
        }
```

33 — task.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum, JSON
from app.extensions import db

class Task(db.Model):
    __tablename__ = 'tasks'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id'), nullable=True)

    title = db.Column(db.String(255), nullable=False)
```

```
description = db.Column(db.Text)
priority = db.Column(Enum('low', 'medium', 'high'), default='medium')
status = db.Column(Enum('today', 'in_progress', 'completed', 'overdue'), default='today')

tags = db.Column(JSON, default=[])
labels = db.Column(JSON, default=[])

due_date = db.Column(db.DateTime, nullable=True)
completed_date = db.Column(db.DateTime, nullable=True)
estimated_hours = db.Column(db.Float, nullable=True)
actual_hours = db.Column(db.Float, nullable=True)

assigned_to = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True)
created_by = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

is_on_time = db.Column(db.Boolean, default=True)
progress_percentage = db.Column(db.Integer, default=0)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

# Relationships - ALL CORRECTED
# All three relationships defined in User model with unique backrefs:
# - task_owner (user_id)
# - task_creator (created_by)
# - task_assignee (assigned_to)

# Startup relationship defined in Startup model as 'parent_startup'

# HELPER FUNCTIONS
def update_status(self, new_status):
    """Update task status and handle completion"""
    self.status = new_status
    if new_status == 'completed':
        self.completed_date = datetime.utcnow()
        self.progress_percentage = 100
        self.check_if_on_time()
    elif new_status == 'in_progress':
        self.progress_percentage = 50
    db.session.commit()

def update_progress(self, percentage):
    """Update progress percentage"""
    self.progress_percentage = max(0, min(100, percentage))
    if percentage >= 100:
        self.update_status('completed')
    elif percentage > 0:
        self.update_status('in_progress')
    db.session.commit()

def check_if_on_time(self):
    """Check if task was completed on time"""
    if self.due_date and self.completed_date:
        self.is_on_time = self.completed_date <= self.due_date
    elif self.due_date and datetime.utcnow() > self.due_date:
        self.is_on_time = False
        self.status = 'overdue'
    db.session.commit()

def assign_to_user(self, user_id):
    """Assign task to user"""
    self.assigned_to = user_id
    db.session.commit()

def add_tag(self, tag):
    """Add tag to task"""
    if not self.tags:
        self.tags = []
    if tag not in self.tags:
        self.tags.append(tag)
    db.session.commit()
def add_label(self, label, color=None):
```

```
        """Add label to task"""
        if not self.labels:
            self.labels = []
        label_data = {'name': label}
        if color:
            label_data['color'] = color
        self.labels.append(label_data)
        db.session.commit()

    def log_time(self, hours):
        """Log actual hours worked"""
        if not self.actual_hours:
            self.actual_hours = 0
        self.actual_hours += hours
        db.session.commit()

    def is_overdue(self):
        """Check if task is overdue"""
        if self.due_date and self.status != 'completed':
            return datetime.utcnow() > self.due_date
        return False

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'startup_id': self.startup_id,
            'title': self.title,
            'description': self.description,
            'priority': self.priority,
            'status': self.status,
            'tags': self.tags or [],
            'labels': self.labels or [],
            'due_date': self.due_date.isoformat() if self.due_date else None,
            'completed_date': self.completed_date.isoformat() if self.completed_date else None,
            'estimated_hours': self.estimated_hours,
            'actual_hours': self.actual_hours,
            'assigned_to': self.assigned_to,
            'created_by': self.created_by,
            'is_on_time': self.is_on_time,
            'progress_percentage': self.progress_percentage,
            'is_overdue': self.is_overdue(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat(),
            'assigned_user': {
                'id': self.task_assignee.id,
                'firstName': self.task_assignee.first_name,
                'lastName': self.task_assignee.last_name
            } if self.task_assignee else None
        }
```

34 — teamMember.py

```
from app.extensions import db
from sqlalchemy import JSON

class TeamMember(db.Model):
    __tablename__ = 'team_members'

    id = db.Column(db.Integer, primary_key=True)
    idea_id = db.Column(db.Integer, db.ForeignKey('ideas.id', ondelete='CASCADE'), nullable=False)
    name = db.Column(db.String(255))
    position = db.Column(db.String(255))
    skills = db.Column(JSON, default={})

    # Relationships
```

```
    idea = db.relationship('Idea', backref='team_members')

    # HELPER FUNCTIONS

    def update_skills(self, new_skills):
        """Update team member skills"""
        self.skills = new_skills
        db.session.commit()

    def add_skill(self, skill_name, proficiency='intermediate'):
        """Add a skill to team member"""
        if not self.skills:
            self.skills = {}
        self.skills[skill_name] = proficiency
        db.session.commit()

    def remove_skill(self, skill_name):
        """Remove a skill from team member"""
        if self.skills and skill_name in self.skills:
            del self.skills[skill_name]
            db.session.commit()

    def to_dict(self):
        return {
            'id': self.id,
            'idea_id': self.idea_id,
            'name': self.name,
            'position': self.position,
            'skills': self.skills or {}
        }
```

35 — teamPerformance.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class TeamPerformance(db.Model):
    __tablename__ = 'team_performance'

    id = db.Column(db.Integer, primary_key=True)
    startup_id = db.Column(db.Integer, db.ForeignKey('startups.id'), nullable=False)

    score_percentage = db.Column(db.Float, default=0)  # 92%
    active_members = db.Column(db.Integer, default=0)  # 12
    tasks_completed = db.Column(db.Integer, default=0)  # 45
    productivity_level = db.Column(Enum('low', 'medium', 'high'), default='medium')

    period_start = db.Column(db.DateTime, nullable=False)
    period_end = db.Column(db.DateTime, nullable=False)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    startup = db.relationship('Startup', backref='team_performance_records')

    # HELPER FUNCTIONS

    def update_score(self, new_score):
        """Update performance score and adjust productivity level"""
        self.score_percentage = new_score

        if new_score >= 80:
            self.productivity_level = 'high'
```

```
        elif new_score >= 60:
            self.productivity_level = 'medium'
        else:
            self.productivity_level = 'low'

        db.session.commit()

    def increment_tasks_completed(self, count=1):
        """Increment completed tasks count"""
        self.tasks_completed += count
        db.session.commit()

    def update_active_members(self, active_count):
        """Update active members count"""
        self.active_members = active_count
        db.session.commit()

    def is_current_period(self):
        """Check if this performance record is for the current period"""
        now = datetime.utcnow()
        return self.period_start <= now <= self.period_end

    def to_dict(self):
        return {
            'id': self.id,
            'startup_id': self.startup_id,
            'score_percentage': self.score_percentage,
            'active_members': self.active_members,
            'tasks_completed': self.tasks_completed,
            'productivity_level': self.productivity_level,
            'period_start': self.period_start.isoformat(),
            'period_end': self.period_end.isoformat(),
            'is_current_period': self.is_current_period(),
            'created_at': self.created_at.isoformat(),
            'updated_at': self.updated_at.isoformat()
        }
```

36 — user.py

```
from datetime import datetime, timedelta
from sqlalchemy import Enum, JSON
from app.extensions import db
from .Enums import UserStatus, Privacy, Theme, EmailDigest
from werkzeug.security import generate_password_hash, check_password_hash

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    first_name = db.Column(db.String(100), nullable=False)
    last_name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(255), unique=True, nullable=False, index=True)
    password = db.Column(db.String(255), nullable=False)
    is_email_verified = db.Column(db.Boolean, default=False)
    last_login = db.Column(db.DateTime, nullable=True)
    status = db.Column(Enum(UserStatus), default=UserStatus.active)
    role = db.Column(db.String(50), default='member')
    xp_points = db.Column(db.Integer, default=0)
    streak_days = db.Column(db.Integer, default=0)
    last_activity_date = db.Column(db.Date)

    # Computed/cached stats
    total_revenue = db.Column(db.Float, default=0.0)
    satisfaction_percentage = db.Column(db.Float, default=100.0)
    active_startups_count = db.Column(db.Integer, default=0)
    # Profile
```

```
    profile_picture = db.Column(db.String(500), nullable=True)
    profile_bio = db.Column(db.Text, nullable=True)
    profile_company = db.Column(db.String(255), nullable=True)
    profile_social_links = db.Column(JSON, default={})
    profile_country = db.Column(db.String(100), nullable=True)
    profile_city = db.Column(db.String(100), nullable=True)
    profile_timezone = db.Column(db.String(50), nullable=True)

    # Preferences
    pref_email_notifications = db.Column(db.Boolean, default=True)
    pref_push_notifications = db.Column(db.Boolean, default=True)
    pref_privacy = db.Column(Enum(Privacy), default=Privacy.public)
    pref_language = db.Column(db.String(10), default='en')
    pref_timezone = db.Column(db.String(50), default='UTC')
    pref_theme = db.Column(Enum(Theme), default=Theme.light)

    # Notification Settings
    notif_new_comments = db.Column(db.Boolean, default=True)
    notif_new_likes = db.Column(db.Boolean, default=True)
    notif_new_suggestions = db.Column(db.Boolean, default=True)
    notif_join_requests = db.Column(db.Boolean, default=True)
    notif_approvals = db.Column(db.Boolean, default=True)
    notif_story_views = db.Column(db.Boolean, default=True)
    notif_post_engagement = db.Column(db.Boolean, default=True)
    notif_email_digest = db.Column(Enum(EmailDigest), default=EmailDigest.weekly)
    notif_quiet_hours_enabled = db.Column(db.Boolean, default=False)
    notif_quiet_hours_start = db.Column(db.String(5), default='22:00')
    notif_quiet_hours_end = db.Column(db.String(5), default='08:00')

    # Timestamps
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # ========== RELATIONSHIPS ==========

    # Core Authentication & Profile
    refresh_tokens = db.relationship('RefreshToken',
        backref='token_owner',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='RefreshToken.user_id')

    notifications = db.relationship('Notification',
        backref='notification_owner',
        lazy='dynamic',
        cascade='all, delete-orphan',
        foreign_keys='Notification.user_id')

    # Content Creation
    knowledge_posts = db.relationship('Knowledge',
        backref='knowledge_author',
        lazy='dynamic',
        foreign_keys='Knowledge.author_id',
        cascade='all, delete-orphan')

    ideas = db.relationship('Idea',
        backref='idea_creator',
        lazy='dynamic',
        foreign_keys='Idea.creator_id',
        cascade='all, delete-orphan')

    startups = db.relationship('Startup',
        backref='startup_creator',
        lazy='dynamic',
        foreign_keys='Startup.creator_id',
        cascade='all, delete-orphan')

    stories = db.relationship('Story',
        backref='story_author',
        lazy='dynamic',
        foreign_keys='Story.author_id',
        cascade='all, delete-orphan')
```

```
user_posts = db.relationship('Post',
    backref='post_owner',
    lazy='dynamic',
    foreign_keys='Post.user_id',
    cascade='all, delete-orphan')

authored_posts = db.relationship('Post',
    backref='post_author',
    lazy='dynamic',
    foreign_keys='Post.author_id')

# Comments & Engagement
idea_comments = db.relationship('IdeaComment',
    backref='comment_author',
    lazy='dynamic',
    foreign_keys='IdeaComment.author_id',
    cascade='all, delete-orphan')

knowledge_comments = db.relationship('KnowledgeComment',
    backref='comment_author',
    lazy='dynamic',
    foreign_keys='KnowledgeComment.author_id',
    cascade='all, delete-orphan')

post_comments_list = db.relationship('PostComment',
    backref='comment_author',
    lazy='dynamic',
    foreign_keys='PostComment.author_id',
    cascade='all, delete-orphan')

# Likes & Bookmarks
post_likes = db.relationship('PostLike',
    backref='like_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='PostLike.user_id')

resource_likes = db.relationship('ResourceLike',
    backref='like_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='ResourceLike.user_id')

startup_bookmarks = db.relationship('StartupBookmark',
    backref='bookmark_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='StartupBookmark.user_id')

knowledge_bookmarks = db.relationship('KnowledgeBookmark',
    backref='bookmark_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='KnowledgeBookmark.user_id')

idea_bookmarks = db.relationship('IdeaBookmark',
    backref='bookmark_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='IdeaBookmark.user_id')

# Tasks & Projects (Three different relationships - all need unique backrefs)
owned_tasks = db.relationship('Task',
    backref='task_owner',
    lazy='dynamic',
    foreign_keys='Task.user_id',
    cascade='all, delete-orphan')

created_tasks = db.relationship('Task',
    backref='task_creator',
    lazy='dynamic',
    foreign_keys='Task.created_by')
```

```
assigned_tasks = db.relationship('Task',
    backref='task_assignee',
    lazy='dynamic',
    foreign_keys='Task.assigned_to')

project_goals = db.relationship('ProjectGoal',
    backref='goal_owner',
    lazy='dynamic',
    foreign_keys='ProjectGoal.user_id',
    cascade='all, delete-orphan')

# Team & Startup Relationships
startup_memberships = db.relationship('StartupMember',
    backref='member_user',
    lazy='dynamic',
    foreign_keys='StartupMember.user_id',
    cascade='all, delete-orphan')

join_requests = db.relationship('JoinRequest',
    backref='request_user',
    lazy='dynamic',
    foreign_keys='JoinRequest.user_id',
    cascade='all, delete-orphan')

# Achievements & Growth
user_achievements = db.relationship('UserAchievement',
    backref='achievement_owner',
    lazy='dynamic',
    foreign_keys='UserAchievement.user_id',
    cascade='all, delete-orphan')

# Change this line in the GrowthMetric relationship:
growth_metrics = db.relationship('GrowthMetric',
    backref='metric_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='GrowthMetric.user_id')

# Chat & Messaging
sent_messages = db.relationship('ChatMessage',
    backref='message_sender',
    lazy='dynamic',
    foreign_keys='ChatMessage.sender_id',
    cascade='all, delete-orphan')

created_conversations = db.relationship('ChatConversation',
    backref='conversation_creator',
    lazy='dynamic',
    foreign_keys='ChatConversation.created_by_id',
    cascade='all, delete-orphan')

conversations = db.relationship('ChatConversation',
    secondary='conversation_participants',
    backref=db.backref('conversation_participants_users', lazy='dynamic'))

# Views & Analytics
resource_views = db.relationship('ResourceView',
    backref='view_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='ResourceView.user_id')

story_views = db.relationship('StoryView',
    backref='view_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='StoryView.user_id')

# Downloads
resource_downloads = db.relationship('ResourceDownload',
    backref='download_owner',
    lazy='dynamic',
```

```
    cascade='all, delete-orphan',
    foreign_keys='ResourceDownload.user_id')

# Suggestions
suggestions = db.relationship('Suggestion',
    backref='suggestion_author',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='Suggestion.author_id')

# Calendar Events
calendar_events = db.relationship('CalendarEvent',
    backref='event_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='CalendarEvent.user_id')

# Goal Milestones
goal_milestones = db.relationship('GoalMilestone',
    backref='milestone_owner',
    lazy='dynamic',
    cascade='all, delete-orphan',
    foreign_keys='GoalMilestone.user_id')

# ========== HELPER FUNCTIONS ==========

def update_last_activity(self):
    """Update last activity date and maintain streak"""
    today = datetime.utcnow().date()

    if self.last_activity_date and self.last_activity_date == today - timedelta(days=1):
        self.streak_days += 1
    elif self.last_activity_date != today:
        self.streak_days = 1

    self.last_activity_date = today
    self.last_login = datetime.utcnow()

    from app.services.achievement_service import AchievementService
    AchievementService.check_achievements(self.id, 'streak_days')

    db.session.commit()

def add_xp_points(self, points: int):
    """Add XP points to user"""
    self.xp_points += points
    db.session.commit()

def verify_email(self):
    """Mark user's email as verified"""
    self.is_email_verified = True
    db.session.commit()

def update_profile(self, profile_data: dict):
    """Update user profile information"""
    if 'picture' in profile_data:
        self.profile_picture = profile_data['picture']
    if 'bio' in profile_data:
        self.profile_bio = profile_data['bio']
    if 'company' in profile_data:
        self.profile_company = profile_data['company']
    if 'socialLinks' in profile_data:
        self.profile_social_links = profile_data['socialLinks']
    if 'country' in profile_data:
        self.profile_country = profile_data['country']
    if 'city' in profile_data:
        self.profile_city = profile_data['city']
    if 'timezone' in profile_data:
        self.profile_timezone = profile_data['timezone']
        self.pref_timezone = profile_data['timezone']

    db.session.commit()
```

```
    def update_preferences(self, preferences_data: dict):
        """Update user preferences"""
        if 'emailNotifications' in preferences_data:
            self.pref_email_notifications = preferences_data['emailNotifications']
        if 'pushNotifications' in preferences_data:
            self.pref_push_notifications = preferences_data['pushNotifications']
        if 'privacy' in preferences_data:
            self.pref_privacy = Privacy(preferences_data['privacy'])
        if 'language' in preferences_data:
            self.pref_language = preferences_data['language']
        if 'timezone' in preferences_data:
            self.pref_timezone = preferences_data['timezone']
        if 'theme' in preferences_data:
            self.pref_theme = Theme(preferences_data['theme'])

        db.session.commit()

    def update_notification_settings(self, notification_data: dict):
        """Update notification settings"""
        if 'newComments' in notification_data:
            self.notif_new_comments = notification_data['newComments']
        if 'newLikes' in notification_data:
            self.notif_new_likes = notification_data['newLikes']
        if 'newSuggestions' in notification_data:
            self.notif_new_suggestions = notification_data['newSuggestions']
        if 'joinRequests' in notification_data:
            self.notif_join_requests = notification_data['joinRequests']
        if 'approvals' in notification_data:
            self.notif_approvals = notification_data['approvals']
        if 'storyViews' in notification_data:
            self.notif_story_views = notification_data['storyViews']
        if 'postEngagement' in notification_data:
            self.notif_post_engagement = notification_data['postEngagement']
        if 'emailDigest' in notification_data:
            self.notif_email_digest = EmailDigest(notification_data['emailDigest'])

        if 'quietHours' in notification_data:
            quiet_hours = notification_data['quietHours']
            if 'enabled' in quiet_hours:
                self.notif_quiet_hours_enabled = quiet_hours['enabled']
            if 'start' in quiet_hours:
                self.notif_quiet_hours_start = quiet_hours['start']
            if 'end' in quiet_hours:
                self.notif_quiet_hours_end = quiet_hours['end']

        db.session.commit()

    def set_password(self, password: str):
        """Hash and set user password"""
        self.password = generate_password_hash(password)
        db.session.commit()

    def check_password(self, password: str) -> bool:
        """Check if provided password matches the hashed password"""
        return check_password_hash(self.password, password)

    def is_active(self):
        """Check if user is active"""
        return self.status == UserStatus.active

    def deactivate(self):
        """Deactivate user account"""
        self.status = UserStatus.inactive
        db.session.commit()

    def activate(self):
        """Activate user account"""
        self.status = UserStatus.active
        db.session.commit()

    def get_full_name(self):
        """Get user's full name"""
```

```
        return f"{self.first_name} {self.last_name}"

    def get_timezone(self):
        """Get user's timezone for message display"""
        from app.utils.timezone_converter import get_user_timezone
        return get_user_timezone(self)

    def get_statistics(self):
        """Get comprehensive user statistics"""
        return {
            'total_ideas': self.ideas.count(),
            'total_tasks': self.owned_tasks.count(),
            'completed_tasks': self.owned_tasks.filter_by(status='completed').count(),
            'total_startups': self.startups.count(),
            'total_achievements': self.user_achievements.filter_by(is_completed=True).count(),
            'total_comments': (self.idea_comments.count() +
                               self.knowledge_comments.count() +
                               self.post_comments_list.count()),
            'total_likes_received': self._calculate_total_likes_received(),
            'engagement_score': self._calculate_engagement_score()
        }

    def _calculate_total_likes_received(self):
        """Calculate total likes received on user's content"""
        total_likes = 0
        total_likes += sum(idea.likes for idea in self.ideas)
        total_likes += sum(knowledge.likes for knowledge in self.knowledge_posts)
        total_likes += sum(post.likes for post in self.user_posts)
        return total_likes

    def _calculate_engagement_score(self):
        """Calculate user engagement score (0-100)"""
        activities = (
            self.ideas.count() * 2 +
            self.owned_tasks.filter_by(status='completed').count() * 1 +
            (self.idea_comments.count() + self.knowledge_comments.count() + self.post_comments_list.coun
            self._calculate_total_likes_received() * 0.5
        )
        return min(activities / 10, 100)

    def get_recent_activity(self, limit=10):
        """Get user's recent activity across all platforms"""
        from models.task import Task
        from models.idea import Idea
        from models.ideaComment import IdeaComment

        activities = []

        for idea in self.ideas.order_by(Idea.created_at.desc()).limit(5).all():
            activities.append({
                'type': 'idea_created',
                'title': idea.title,
                'timestamp': idea.created_at,
                'data': idea.to_dict()
            })

        for task in self.owned_tasks.filter_by(status='completed').order_by(Task.completed_date.desc()).
            activities.append({
                'type': 'task_completed',
                'title': task.title,
                'timestamp': task.completed_date,
                'data': task.to_dict()
            })

        for comment in self.idea_comments.order_by(IdeaComment.created_at.desc()).limit(5).all():
            activities.append({
                'type': 'comment_made',
                'title': f'Comment on idea',
                'timestamp': comment.created_at,
                'data': comment.to_dict()
            })
        activities.sort(key=lambda x: x['timestamp'], reverse=True)
```

```
        return activities[:limit]

    def to_dict(self, include_password=False, include_statistics=False, include_recent_activity=False):
        data = {
            'id': self.id,
            'firstName': self.first_name,
            'lastName': self.last_name,
            'email': self.email,
            'isEmailVerified': self.is_email_verified,
            'lastLogin': self.last_login.isoformat() if self.last_login else None,
            'status': self.status.value,
            'role': self.role,
            'xp_points': self.xp_points,
            'streak_days': self.streak_days,
            'last_activity_date': self.last_activity_date.isoformat() if self.last_activity_date else No
            'total_revenue': self.total_revenue,
            'satisfaction_percentage': self.satisfaction_percentage,
            'active_startups_count': self.active_startups_count,
            'profile': {
                'picture': self.profile_picture,
                'bio': self.profile_bio,
                'company': self.profile_company,
                'socialLinks': self.profile_social_links or {},
                'country': self.profile_country,
                'city': self.profile_city,
                'timezone': self.profile_timezone
            },
            'preferences': {
                'emailNotifications': self.pref_email_notifications,
                'pushNotifications': self.pref_push_notifications,
                'privacy': self.pref_privacy.value,
                'language': self.pref_language,
                'timezone': self.pref_timezone,
                'theme': self.pref_theme.value
            },
            'notificationSettings': {
                'newComments': self.notif_new_comments,
                'newLikes': self.notif_new_likes,
                'newSuggestions': self.notif_new_suggestions,
                'joinRequests': self.notif_join_requests,
                'approvals': self.notif_approvals,
                'storyViews': self.notif_story_views,
                'postEngagement': self.notif_post_engagement,
                'emailDigest': self.notif_email_digest.value,
                'quietHours': {
                    'enabled': self.notif_quiet_hours_enabled,
                    'start': self.notif_quiet_hours_start,
                    'end': self.notif_quiet_hours_end
                }
            },
            'createdAt': self.created_at.isoformat(),
            'updatedAt': self.updated_at.isoformat(),
            'fullName': self.get_full_name(),
            'timezone': self.get_timezone()
        }

        if include_password:
            data['password'] = self.password

        if include_statistics:
            data['statistics'] = self.get_statistics()

        if include_recent_activity:
            data['recentActivity'] = self.get_recent_activity()

        return data
```

37 — userAchievement.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timedelta
from sqlalchemy import Enum
from app.extensions import db

class UserAchievement(db.Model):
    __tablename__ = 'user_achievements'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    achievement_id = db.Column(db.Integer, db.ForeignKey('achievements.id'), nullable=False)

    unlocked_at = db.Column(db.DateTime, default=datetime.utcnow)
    progress_percentage = db.Column(db.Integer, default=0)
    is_completed = db.Column(db.Boolean, default=False)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Relationships
    user = db.relationship('User', backref='user_achievements_list', foreign_keys=[user_id])
    achievement = db.relationship('Achievement', backref='user_achievements_list', foreign_keys=[achieve

    # HELPER FUNCTIONS

    def update_progress(self, progress):
        """Update progress and check if achievement is unlocked"""
        self.progress_percentage = max(0, min(100, progress))

        if self.progress_percentage >= 100 and not self.is_completed:
            self.unlock()

        db.session.commit()

    def unlock(self):
        """Unlock achievement and award points"""
        self.is_completed = True
        self.unlocked_at = datetime.utcnow()
        self.progress_percentage = 100

        # Award XP points to user
        if self.user and self.achievement:
            self.user.add_xp_points(self.achievement.points)

        db.session.commit()

    def increment_progress(self, increment=10):
        """Increment progress by specified amount"""
        new_progress = self.progress_percentage + increment
        self.update_progress(new_progress)

    def get_time_since_unlock(self):
        """Get time since achievement was unlocked"""
        if self.is_completed and self.unlocked_at:
            return datetime.utcnow() - self.unlocked_at
        return None

    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'achievement_id': self.achievement_id,
            'unlocked_at': self.unlocked_at.isoformat() if self.unlocked_at else None,
            'progress_percentage': self.progress_percentage,
            'is_completed': self.is_completed,
            'created_at': self.created_at.isoformat(),
            'achievement': self.achievement.to_dict() if self.achievement else None,
            'time_since_unlock': str(self.get_time_since_unlock()) if self.get_time_since_unlock() else
        }
```

38 — Enums.cpython-312.pyc

```
■■■■'■iW■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■d■d■l■Z■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■
■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■G■d■■d
e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■G■d■■d■e■j■■■
■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■y■)■■■
UserStatus■active■deletedN)■■__name__
__module__■__qualname__r■■■■r■■■■■■■■■■ E:\LLMs_test\app\models\Enums.pyr■■■■r■■■■■■■■■■■■■■
■F■■Gr■■■■r■■■■c■■■■■■■■■■■■■■■■■■■
```

```
__module__■__qualname__
__tablename__r■■■■■■Column■Integerr&■■■
ForeignKeyr*■■■r+■■■■DateTimer■■■■r■■■■r■■■■r,■■■r-■■■r■■■■r■■■■r$■■■r6■■■■r■■■■r■■■■r■■■■
■

ny(Ydi■jK■■bii■

MBMM*y$Q\a■bG■■BIIbkk8??■CM■■■■■■2;;■■■■■@J■■■■■■2;;■■■(//■ZJ■■■6■■■9
■■■■■■■
r■■■■r■■■■N)■r■■■■■■app.extensionsr■■■■■■Modelr■■■■r@■■■r■■■■r■■■■■■<module>rC■■■■■■■■s■■■■■■■■■
rxx■5■
r■■■■■
```

40 — ResourceLike.cpython-312.pyc

```
■■■■]■i;■■■■■■■■■■■■■■■■■■■■■■■■■F■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

41 — ResourceView.cpython-312.pyc

```
[redacted]
e[redacted]
[redacted]
e[redacted]
[redacted]
e[redacted]
```

43 — achievement.cpython-312.pyc

```
■■■■6■i■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■■■■
SQLAlchemy)■■datetime■timedelta)■■Enum)■■dbc■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■■■
```

[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j8[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j8[redacted]e[redacted]j8[redacted]
CalendarEvent[redacted]calendar_eventsT)[redacted]primary_keyz[redacted]users.idF)[redacted]nullablez[redacted]startups.id[redacted])[redacted]default[redacted]meeting
start_date[redacted]end_date[redacted]all_dayr[redacted]session[redacted]commit)[redacted]selfr[redacted]r[redacted]r[redacted]s[redacted] (E:\LLMs_test\app\m
[redacted]

[redacted]c[redacted]t[redacted]|[redacted]j[redacted]s[redacted]y[redacted]|[redacted]j[redacted]
[redacted]}[redacted]|[redacted]j[redacted]d[redacted]z[redacted]S[redacted])[redacted]z[redacted]Get event duration in minutesr[redacted]<[redacted])[redacted]r[redacted]r
total_seconds)[redacted]r[redacted]durations[redacted] r[redacted]get_durationz[redacted]CalendarEvent.get_duration-[redacted]s2[redacted]}}[redacted]

t}}[redacted]4[redacted]!4[redacted]`4??]`K`[redacted]`r[redacted]c[redacted]t[redacted]j[redacted]
is_ongoingz[redacted]CalendarEvent.is_ongoing9[redacted]s\[redacted]oo[redacted]==[redacted]??c[redacted]:T]][redacted]:[redacted]:[redacted]:[redacted]:7;||t[redacted]#[redacted]#[redacted]%[redacted]3[redacted]
[redacted]S[redacted])[redacted]z[redacted]Get reminder time)[redacted]minutes)[redacted]r[redacted]r[redacted]reminder_minutes[redacted]r[redacted]s[redacted] r[redacted]get_reminder_t
reminder_timer([redacted]s[redacted] r[redacted]
should_remindz[redacted]CalendarEvent.should_remindD[redacted]sG[redacted] [redacted] A[redacted]%[redacted].[redacted].[redacted]0
[redacted]oo[redacted]6t[redacted]6[redacted]6[redacted]6[redacted]6r[redacted]c[redacted]d[redacted]d[redacted]d[redacted]d[redacted]d[redacted]}[redacted]|[redacted]j[redacted]
[redacted]zz[redacted]AVZZ[redacted]

y[redacted]A[redacted]Ar[redacted]c[redacted]0[redacted]i[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]
[redacted]j
[redacted]d[redacted]|[redacted]j[redacted]r[redacted]|[redacted]j[redacted]j
[redacted]n[redacted]d[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]d
|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]d
|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]j
[redacted]d[redacted]|[redacted]j [redacted]j
[redacted]d[redacted]|[redacted]j#[redacted]|[redacted]j%[redacted]
[redacted]n[redacted]d[redacted]|[redacted]j+[redacted]|[redacted]j,[redacted]rB|[redacted]
startup_id[redacted]title[redacted]descriptionr[redacted]r[redacted]r[redacted]r8[redacted]r6[redacted]location[redacted]is_recurring[redacted]recurrence_ruler/[redacted]
created_at
updated_at[redacted]duration_minutesr[redacted])[redacted]r<[redacted]firstName[redacted]lastName)[redacted]r<[redacted]name)[redacted]r)[redacted]r,[redacted]r3[redacted]r4[redacted]use
first_name[redacted]last_namerK[redacted]rI[redacted]r0[redacted]s[redacted] r[redacted]to_dictz[redacted]CalendarEvent.to_dictW[redacted]s[redacted]
[redacted]$''[redacted]
[redacted]t||[redacted]
[redacted]
[redacted]$//[redacted]
[redacted]
[redacted]TZZ[redacted]

[redacted]
[redacted]4[redacted]+[redacted]+[redacted]
[redacted]
[redacted]$//[redacted]3[redacted]3[redacted]5
[redacted]
[redacted]
[redacted]T]][redacted]

[redacted]/[redacted]/[redacted]1[redacted]
[redacted]
[redacted]t||[redacted]
[redacted]
[redacted]

[redacted]
[redacted]
[redacted]T[redacted],[redacted],[redacted].[redacted]
[redacted]
[redacted]

[redacted]
[redacted]
[redacted]D[redacted]-[redacted]-[redacted]
[redacted]
[redacted]t[redacted]3[redacted]3[redacted]
[redacted]
[redacted] 5 5[redacted]
[redacted]
[redacted]$//[redacted]3[redacted]3[redacted]5[redacted]
 [redacted]
[redacted]$//[redacted]3[redacted]3[redacted]5![redacted]

```
"■
■■ 1 1 3#■■
$■■■||~■■//■+EIEZEZ]^E^T■3■3■5■?■?■Adh■!■/■/■1
■■■■■■■■■■iill■!YY■1■1■ II■/■/■■■■■■!%■■■■■■■■■■■lloo■■■■■■)■)■■■■7■■
■■■
<■$(=■■
■■■
r■■■■■)■NF)'■__name__
__module__■__qualname__
__tablename__r■■■■■■Column■Integerr<■■■
ForeignKeyr=■■■r>■■■■■Stringr?■■■■■Textr@■■■■■DateTimer■■■■r■■■■■■Booleanr■■■■r■■■■r8■■■r6■■■rA■■■

MBMM*$=■■NG■■■■■■2::}r}}]'Cd■SJ■■BIIibii■nu■5E■■"))BGG■$K■■■■■■2;;■■7J■■ryy■■■t■4H■■bii■

E■2G■■ryy■i■Z■■ISZ■[H■■BIIibii■m■$E■■ryy■■■■3■■(H■■299RZZ■■7L■■bii■■■■#■■/O■ ryy■■■R■8■■■■■■■
■■O■■■■J■■■■7■■■B■■ ■
r■■■■■r■■■■■N)
■flask_sqlalchemyr■■■■r■■■■r■■■■■
sqlalchemyr■■■■■■app.extensionsr■■■■■■Modelr■■■■■r[■■■r■■■■r■■■■■■<module>r`■■■■■■■■s#■■■■■■■■■■'■(■
BHH■q■■
r■■■■■
```

45 — chatConversation.cpython-312.pyc

```
■■■■■P■i■■■■■■■■■■■
```

```
********"*****t**********|*j**********************|*j**********************|*|*|*x*s****i*|*|*j
**********j*****************************|*_**********t**********j**********************j******
r*****r&*****get_timezoner*****r *****addr******utcnow
updated_atr)****)*r*****rM*****rH*****rW*****rX*****rY*****rD*****s*****       r.*****add_messagez*ChatConversa
******

**w****"//*+****


**********r0****c**********************@******t************f*d**|*j**********************D******
**********************|*j**********************j*****************************|*j*****************
avatar_urlis_activer;****r^****rg****rQ****
message_countunread_count)*r&****rr****rs****rt****ru****rv****r;*****isoformatr^****rg****rN****rO*****
**-**********gg*!__* NN"&"6"6* *-*-*/********-****!*9*9(*C*!]]*0*0*2JRD*9*9(++*FXY%**
*********-s******A*D**)**member)*2****r*****rb****)**textNN)$*__name__
__module__*__qualname__
__tablename__r******Column*Integerr&*****Stringrr****rs****
ForeignKey
created_by_id*Textrt****ru*****Booleanrv****r******settings*DateTimer*****r]****r;****r^*****relationshi
*Ce*TM*****"))BGGd*3K******9299S>D*9J********"**d*3I**ryy*r**H******2;;*****@J******2;;****(//*ZJ
****r**}n9^r*sH**** *********/****"**********$**S*****%*****r0****r*****r"****r*****z*chat_conversation
BII******]R]];R-Sae*f*
BIIi****]R]]:%>D*Q*
BIIk2;;*****@*
BIIfibii*mX*6********r0****


## 46 — chatMessage.cpython-312.pyc

******i*****************************R*****d*d*l*m*Z****d*d*l*m*Z****d*d*l*m*Z******G*d**d*e*j
*****************************Z*y*)******)**datetime)**db)**JSONc*****************************e*Z*
**********************e*j**********************d***********Z****e*j
**********************e*j**********************e*j**********************d**********d***********
**********************e*j**********************e*j**********************d**********d***********
**e*j
**********************e*j**********************d***********Z****e*j
***********************e*j**********************d**********d*
*********Z****e*j
***********************e*j**********************d**********d*
********Z****e*j
**********************e*i*
********Z****e*j
**********************e*j$*********************d*
********Z****e*j
**********************e*j(**********************d***********Z****e*j
**********************e*j**********************e*j**********************d
********d***********Z****e*j
**********************e*j(**********************e*j0********************
********Z****e*j
**********************e*j(**********************e*j0**********************e*j0**********************
chat_messagesT)**primary_keyz*chat_conversations.idF)**nullablez*users.id2*****UTC)**default******tex
*****onupdate*replies)**remote_side*backref*foreign_keysNc*****************************d*d*l*m*}****
*********************|*j*****************************}*|**d*|*j***************************S*)*z6
created_at*original_content)**self*sender_userr******time_placeholders*****    &E:\LLMs_test\app\models
******#*#1T%:%:$;*<*<*****c*****************************d*d*l*m*}****|*j***************************
**********************|*|*j*****************************}*|*S*)*z:Get message content converted fo
******!* r$****c**************
********v*****|*j**********************|*j**********************|*j**********************|*j****
**********************x*s****i*|*j**********************|*j**********************r*|*j**********
updated_atr******content*display_time)*r,*****firstName*lastName*profilePicture*sender)*r,****r5****r.
first_name*last_name*profile_picturer;****)*r******for_user*base_datas*****   r"*****to_dictz*ChatMess

*+******7;~~*****1*1*34****+*+**//*3*3*5**//*3*3*5*#*3*3*
*
* ****#'#<#<X#FIi* (,(B(B8(LIn*%#'#;#;#=Ii* *****;;**kknn*![[*3*3* KK*1*1"&++"="=***#*Ih*******==***m
**********j**********************j*****************************y*)*z*Edit message contentTN)*r
*******r$****c*****************************y*)*z*Mark message as read by userN*)*r******user_ids****
```

```
r$[redacted])[redacted]N)#[redacted]__name__
__module__[redacted]__qualname__
__tablename__r[redacted]Column[redacted]Integerr,[redacted]
ForeignKeyr-[redacted]r.[redacted]Textr[redacted]Stringr[redacted]r/[redacted]r[redacted]
metadata_data[redacted]Booleanr1[redacted]DateTimer2[redacted]r3[redacted]r[redacted]rG[redacted]r[redacted]r4[redacted]relationshipr;[redacted]r#[redacted]r*[redacted]r

MBMM:Q,R]b[redacted]cO[redacted]"**mbmmJ&?%[redacted]PI[redacted]!ryy[redacted]5[redacted]9[redacted] bii[redacted]"
u[redacted]=O[redacted]299YRYYr]F[redacted];L[redacted]BIIdB[redacted]/M[redacted]"**e[redacted]4I[redacted]"++[redacted]5I[redacted]"))BJJ
[redacted]

6H(ITX[redacted]YK[redacted]2;;[redacted]@J[redacted]2;;[redacted](//[redacted]ZJ
[redacted]r[redacted]}2$[redacted]al`m[redacted]nH[redacted]= [redacted]! )[redacted]V[redacted]Q[redacted]
r$[redacted]r[redacted]N)[redacted]r[redacted]app.extensionsr[redacted]
sqlalchemyr[redacted]Modelr[redacted]rM[redacted]r$[redacted]r"[redacted]<module>r`[redacted]s [redacted]~[redacted]
"([redacted]~[redacted]
r$[redacted]
```

47 — goalMilstone.cpython-312.pyc

```
[redacted]U[redacted]i[redacted]b[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]
SQLAlchemy)[redacted]datetime[redacted]timedelta)[redacted]Enum)[redacted]dbc[redacted]6[redacted]e[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j [redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j$[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j$[redacted]e[redacted]j*[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j$[redacted]e[redacted]j*[redacted]e[redacted]j*[redacted]
e[redacted]g[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]y[redacted])[redacted]
GoalMilestone[redacted]goal_milestonesT)[redacted]primary_keyz[redacted]project_goals.idF)[redacted]nullablez[redacted]users.id[redacted]r[redacted])[redacted]defau
milestones)[redacted]backref[redacted]foreign_keysc[redacted]d[redacted]|[redacted]_[redacted]t[redacted]j[redacted]

[redacted]c[redacted]d[redacted]|[redacted]_[redacted]d[redacted]|[redacted]_[redacted]|[redacted]j[redacted]
[redacted]|[redacted]j[redacted]_[redacted]|[redacted]j[redacted]j[redacted]
uncompletez[redacted]GoalMilestone.uncomplete([redacted]s^[redacted]![redacted]"[redacted]99-0[redacted]DII4R4RUV4V-WDII[redacted]*[redacted]II[redacted]%[redacted]%[redacted]'[redacted]

[redacted]r [redacted]c[redacted]N[redacted]|[redacted]|[redacted]_[redacted]t[redacted]j[redacted]j[redacted]
[redacted]

[redacted]r [redacted]c[redacted]|[redacted]j[redacted]|[redacted]j[redacted]|[redacted]j[redacted]
[redacted]|[redacted]j[redacted]|[redacted]j[redacted]r[redacted]|[redacted]j[redacted]
created_at
updated_at
goal_title)[redacted]r,[redacted]r-[redacted]r.[redacted]r/[redacted]r0[redacted]r([redacted]r[redacted]r[redacted]isoformatr1[redacted]r2[redacted]r[redacted]r[redacted]s[redacted] r[redacted]
[redacted]
[redacted]=A[redacted]
[redacted]
r [redacted]N)[redacted]__name__
__module__[redacted]__qualname__
__tablename__r[redacted]Column[redacted]Integerr,[redacted]
ForeignKeyr-[redacted]r.[redacted]Stringr/[redacted]Textr0[redacted]r([redacted]Booleanr[redacted]DateTimer[redacted]r[redacted]r[redacted]r1[redacted]r2[redacted]

MBMM2D$EPU[redacted]VG[redacted]bii[redacted]

MBMM*$=[redacted]NG[redacted]BIIibii[redacted]nu[redacted]5E[redacted]"))BGG[redacted]$K[redacted]BIIbjj![redacted],E[redacted]299RZZ[redacted]7L[redacted]RYYr{{T[redacted]:N[redacted]2;;[redacted]@J[redacted]2;;
[redacted]
```

```
■■■■■■

■
r ■■■r■■■■N)
■flask_sqlalchemyr■■■■r■■■■r■■■■
sqlalchemyr■■■■■app.extensionsr■■■■■Modelr■■■■rB■■■r ■■■r■■■■■<module>rG■■■■■■■■s#■■■■■■■■■'■(■
BHH■@■■
r ■■■
```

48 — growthMetric.cpython-312.pyc

```
■■■■L■iY■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■■■
SQLAlchemy)■■datetime■timedelta)■■Enum)■■dbc■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■d■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■d■■■■■■■■■■■■
■■e■j
■■■■■■■■■■■■■■■■■■■■■■■■e■d■d■d■d■d
■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■
```

```
created_at
updated_atr*■■■■period_duration■trend_direction)■r8■■■■name)■r8■■■■firstName■lastName)■■formatted_v
first_name■last_namer■■■■s■■■■ r■■■■■to_dictz■GrowthMetric.to_dictX■■■s■■■■■■■
■■$''■■■
■■$//■■■
■■
■t||■■■
■■
■4■+■+■■■

■
■T■/■/■■■
■■
■d■1■1
■■
■■
 ■!7!7■■■
■■
%d&A&A■■■
■■
■d■1■1■■■
■■
&t'C'C■■■
■■
■D■-■-■7■7■9■■■
■■
■$//■3■3■5■■■
■■
■$//■3■3■5■■■
■■
■$//■3■3■5■■■
■■
 ■!7!7!9■■■
 ■
■t■7■7■9!■■
"■
■t■7■7■9#■■
$■ $■0■0■2■■■■■■■■■■■lloo■■■■■)■)■■■■■■$(
■■■■■■■■■■iill■!YY■1■1■ II■/■/■■■■/■■
■■■
6■!%7■■
■■■
r■■■■)■N)"■__name__
__module__■__qualname__
__tablename__r■■■■■Column■Integerr8■■■
ForeignKeyr9■■■r:■■■r■■■■r■■■■■Floatr■■■■r■■■■r■■■■r■■■■r ■■■r!■■■■DateTimer'■■■r(■■■r■■■■r

MBMM*$=■■MG■■"))D■■>9■U`e■fK■■BIIbhh■■2M■■RYYrxx■■3N■!■■■"((A■6■■■&RYYrxx■■;■■■■RYYrxx■■3N■'bii
■■:■■■■■■■0■■■
r■■■■r■■■■N)
■flask_sqlalchemyr■■■■r■■■■r■■■■
sqlalchemyr■■■■■app.extensionsr■■■■■Modelr■■■■rR■■■r■■■■r■■■■■<module>rW■■■■■■■s#■■■■■■■■■'■(■
288■o■■
r■■■■
```

49 — idea.cpython-312.pyc

```
■■■■■■ic■■■■■■■■■■■■■■■■■■■■■■■■■n■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■
■■■■■)■■datetime)■■db)■■Enum■JSON■■■■)■■ResourceStatus)■■IdeaCommentc■■■■■■■■■■■■■■■■■■■
```

[redacted]

a[redacted]
[redacted]

[redacted]

[redacted]

50 — ideaBookmark.cpython-312.pyc

```
■■■■T(■i■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■■■
SQLAlchemy)■■datetime■timedelta)■■Enum)■■dbc■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■d■■■■■■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■d■■■■■■■
■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■d
■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j ■■■■■■■■■■■■■■■■■■■■e■j$■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■e■j(■■■■■■■
d■e■g■■■■■■■■■■Z■■■e■j(■■■■■■■■■■■■■■■■■■■■d■d■e
g■■■■■■■■■■Z■d■d■■Z■d■■Z■d■d■■Z■d■■Z■y■)■■IdeaBookmark■idea_bookmarksT)■■primary_keyz■users.id■C


■■■■■■■■■■■c■■■■■■■■■■■■■■■■■■■■■■z■■■■■|■j■■■■■■■■■■■■■■■■■■■■■r|■j■■■■■■■■■■■■■■■■■■■■■j■■
■■■■■■■■■■■■■■■■■■■■■d■|■j■■■■■■■■■■■■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■
```

[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j [redacted]e[redacted]j$[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j [redacted]e[redacted]j$[redacted]e[redacted]j$[redacted]
d[redacted]e[redacted]g[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]y[redacted])[redacted]IdeaComment
idea_commentsT)[redacted]primary_keyz[redacted]ideas.id[redacted]CASCADE)[redacted]ondeleteF)[redacted]nullablez[redacted]users.idd[redacted])[redacted]default)[redacted]r
[redacted]onupdate[redacted]Idea)[redacted]back_populates[redacted]foreign_keysc[redacted]N[redacted]|[redacted]|[redacted]_[redacted]t[redacted]

[redacted]c[redacted]8[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]
[redacted]|[redacted]j
[redacted]d[redacted]|[redacted]j[redacted]j[redacted]|[redacted]j[redacted]
r"[redacted]idea_idr[redacted]author_idr[redacted]r[redacted]r [redacted]
created_at[redacted]isoformat
updated_atr[redacted]s[redacted] r[redacted]to_dictz[redacted]IdeaComment.to_dict![redacted]sm[redacted]''[redacted]ll[redacted]||[redacted]nn[redacted]![redacted]3[redacted]3[redacted] [redacted]1[redacted]1[redacted]
[redacted]
r[redacted]N)[redacted]__name__
__module__[redacted]__qualname__
__tablename__r[redacted]Column[redacted]Integerr"[redacted]
ForeignKeyr*[redacted]Textr[redacted]r+[redacted]Stringr[redacted]r[redacted]DateTimer[redacted]utcnowr,[redacted]r.[redacted]relationship[redacted]par

MBMM*y$Q\a[redacted]bG[redacted]bii[redacted]%[redacted]0G[redacted]"**mbmmJ&?%[redacted]PI[redacted]![redacted])"))C.[redacted]1[redacted] ryy[redacted]3[redacted]0[redacted]2;;[redacted]@J[redacted]
[redacted]C[redacted]
[redacted]
r[redacted]r[redacted]N)[redacted]r[redacted]app.extensionsr[redacted]Modelr[redacted]r=[redacted]r[redacted]r[redacted]<module>r@[redacted]s[redacted]
"([redacted]*[redacted]
r[redacted]

52 — joinRequest.cpython-312.pyc

[redacted]y![redacted]i8[redacted]^[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d
d[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z
[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]e[redacted]e[redacted]j([redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]e[redacted]j0[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]e[redacted]j0[redacted]e[redacted]j0[redacted]
join_requestsT)[redacted]primary_keyz[redacted]startups.id[redacted]CASCADE)[redacted]ondeleteF)[redacted]nullable[redacted]z[redacted]users.idd[redacted]member)[redacted]
StartupMember)[redacted]
startup_id[redacted]user_id
first_name[redacted]last_name[redacted]role)[redacted]r[redacted]approved[redacted]statusr[redacted]utcnow
updated_at[redacted]models.startUpMemberr[redacted]r[redacted]r[redacted]r[redacted]r[redacted]r[redacted]r[redacted]session[redacted]add[redacted]startup[redacted]update_m
[redacted]

[redacted]v[redacted]<<[redacted]LL[redacted],[redacted],[redacted].[redacted]

[redacted]
[redacted]c[redacted]t[redacted]j[redacted]|[redacted]_[redacted]t[redacted]j[redacted]

[redacted]r.[redacted]c[redacted]t[redacted]j[redacted]|[redacted]_[redacted]t[redacted]

[redacted]r.[redacted]c[redacted]<[redacted]|[redacted]j[redacted]t[redacted]j[redacted]
is_pendingz[redacted]JoinRequest.is_pending?[redacted]s[redacted]{{[redacted]/[redacted]7[redacted]7[redacted]7[redacted]7r.[redacted]c[redacted]D[redacted]t[redacted]
[redacted]S[redacted])[redacted]z"Get time since request was created)[redacted]r[redacted]r"[redacted]

```
created_atr4■■■s■■■■ r,■■■■get_durationz■JoinRequest.get_durationC■■■s■■■■■■■■■■ 4??■2■2r.■■■c■■
■■■■■■■■■■■■■■■■■■■■|■j■■■■■■■■■■■■■■■■■■■■■■|■j■■■■■■■■■■■■■■■■■■■
```

```
new_statuss■■■■  r#■■■
update_statusz■Knowledge.update_statush■■■s■■■■■■$Z■0■■■

■■■■■■r&■■■c■■■■■■■■■■■■■■■■■■■■6■■■■|■j■■■■■■■■■■■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■
```

[redacted]

55 — knowledgeComment.cpython-312.pyc

[redacted]

ny(Ydi■jK■■bii■■■%■0G■■■■■"**mbmmJ&?%■PI■!■■■)"))C.■1■■■ ryy■■■■3■■0■■■■■■■■2;;■■■■■@J■■■■■■2;;■
■R__V-FV_U`
aF■■■■■
■■C■■■■■@■■■■■
r■■■■r■■■■■N)■r■■■■■■app.extensionsr■■■■■■Modelr■■■■■rE■■■■r■■■■■r■■■■■■<module>rH■■■■■■■■s■■■■■■■■■
rxx■0■
r■■■■■

56 — notification.cpython-312.pyc

■■■■■^#■ia■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■m■Z■■■■d■d■l■m■Z■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■■d■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■d
■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■

57 — post.cpython-312.pyc

```
    ■■■■q\■i■■■■■■■■■■■■■■■■■■■■■■■■■■n■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■
    ■■■■)■■datetime)■■db)■■Enum■JSON■■■■)■■PostType)■■PostCommentc■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■
    ■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
    ■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

```
__module__■__qualname__
__tablename__r■■■■■■Column■IntegerrR■■■
ForeignKeyr@■■■■ra■■■■■Stringrb■■■■rc■■■■■TextrX■■■■r■■■■■r■■■■■■professionalrY■■■■r■■■■■r7■■■■r■■■■■r,■

MBMM*y$Q\a■bG■■■■■■"**mbmmJ&?%■PI■!■■■■)"))C.■1■■■■ ryy■■■■■3■■■0■■■■■bii■■■■%■0G■■299T(^X-B-B■CD■■299
■RYYrzz1
-F■■BIIbjj!■,E■■■■■■2;;■■■■■@J■■■■■■2;;■■■■(//■ZJ
■■!■■■■■■■
■■$■'■■■■)J■■■■$BOOM■■
■■$■*■■■■,M■■■■""//+■■
■■$■(■■■■*K■■■■■
■■■■■■■■
■■■■■■■■
■■■■
■■■■■■■■■■■■N■■■■■]■■■■■(■■■■■r'■■■■r■■■■■N)■r■■■■■■app.extensionsr■■■■■
sqlalchemyr■■■■■r■■■■■■Enumsr■■■■■■postCommentr■■■■■■Modelr■■■■■r|■■■■r'■■■■r$■■■■■<module>r■■■■■■■■s&
```

58 — postComment.cpython-312.pyc

```
■■■■■*■i■■■■■■■■■■■■■■■■■■■■■■■■■■■■■^■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■■■G■
■■■■■■■■■■■■■■■■■■■
```

[redacted]

60 — postMedia.cpython-312.pyc

[redacted]

```
=[redacted]!<[redacted]<[redacted]
r[redacted]c[redacted]j[redacted]|[redacted]j[redacted]r[redacted]y[redacted]|[redacted]j[redacted]
[redacted]]]_[redacted]
[redacted]
[redacted]
[redacted]
[redacted]r[redacted]c[redacted]N[redacted]|[redacted]|[redacted]_[redacted]t[redacted]j[redacted]j[redacted]


[redacted]r[redacted]c[redacted]z[redacted]|[redacted]j[redacted]|[redacted]j[redacted]|[redacted]j
isDocument[redacted]post)[redacted]r2[redacted]post_idr[redacted]file_namer[redacted]r[redacted]r,[redacted]
created_at[redacted]isoformat
updated_atr*[redacted]r[redacted]r"[redacted]r$[redacted]r?[redacted]lenr3[redacted]r[redacted]s[redacted] r[redacted]to_dictz[redacted]PostMedia.to_dict?[redacted]s[redacted]
[redacted]
"[redacted]!%#[redacted]
[redacted]
r[redacted]N) [redacted]__name__
__module__[redacted]__qualname__
__tablename__r[redacted]Column[redacted]Integerr2[redacted]
ForeignKeyr@[redacted]LargeBinary[redacted]data[redacted]Stringr[redacted]rA[redacted]r[redacted]Textr,[redacted]DateTimer[redacted]utcnowrB[redacted]rD[redacted]

MBMM*y$Q\a[redacted]bG[redacted]299R^^[redacted]$D[redacted]299YRYYs^[redacted],L[redacted])"))C.[redacted])I[redacted]"**[redacted]%I[redacted]bii[redacted] G[redacted]2;;[redacted]@J[redacted]
[redacted]
[redacted]
r[redacted]r[redacted]N)[redacted]r[redacted]app.extensionsr[redacted]Modelr[redacted]rU[redacted]r[redacted]r[redacted]<module>rX[redacted]s[redacted]
[redacted]N[redacted]
r[redacted]
```

61 — projectGoal.cpython-312.pyc

```
[redacted]+[redacted]iy[redacted]n[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]
[redacted])[redacted]
SQLAlchemy)[redacted]datetime[redacted]timedelta)[redacted]Enum)[redacted]db[redacted])[redacted]
GoalMilestonec[redacted]e[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j&[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]d[redacted]d[redacted]
d[redacted]d[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]e[redacted]j8[redacted]
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j,[redacted]e[redacted]j8[redacted]e[redacted]j8[redacted]
g[redacted]Z [redacted]e[redacted]j>[redacted]d[redacted]d[redacted]e[redacted]g[redacted]Z![redacted]e[redacted]j>[redacted]d[redacted]d[redacted]d[redacted]d[redacted]
project_goalsT)[redacted]primary_keyz[redacted]startups.id)[redacted]nullablez[redacted]users.idF[redacted]r[redacted])[redacted]default[redacted]active[redacted]completed[redacted]
[redacted]d[redacted]k7[redacted]r[redacted]|[redacted]j
[redacted]t[redacted]j[redacted]j[redacted]
[redacted]milestones_completed[redacted]milestones_total[redacted]progress_percentage[redacted]_check_if_on_track[redacted]is_on_track[redacted]status
complete_goalr[redacted]session[redacted]commit)[redacted]self[redacted]completed_milestones[redacted]total_milestoness[redacted]   &E:\LLMs_test\


[redacted]c[redacted]0[redacted]|[redacted]j[redacted]r[redacted]|[redacted]j[redacted]d[redacted]
```

■■j
■■■■■■■■■■■■■■■■■■■}■|■j■■■■■■■■■■■■■■■■■■■■■|■j■■■■■■■■■■■■■■■■■■■■■z
■■j
■■■■■■■■■■■■■■■■■■■}■|■d■k■■■r■|■j■■■■■■■■■■■■■■■■■■■■■d■k\■■S■|■|■z■■■d■z■■■}■|■j■■■■■■■■
created_at■daysr#■■■)■r*■■■■today■days_passed
total_days■expected_progresss■■■■   r-■■■r$■■■z■ProjectGoal._check_if_on_track<■■■s■■■■■■■■■■■4;
■■■?■■■+■+s■2■2■(:■5■■<■■■■■'■'+<■<■<r/■■■c■■■■■■■■■■■■■■■■■■■■■■■■■■■d■|■_■■■■■■■■■■d■|■_■
■■■■■■■■■■■■■■■■■■■

62 — refreshToken.cpython-312.pyc

```
■■■■■i■■■■■■■■■■■■■■■■■■■■■■■■F■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

```
    ■■■■■"■iO■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■
    ■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
    ■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■d■d■■■■■■■■■■Z■■■e■j
    ■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■d■■■■■■■■■■Z
    ■■e■j
    ■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■Z■■■e■j
    ■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

```
created_at■isoformat
updated_atr*■■■s■■■■ r,■■■■to_dictz■Startup.to_dict■■■s■■■■■■''■■II■■

■■

■■■+■+■■ZZ■%■%■■■■■■ZZ■%2■■oo■!■4■4■ ■2■2■■■■
■■■kk■■■,■,■■ZZ■■■■■2■2■4■■■■■2■2■4%■■
■■■
r.■■■N)■rA■■■)■r■■■■)8■__name__
__module__■__qualname__
__tablename__r■■■■■Column■Integerr?■■■■StringrX■■■rY■■■rZ■■■■Textr[■■■r■■■■r■■■■■idearG■■■■Lar
ForeignKeyrb■■■rc■■■rd■■■r]■■■r5■■■r'■■■■DateTimer■■■■■utcnowre■■■rg■■■■relationshipr2■■■
join_requests■startup_bookmarks
startup_tasks
project_goals■team_performance_records■growth_metrics_startup■calendar_eventsr-■■■r7■■■rB■■■rE■■■rI■
■■■r
■■■■■■■s5■■■■■■M■■■■■■2::4■0B■■299YRYYs^D5■AD■■ryy■■■■3■%■8H■■ryy■■■■3■■(H■■"))BGG■$K■■BIId<■
■RYYyryy■}h
7F■■299RZZ■■3L■■BIIbjj!■,E■■■■■2;;■■■■@J■■■■■2;;■■■(//■ZJ
■■■booo■
■■$■/■■■1G■■■$BOOM■
■■$■-■■■/M■■■(■■■(9■$
■■$■1■■■3■■■■■$BOOF■
■■$■&■■■(M■■■$BOOM■
■■$■-■■■/M■■ /r■■/@■
■■$■1■■ 3■■■■■-R__^■
■■$■.■■■0■■■■■&booo■
■■$■/■■■1O■■■■
■■■■■■■■■■'■■■■
■■■■■■■ ■■■<■■■
r.■■■r
■■■N)
r■■■■■app.extensionsr■■■■
sqlalchemyr■■■■r■■■■■Enumsr■■■■■Modelr
■■■r■■■r.■■■r,■■■■<module>r■■■■■■■s#■■■■■■■■■■■■■!■■■_■■
bhh■_■■
r.■■■
```

65 — startupBookmark.cpython-312.pyc

```
■■■■■■iV■■■■■■■■■■■■■■■■■■■■■■■■F■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■d■■■■■■■■
■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■d■■■■■■■■
■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d
■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

```
__module__■__qualname__
__tablename__r■■■■■■Column■Integerr■■■■■
ForeignKeyr ■■■r!■■■■■Stringr■■■■■■Textr■■■■■r■■■■■■DateTimer■■■■■■utcnowr"■■■■■relationship■userr#■■■

MBMM*y$Q\a■bG■■■■■■2::}r}}]Y'Wbg■hJ■■BIIibii■n■%E■■bii■■■■■(O
■"))IBIIcN
#C■■■■■■2;;■■■■■@J■■2??6+>■?D■■booi■■AG■■■■■■■■
r■■■■■r■■■■■N)■r■■■■■■app.extensionsr■■■■■■Modelr■■■■■r3■■■■r■■■■■r■■■■■■<module>r6■■■■■■■■s■■■■■■■■■■
bhh■(■
r■■■■■
```

66 — story.cpython-312.pyc

```
■■■■■\■i
■■■■■■■■■■■■■■■■■■■■■■■■■j■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■
y■)
■■■■■)■■datetime)■■db)■■Enum■■■■■)■■StoryType)■■StoryViewc■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■d■d■■■■■■■■■■d■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■d■■■■■■■■■■■■
■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■Z
■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d
■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■■■e■e■■■■■■■■■■e■j&■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j,■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j,■■■■■■■■■■■■■■■■■■■■■■■■■e■j2■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j,■■■■■■■■■■■■■■■■■■■■■■■■■e■j2■■■■■■■■■■■■■■■■■■■■■■■■■e■j2■■■■■■■■■■■■■■■■■■■
```

67 — storyView.cpython-312.pyc

```
■■■■[1■i■■■■■■■■■■■■■■■■■■■■■■■■■■F■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■
```

[redacted]e[redacted]j([redacted]e[redacted]j,[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j([redacted]e[redacted]j,[redacted]e[redacted]j,[redacted]
d[redacted]e[redacted]g[redacted]Z[redacted]e[redacted]j2[redacted]d[redacted]d[redacted]e[redacted]g[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z d[redacted]Z
Suggestion[redacted]suggestionsT)[redacted]primary_keyz[redacted]ideas.id[redacted]CASCADE)[redacted]ondeleteF)[redacted]nullablez[redacted]users.idd[redacted])[redacted]defau
new_statuss[redacted] %E:\LLMs_test\app\models\suggestion.py
update_statusz[redacted]Suggestion.update_status[redacted]s[redacted]&z[redacted]2[redacted]

[redacted]c[redacted]&[redacted]|[redacted]j[redacted]d[redacted]y[redacted])[redacted]z[redacted]Approve sug
[redacted]r[redacted]c[redacted]<[redacted]|[redacted]j[redacted]t[redacted]j[redacted]
is_pendingz[redacted]Suggestion.is_pending8[redacted]s[redacted]{{[redacted].[redacted]6[redacted]6[redacted]6[redacted]6r[redacted]c[redacted]<[redacted]|[redacted]j[redacted]
[redacted]|[redacted]j
[redacted]|[redacted]j[redacted]r[redacted]|[redacted]j[redacted]j[redacted]
isApproved
isRejectedr2[redacted])[redacted]r=[redacted]idea_idrC[redacted]author_idr*[redacted]r+[redacted]r,[redacted]rD[redacted]profile_picturer[redacted]value
created_at[redacted]isoformat
updated_atr6[redacted]r8[redacted]r;[redacted]r3[redacted]r"[redacted]s[redacted] r[redacted]to_dictz[redacted]Suggestion.to_dictD[redacted]s[redacted]''[redacted]ll[redacted]|
[redacted]
r[redacted]N)%[redacted]__name__
__module__[redacted]__qualname__
__tablename__r[redacted]Column[redacted]Integerr=[redacted]
ForeignKeyrJ[redacted]TextrC[redacted]rK[redacted]Stringr*[redacted]r+[redacted]r[redacted]r[redacted]r5[redacted]r[redacted]DateTimer[redacted]utcnowrN[redacted]

MBMM*y$Q\a[redacted]bG[redacted]bii[redacted]%[redacted]0G[redacted]"**mbmmJ&?%[redacted]PI[redacted]![redacted])"))C.[redacted]1[redacted] ryy[redacted]3[redacted]0[redacted]
[redacted]RYYt[redacted],[redacted]-7G7O7O
PF[redacted]2;;[redacted]@J[redacted]2;;[redacted](//[redacted]ZJ[redacted]2??6=[redacted]y[redacted]QD
[redacted]R__V])[redacted]
UF[redacted]
[redacted]'[redacted]'[redacted]C[redacted]7[redacted]8[redacted]8[redacted]
r[redacted]r[redacted]N)[redacted]r[redacted]app.extensionsr[redacted]
sqlalchemyr[redacted]Enumsr[redacted]Modelr[redacted]r^[redacted]r[redacted]r[redacted]<module>rc[redacted]s#[redacted]#[redacted]Q[redacted]
[redacted]Q[redacted]
r[redacted]

69 — task.cpython-312.pyc

[redacted]9][redacted]i[redacted]f[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]Z[redacted]m[redacted]Z[redacted]d[redacted]d[redacted]l[redacted]m[redacted]
y[redacted])[redacted])[redacted]
SQLAlchemy)[redacted]datetime[redacted]timedelta)[redacted]Enum[redacted]JSON)[redacted]dbc[redacted]\[redacted]e[redacted]Z[redacted]d[redacted]Z[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]d[redacted]d
d[redacted]d
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]d
d[redacted]d[redacted]d[redacted]d
[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]g[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]g[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j*[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j*[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j0[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j0[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j[redacted]e[redacted]j[redacted]d[redacted]d[redacted]
[redacted]e[redacted]j:[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j[redacted]d[redacted]Z[redacted]e[redacted]j
[redacted]e[redacted]j*[redacted]e jB[redacted]Z"[redacted]e[redacted]j
[redacted]e[redacted]j*[redacted]e jB[redacted]e jB[redacted]
new_statuss[redacted] [redacted]E:\LLMs_test\app\models\task.py
update_statusz[redacted]Task.update_status,[redacted]sT[redacted] [redacted]$"*//"3D[redacted]'*D[redacted]$[redacted]![redacted]![redacted]#
[redacted]=

(')D■$■

■■■■■■■■■■■c■■■■■■■■■■■■■■■■■■■■■■■■■■t■■■■■■■■■■d■t■■■■■■■■■■d■|■■■■■■■■■■■■■■■■■|■_■■■■■■■
■■■■■■■■■■■■■■■■■■■■■j
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■y■)■z■Update progress percentager■■■■■r■■■■■r■■■■■r■■■■■N)■■max■minr
percentages■■■■■ r$■■■■■update_progressz■Task.update_progress7■■■■sS■■■■■#&q#c:*>#?■■ ■■■■■■■■■■■■{■
■!^■■■■■■■}■-■

■■■■■■■r&■■■■c■■■■■■■■■■■■■■■■■■■■$■■■■■|■j■■■■■■■■■■■■■■■■■■■■■r+|■j■■■■■■■■■■■■■■■■■■■■■■rl
is_on_timer■■■■■r■■■■■r■■■■■r■■■■■r ■■■■r!■■■■■r"■■■■s■■■■■ r$■■■■r■■■■■z■Task.check_if_on_time@■■■■s^■■
■]]x■■■■04==■@■#DO■#DK■

■■■■■■■r&■■■■c■■■■■■■■■■■■■■■■■■■■■N■■■■■|■|■_■■■■■■■■■■t■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■■■j■■■

■■■■■■■r&■■■■c■■■■■■■■■■■■■■■■■■■

```
■D■-■-■■■
■■
■4■+■+■■■
■■
■$//■■■
 ■
■$//!■■
"■
"4#;#;#■■
$■■■//■+■■//■3■3■5■■//■3■3■5
■■■■#■#■■■■■(■(■+■+■!■/■/■:■:■ ■.■.■8■8■■■■+■■
■■■
2■*.3■■
■■■
r&■■■)■N)-■__name__
__module__■__qualname__
__tablename__r■■■■■Column■IntegerrG■■■
ForeignKeyr2■■■rH■■■■StringrI■■■■TextrJ■■■r■■■■rK■■■r■■■■r■■■■r5■■■r<■■■■DateTimer-■■■r■■■■■
■■■r
■■■■■■■s"■■■■■■M■■■■■2::4■0B■■bii■

MBMM*$=■■NG■■■■■2::}r}}]'Cd■SJ■■BIIibii■nu■5E■■"))BGG■$K■■ryy■eXv■6■■IH
■RYYtG]K■■KU\
]F■■299T2■&D
■RYYtR
(F■■ryy■■■t■4H■■RYYr{{T■:N■■bii■■■4■8O■■299RXX■■5L■■"))BJJ
■

j(AD■QK■■■■■2::}r}}Z'@5■QJ■■■■■2::t■4J■#"))BJJ■■:■■■■■■■2;;■■■■@J■■■■■2;;■■■(//■ZJ■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■
r&■■■r
■■■N)■■flask_sqlalchemyr■■■■r■■■■r■■■■
sqlalchemyr■■■■r■■■■■app.extensionsr■■■■■Modelr
■■■rd■■■r&■■■r$■■■■<module>ri■■■■■■■s#■■■■■■■■■'■(■!■■■B■■
288■B■■
r&■■■■
```

70 — teamMember.cpython-312.pyc

```
■■■■■if■■■■■■■■■■■■■■■■■■■■■■■■F■■■■d■d■l■m■Z■■■d■d■l■m■Z■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■
```

```
■RYYtR
(F■■■■■2??6>■:D■■■■■
■■■■■■■ ■■■■
r■■■■■r■■■■■N)■■app.extensionsr■■■■■
sqlalchemyr■■■■■■Modelr■■■■■r2■■■■r■■■■■r■■■■■■<module>r6■■■■■■■■s■■■■■■■■■■■■■■'■
■■■■■'■
r■■■■■
```

71 — teamPerformance.cpython-312.pyc

```
■■■■■■■io
■■■■■■■■■■■■■■■■■■■■■■■■■b■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■■■G■d■■■d■
SQLAlchemy)■■datetime■timedelta)■■Enum)■■dbc■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■
```

72 — user.cpython-312.pyc

```
■■■■■■■i!T■■■■■■■■■■■■■■■■■■■■■■■■■■■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■m■Z■■■d■d■l■m■Z■■■d■d■l■m■Z
Z
m■Z■■■d■d■l■m
Z
m■Z■■■■■■G■d■■d■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■y■)
■■■■■)■■datetime■timedelta)■■Enum■JSON)■■db■■■■■)■
UserStatus■Privacy■Theme■EmailDigest)■■generate_password_hash■check_password_hashc■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■■■Z
■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■d■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■e■■■■■■■■■e■j&■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d
■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j0■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j4■■■■■■■■■■■■■■■■■■■■d
■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j4■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■
```

$
$■
-■ D■■"'■■■■"//■+■■■G■■■-■-dgg}■E■

■■■■■■■■■■■pointsc■■■■■■■■■■■■■■■■■■■■■j■■■■■|■x■j■■■■■■■■■■■■■■■■■■■■■|■z
■■c■_■■■■■■■■■■t■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■y■)■z■Add XP
add_xp_pointsz■User.add_xp_points"■■■s ■■■■■■■■■&■ ■■

■■■■■■■rp■■■■c■■■■■■■■■■■■■■■■■■■■■N■■■■■d■|■_■■■■■■■■■■t■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■■■j■■■

■■■■■■■rp■■■■■profile_datac■■■■■■■■■■■■■■■■■■■■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r■|■d■■■■■■|■_■■■■■■■■■■|■d■■■■■■|■_■■■■■■■■■■t■■■■■■■■■■j■■■■■■■■
pref_timezoner■■■■■rj■■■■rk■■■■)■rl■■■■ry■■■■s■■■■■ rn■■■■■update_profilez■User.update_profile,■■■■s■■■

■■■■■■■rp■■■■■preferences_datac■■■■■■■■■■■■■■■■■■■■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r■t■■■■■■■■■■|■d■■■■■■■■■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r
|■d■■■■■■|■_■■■■■■■■■■d■|■v■r■t
■■■■■■■■■|■d■■■■■■■■■■■■■■|■_■■■■■■■■■■t■■■■■■■■■■j■■■■■■■■■■■■■■■■■■■

[redacted]
rp[redacted]c[redacted]d[redacted]}[redacted]|[redacted]t[redacted]d[redacted]|[redacted]j[redacted]D[redacted]
[redacted]}[redacted]|[redacted]t[redacted]d[redacted]|[redacted]j[redacted]D[redacted]z
[redacted]}[redacted]|[redacted]t[redacted]d[redacted]|[redacted]j[redacted]D[redacted]z
[redacted]}[redacted]|[redacted]S[redacted])[redacted]z0Calculate total likes received on user's contentr[redacted]c[redacted]3[redacted]4[redacted]K[redacted]
user_posts)[redacted]rl[redacted]total_likess[redacted] rn[redacted]r[redacted]z$User._calculate_total_likes_received[redacted]sZ[redacted]
[redacted]j[redacted]z[redacted]|[redacted]j[redacted]j[redacted]
activitiess[redacted] rn[redacted]r[redacted]z User._calculate_engagement_score[redacted]s[redacted]
[redacted]JJ[redacted]"[redacted]&[redacted]&k[redacted]&[redacted]:[redacted]@[redacted]@[redacted]BQ[redacted]F[redacted]
G[redacted]
[redacted]
[redacted]
[redacted]
%
%
'$*A*A*G*G*I
IDLcLcLiLiLk
kor[redacted]r[redacted]
s[redacted]
[redacted]0[redacted]0[redacted]2S[redacted]8[redacted]
9[redacted]:[redacted]?C[redacted](rp[redacted]c[redacted]d[redacted]d[redacted]l[redacted]m[redacted]}[redacted]d[redacted]d[redacted]l[redacted]m[redacted]}[redacted]d[redacted]d[redacted]l[redacted]m[redacted]}[redacted]g[redacted]}[redacted]
|[redacted]j[redacted]|[redacted]j"[redacted]|[redacted]j[redacted]d[redacted]
[redacted]d[redacted]|[redacted]d[redacted]|[redacted]S[redacted])[redacted]z/Get user's recent activity across all platformsr[redacted])[redacted]r?[redacted])[redacted]r+[redacted])[redacted]
[redacted]a[redacted]n[redacted]rp[redacted]T)[redacted]key[redacted]reverseN)[redacted]models.taskr?[redacted]models.idear+[redacted]models.ideaCommentr4[redacted]r[redacted]or
created_at[redacted]desc[redacted]limit[redacted]all[redacted]appendr[redacted]to_dictr[redacted]r[redacted]completed_dater[redacted]sort)[redacted]rl[redacted]r
[redacted]r?[redacted]r+[redacted]r4[redacted]r[redacted]r[redacted]task[redacted]comments[redacted] rn[redacted]get_recent_activityz[redacted]User.get_recent_
[redacted]JJ[redacted]'[redacted]'[redacted](<(<(>[redacted]?[redacted]E[redacted]Ea[redacted]H[redacted]L[redacted]L[redacted]N[redacted]D[redacted]&[redacted]!__[redacted]
[redacted]$[redacted]$[redacted].[redacted].k[redacted].[redacted]B[redacted]K[redacted]KDL_L_LdLdLf[redacted]g[redacted]m[redacted]mno[redacted]p[redacted]t[redacted]t[redacted]v[redacted]D[redacted](([redacted]!([redacted]0[redacted]0[redacted]
[redacted])[redacted])[redacted]2[redacted]2;3I3I3N3N3P[redacted]Q[redacted]W[redacted]WXY[redacted]Z[redacted]^[redacted]^[redacted]`[redacted]G[redacted]&[redacted]*[redacted]$[redacted]/[redacted]/[redacted])[redacted]
[redacted]4d[redacted]C[redacted]&5[redacted]![redacted]!rp[redacted]c[redacted]i[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|
[redacted]r[redacted]|[redacted]j
[redacted]j
[redacted]n[redacted]d[redacted]d[redacted]|[redacted]j[redacted]j[redacted]d[redacted]|[redacted]j[redacted]
|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]r[redacted]|[redacted]j[redacted]j
[redacted]n[redacted]d[redacted]d[redacted]|[redacted]j[redacted]d
|[redacted]j[redacted]d[redacted]|[redacted]j[redacted]d[redacted]|[redacted]j [redacted]|[redacted]j"[redacted]
[redacted]|[redacted]jR[redacted]j
[redacted]|[redacted]jU[redacted]|[redacted]jW[redacted]
total_revenue[redacted]satisfaction_percentage[redacted]active_startups_count[redacted]profile)[redacted]r{[redacted]r|[redacted]r}[redacted]r~[redacted]r[redacted]r[redacted]
statistics[redacted]recentActivity)/ri[redacted]r[redacted]r[redacted]r[redacted]rv[redacted]rf[redacted]isoformatr[redacted]valuer[redacted]rs[redacted]rb[redacted]re[redacted]
updated_atr[redacted]r[redacted]r[redacted]r[redacted]r[redacted])[redacted]rl[redacted]include_password[redacted]include_statistics[redacted]include_recent_activity
[redacted]$''[redacted]3[redacted]
[redacted]3[redacted]
[redacted]
[redacted]3[redacted]
[redacted]
[redacted]TZZ[redacted]3[redacted]

[redacted]
[redacted]t[redacted]5[redacted]5[redacted]3[redacted]
[redacted]
[redacted]2[redacted]2[redacted]4T
3[redacted]
[redacted]
[redacted]dkk[redacted]'[redacted]'[redacted]3[redacted]
[redacted]
[redacted]DII[redacted]3[redacted]
[redacted]
[redacted]3[redacted]
[redacted]
[redacted]4[redacted]+[redacted]+[redacted]3[redacted]
[redacted]
![redacted]I`I`$"9"9"C"C"Efj[redacted]3[redacted]
[redacted]
[redacted]T[redacted]/[redacted]/[redacted]3[redacted]
[redacted]
&t'C'C[redacted]3[redacted]
[redacted]
$T%?%?[redacted]3[redacted]
[redacted]
[redacted]/[redacted]/[redacted]'[redacted]'[redacted]/[redacted]/[redacted]#[redacted]8[redacted]8[redacted]>B[redacted]/[redacted]/[redacted])[redacted])[redacted] [redacted]1[redacted]1[redacted]3[redacted]
0[redacted]

■&*&C&C%)%A%A■■■,■,■2■2■ ■.■.■ ■.■.■■■■■.■.
■■■13■
@■■
#■#■6■6■ ■0■0"&"<"< $ 8 8■!■1■1■"■4■4"&"<"<■#■6■6■<■<■#■=■=■!■9■9■■■5■5■■■■■■%■A■3■
^■■■■■■■2■2■4■■■■■2■2■4■■■*■*■,■■■)■)■+e■3■
■j■■■■■#}}D■■■■■!%!4!4!6D■■■■"%)%=%=%?D■!■"■■■rp■■■N)■r■■■■)■FFF){■__name__
__module__■__qualname__
__tablename__r■■■■■Column■Integerri■■■■Stringr■■■r■■■r■■■■r■■■■Booleanrv■■■■DateTimerf■■■r■■■■r
■■■■publicr■■■r■■■r■■■r■■■■■lightr■■■r■■■r■■■r■■■r■■■r■■■r■■■r■■■r■■■r■■■■■weeklyr■■■r■■■r■■■
notificationsr■■■r■■■r■■■■storiesr■■■■authored_postsr■■■r■■■r■■■
post_likes■resource_likes■startup_bookmarks■knowledge_bookmarks■idea_bookmarksr■■■
created_tasks■assigned_tasks
project_goals■startup_memberships
join_requestsr■■■■growth_metrics
sent_messages■created_conversationsr#■■■
conversations■resource_views■story_views■resource_downloads■suggestions■calendar_events■goal_milestone
■RYYtJ■'■1B1B
CF■■299YRYYr]H■5D■■■■■"**a■0I■■"))BJJ■■2K■"■■■277■+■■■■■■BIIbhh■■4M■'bii■■■%■@■■■%BIIbjj!■<■■■

D■A■■■■299T']GNN■CL■■BIIibii■mT■:M■■BIIibii■mU■;M■■■■■4■;■■■■<J■■■#■■■2::t■<■■■■bii■

D■9O■%BIIbjj$■?■■■#"))BJJ■■=■■■■bii■

D■9O■!■■■"**d■;■■■%BIIbjj$■?■■■"■■■4■#4k>P>P■Q■■)■■■"**e D■■■'bii■■■■!■g■F■■■%BIIibii■lG■D■■■
■■%R__^■■
■■$■+■■■-N■■■$BOON■$
■■$■+■■■-M■■■&book■"
■■*■$■■■&O■■
■BOOF■■
■■&■$■■
&E■■■■r■■y■!
■■)■$■■■&H■■■■boog■■
■■&■$■■■&G■■■!■■■■■■
■■#■$■■■&J■■■%R__V■■
■■%■■■'N■■■$BOOM■
■■,■$■■■&M■■■)■■■);■
■■1■$■■■&■■■■■)■■■■■
■■,■$■■■&■■■■■!■■■■■■
■■$■'■■■)J■■■%R__^■■
■■$■+■■■-N■■■(■■■(9■
■■$■.■■■0■■■■■*"//*=■
■■$■0■■■2■■■■■%R__^■
■■$■+■■■-N■■■""//&■■
■■#■$■■■&K■■■$BOOF■■
■■&■■■(M
■■%R__V■■
■■'■■■)N
■■$BOOM■■
■■*■$■■■&M■■■*"///■■
■■,■$■■■&■■■■■$BOOM■■
■■*■$■■■&M■■■(■■■(9■#
■■.■$■■■&■■■■■%R__^■■
■■$■+■■■-N■■■$BOOM■
■■,■$■■■&M■■■,BOO,>■&
■■5■$■■■&■■■■■$BOO$6■-■■■

■<9■M■■■O■M
■■%R__^■■
■■$■+■■■-N■■■""//+■■
■■$■(■■■*K■■■)■■■);■
■■$■/■■■1■■■■■""//,■#
■■$■+■■■-K■■■&booo■■
■■$■,■■■.O■■■&booo■!
■■$■,■■■.O■■■■"■■■C■■■■
■■■
■■■4■■■■(■■■4■■■■"■■■d■■■■<■■■S■■■■
■■<s■■■<t■■■<■■■0■■■■
■■■
■■5■■■'

■
■■■■■■■)■!■"F■?■■rp■■■r■■■■N)■r■■■■r■■■■

sqlalchemyr■■■■r■■■■■■app.extensionsr■■■■■■Enumsr■■■■■r
■■■■r■■■■■r■■■■■■werkzeug.securityr
■■■■r■■■■■■Modelr■■■■■r■■■■■rp■■■■rn■■■■■<module>r_■■■■■■■■s)■■■■■■■■■■(■!■■■■:■:■I■K■■■288■K■■■■rp■■

73 — userAchievement.cpython-312.pyc

■■■■■H■i
■■■■■■■■■■■■■■■■■■■■■■■■b■■■■■d■d■l■m■Z■■■■d■d■l■m■Z■m■Z■■■■d■d■l■m■Z■■■■d■d■l■m■Z■■■■■■G■d■■■d■
SQLAlchemy)■■datetime■timedelta)■■Enum)■■dbc■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■e■Z■d■Z■d■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■Z■■■■e■j
■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■■■■■e■j■■■■■■■■■■■■■■■■■■■■■■d■■■■■■■■■■d■■■■■■■■■■■■■■■■■■■